

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования

«Новосибирский национальный исследовательский государственный
университет»

(Новосибирский государственный университет, НГУ)

Структурное подразделение Новосибирского государственного университета –

Высший колледж информатики Университета (ВКИ НГУ)

КАФЕДРА ИНФОРМАТИКИ

**«Разработка программного комплекса "Garden Nook" для
автоматизации кофейни здорового питания»**

Квалификация программист

Руководитель
м. н. с. ИВМиМГ СО РАН

Ткачев К. В.
«___»_____ 2026 г.

Студент 4 курса
гр. 2207в

Косухин С. Д.
«___»_____ 2026 г.

Новосибирск

2026

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ, УСЛОВНЫХ ОБОЗНАЧЕНИЙ И ТЕРМИНОВ	5
ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ ВКР	8
1.1 Бизнес-требования	8
1.2 Пользовательские требования	8
1.3 Системные требования.....	10
1.4 Требования к пользовательскому интерфейсу	11
1.5 План-график выполнения ВКР	11
2 АНАЛИЗ ТРЕБОВАНИЙ И ОПРЕДЕЛЕНИЕ СПЕЦИФИКАЦИЙ	13
2.1 Описание предметной области задачи ВКР	13
2.1.1 Информационные объекты предметной области и взаимосвязи между ними ...	13
2.1.2 Информационные потребности пользователей	14
2.1.3 Методы работы с информационными объектами предметной области	16
2.1.3.1 Способы хранения информации об объектах предметной области	16
2.1.3.2 Алгоритм расчета пищевой ценности и себестоимости позиций меню	16
2.1.3.3 Способы интерпретации и визуального представления информации	18
2.1.3.4 Технологии получения и передачи информации.....	19
2.1.4 Обзор существующих программных реализаций решения задачи	20
2.1.5 Концептуальное обоснование разработки	23
2.2 Классы и характеристики пользователей.....	23
2.3 Функциональные требования.....	24
2.3.1 Определение функциональных возможностей	24
2.3.2 Описание прецедентов	30
2.4 Нефункциональные требования	31
2.4.1 Производительность и масштабируемость.....	31
2.4.2 Надежность и доступность.....	32
2.4.3 Безопасность	32
2.4.4 Переносимость и совместимость	32
2.4.5 Сопровождаемость и расширяемость.....	33
2.4.6 Удобство использования и локализация	33
3 ВЫБОР ПРОГРАММНЫХ СРЕД И СРЕДСТВ РАЗРАБОТКИ	34
3.1 Сравнительный анализ имеющихся возможностей по выбору средств разработки .	34
3.2 Характеристика выбранных программных сред и средств	36
4 АЛГОРИТМЫ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ.....	38

4.1 Этапы реализации	38
4.2 Пользовательский интерфейс	39
4.2.1 Описание взаимодействия с пользователем	39
4.2.2 Определение операций пользователей и составление функциональных блоков	45
4.2.3 Проектирование структуры экранов и схемы навигации	47
4.2.4 Разработка дизайна интерфейса	49
4.3 Входные, выходные и промежуточные данные	52
4.3.1 Входные данные	52
4.3.2 Выходные данные	52
4.3.3 Промежуточные данные	53
4.4 Реализация используемых методов хранения, обработки и передачи информации об объектах предметной области	53
4.4.1 Методы хранения данных	53
4.4.2 Алгоритмы реализации используемых формул расчета	62
4.4.2.1 Алгоритм определения рабочего веса компонента технологической карты	62
4.4.2.2 Алгоритм расчета складских остатков	63
4.4.2.3 Алгоритм расчета заказа	65
4.4.3 Алгоритмы применения методов графического анализа данных	67
4.4.4 Алгоритмы использования технологий передачи данных	69
4.5 Описание архитектурного решения	71
4.5.1 Структурная организация программной системы	71
4.5.2 Архитектура программного кода	73
5 ТЕСТИРОВАНИЕ И ОПТИМИЗАЦИЯ	80
5.1 План тестирования	80
5.2 Результаты тестирования и оптимизация	81
5.2.1 Функциональное тестирование	81
5.2.2 Тестирование пользовательского интерфейса	83
5.2.3 Тестирование API-взаимодействия	85
5.2.4 Тестирование данных и бизнес-правил	86
5.2.5 Unit-тестирование	87
5.2.6 Тестирование безопасности	88
5.2.7 Базовое тестирование производительности и оптимизация	89
6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	90
6.1 Руководство для клиента	90
6.2 Руководство для повара	93
6.3 Руководство для бариста-кассира	96

6.4 Руководство для администратора	98
6.5 Обработка ошибок и завершение работы	100
ЗАКЛЮЧЕНИЕ	101
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	103
ПРИЛОЖЕНИЯ.....	105
Приложение А	105
Приложение Б	107
Приложение В.....	108
Приложение Г	114
Приложение Д.....	145
Приложение Е	145

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ, УСЛОВНЫХ ОБОЗНАЧЕНИЙ И ТЕРМИНОВ

Point of Sale (POS) – программно-аппаратный комплекс для регистрации заказов и проведения финансовых операций в кофейне. Обеспечивает приём оплаты, печать чеков и учёт продаж в реальном времени.

Customer Relationship Management (CRM) – система для управления взаимодействием с клиентами: хранение истории посещений, предпочтений, бонусов и формирование персональных предложений.

Application Programming Interface (API) – интерфейс программного взаимодействия, используемый для связи веб-приложения кофейни с сервером и базой данных.

Stock Keeping Unit (SKU) – уникальный идентификатор позиции товара (ингредиента или блюда), применяемый при учёте складских остатков.

Loyalty Program (LP) – система лояльности, позволяющая клиентам накапливать бонусы за покупки и обменивать их на скидки или специальные предложения.

JSON (JavaScript Object Notation) – формат структурированных данных, применяемый для обмена информацией между веб-приложением и сервером.

UI (User Interface) – пользовательский интерфейс, представляющий собой визуальную часть цифрового продукта (кнопки, экраны, шрифты, иконки), обеспечивающую взаимодействие между пользователем и компьютерной системой. Он отвечает за визуальное оформление, а не за логику работы.

KPI (Key Performance Indicator) – числовые метрики, измеряющие результативность и эффективность работы компании, подразделения или сотрудника в достижении поставленных целей.

ORM (Object-Relational Mapping) – технология программирования, связывающая базы данных с концепциями объектно-ориентированных языков.

ВВЕДЕНИЕ

Современная индустрия общественного питания развивается в условиях высокой конкуренции, цифровизации бизнес-процессов и роста требований клиентов. Пользователь ожидает не просто получения продукта, а удобного и быстрого взаимодействия с заведением: возможности оформления предварительного заказа, прозрачной информации о составе и пищевой ценности позиций, персонализированных рекомендаций, участия в программах лояльности. Особенно актуально это для заведений, ориентированных на здоровое питание, где осознанность выбора является базовой ценностью, а состав блюд и свежесть ингредиентов напрямую влияют на лояльность клиента. [1]

Кроме того, развитие цифровых сервисов в сфере общественного питания формирует у клиентов новый тип потребительского поведения, основанный на принципах скорости, персонализации и автономности при принятии решения о покупке. Клиент хочет совершать покупку самостоятельно – без ожидания, без очереди, без уточнения состава у персонала. Это становится возможным благодаря интеграции программного обеспечения, где данные о меню, ингредиентах и наличии товаров находятся в единой системе. Для заведения это означает снижение нагрузки на сотрудников и сокращение человеческих ошибок: бариста перестает выполнять роль «справочного центра» и концентрируется на качестве приготовления напитка.

В сегменте здорового питания такие цифровые сервисы особенно востребованы. Покупатели этой категории чаще анализируют состав блюд и напитков, интересуются пищевой ценностью, наличием аллергенов, заменой ингредиентов (сахар → сироп топинамбура, молоко → растительное и т. д.). Однако традиционные POS-системы и CRM-сервисы не учитывают необходимость отображения информации о БЖУ, составе и возможных модификациях. Это создаёт информационный разрыв между ожиданиями клиентов и возможностями бизнеса: клиент спрашивает персонал о составе, бариста отвлекается, за ним задерживается очередь, растёт время ожидания. Автоматизация таких процессов повышает качество обслуживания и улучшает клиентский опыт.

На рынке отсутствуют комплексные решения, одновременно покрывающие и кассовые процессы, и CRM, и складской учет с особенностями здорового питания. Это приводит к необходимости использования нескольких разрозненных программ, увеличивает трудозатраты и приводит к ошибкам при работе персонала.

Отдельного внимания заслуживает проблема отсутствия аналитики. Без аналитики невозможно оценить, какие позиции являются самыми прибыльными, какие позиции следует убрать из меню, как распределяется нагрузка на сотрудников и в какое время наблюдается пик заказов. Внедрение аналитического модуля позволяет принимать управленческие решения,

основанные на анализе фактических данных: корректировать меню, прогнозировать закупки и снижать издержки.

Разработка программного комплекса Garden Nook направлена на устранение перечисленных проблем. Система объединяет оперативный уровень (приём заказов), управленческий уровень (CRM + складской учёт) и стратегический уровень (аналитика и отчётность), обеспечивая комплексный подход к автоматизации. Программное решение позволяет владельцу кофейни видеть полную картину: от количества заказов и их состава до динамики продаж по типу напитков и эффективности акций.

Актуальность разработки программного обеспечения для кофейни здорового питания Garden Nook заключается в необходимости объединить в одной системе: прием заказов (POS), учет меню и пищевой ценности, управление складом, анализ продаж и CRM. Использование программной системы позволит минимизировать человеческие ошибки, сократить время обслуживания на кассе, повысить прозрачность процессов и обеспечить клиенту современный цифровой сервис. [2]

Таким образом, автоматизация бизнес-процессов кофейни, ориентированной на здоровое питание, является не просто оптимизацией, а необходимостью для удержания конкурентного преимущества. Разработка единой программной системы обеспечит повышение качества сервиса, рост повторных покупок и устойчивое развитие бизнеса в условиях рынка, ориентированного на цифровизацию.

Цель работы – разработка программного комплекса для автоматизации бизнес-процессов кофейни Garden Nook, включающего POS-модуль, CRM-модуль, систему лояльности, учет ингредиентов и аналитическую подсистему.

Для достижения цели необходимо выполнить следующие задачи:

- исследовать предметную область и существующие аналогичные решения;
- определить функциональные, пользовательские и бизнес-требования;
- спроектировать архитектуру программного комплекса и базу данных;
- разработать интерфейсы POS-системы и клиентского приложения;
- реализовать модули обработки заказов, складского учета и аналитики;
- провести тестирование системы и оценку достигнутых показателей.

1 ПОСТАНОВКА ЗАДАЧИ ВКР

1.1 Бизнес-требования

Бизнес-требования определяют, какие задачи должна решить система на уровне бизнеса, и какой ожидается результат внедрения. Они отражают потребности владельца кофейни и описывают цели, ради которых создаётся программный продукт.

Ключевые бизнес-требования к системе:

- Повышение скорости обслуживания клиентов. Система должна обеспечить сокращение времени оформления заказа за счёт автоматизации процессов через POS-интерфейс и клиентское веб-приложение. Ускорение обслуживания позволит снизить нагрузку на персонал, уменьшить очереди и повысить пропускную способность заведения.
- Повышение прозрачности и ценности продукта для клиента. Система должна предоставлять клиенту полную информацию о составе блюд и напитков, включая калорийность, БЖУ и возможные аллергены. Это позволит поддерживать концепцию здорового питания, формировать доверие клиентов и повышать их лояльность.
- Снижение потерь и оптимизация складских расходов. Система должна обеспечивать автоматизированный контроль остатков ингредиентов и сроков годности с автоматическим списанием используемых продуктов. Это позволит сократить количество списаний, минимизировать потери и повысить эффективность управления запасами.
- Увеличение количества повторных продаж. Система должна обеспечивать ведение клиентской базы, хранение истории заказов и поддержку программы лояльности. Использование CRM-механизмов позволит повысить вовлеченность клиентов и стимулировать рост повторных покупок.
- Поддержка управленческих решений на основе данных. Система должна обеспечивать сбор и анализ информации о продажах, популярности позиций, сезонности спроса и поведении клиентов. Использование аналитических отчетов позволит оптимизировать меню, корректировать маркетинговые активности и повышать рентабельность бизнеса.

1.2 Пользовательские требования

Пользовательские требования описывают, какой функционал ожидают пользователи при работе в системе и какие сценарии работы поддерживаются. Они определяют ключевые требования к программному продукту и его структуру, ориентированную на разделение функциональности по ролевому принципу. Требования формируются для разных ролей: клиент, сотрудник, администратор:

1. Администраторы должны иметь возможность:

- Авторизоваться в системе;
- просматривать, добавлять, удалять и редактировать позиции меню с указанием калорийности, БЖУ и состава;
- просматривать, добавлять, удалять и редактировать категории товаров;
- контролировать количество сырья и полуфабрикатов на складе;
- выполнять операции пополнения остатков;
- просматривать, добавлять, удалять и редактировать категории сырья и полуфабрикатов;
- просматривать, добавлять, удалять и редактировать технологические карты полуфабрикатов;
- просматривать, добавлять, удалять и редактировать технологические карты готовых товаров;
- просматривать и управлять данными о клиентах: просматривать историю заказов и изменять категорию;
- просматривать, добавлять, удалять и редактировать заказы;
- просматривать историю заказов;
- помещать и удалять позиции из стоп-листа;
- создавать задачи по заготовкам и фиксировать выполнение заготовок;
- формировать, просматривать и удалять акты списания;
- просматривать аналитические отчёты, АВС-анализ и данные инвентаризации;
- просматривать, добавлять, удалять редактировать сотрудников заведения
- управлять правами доступа сотрудников;
- управлять программой лояльности.

2. Сотрудники должны иметь возможность:

- Авторизоваться в системе;
- формировать заказы и изменять состав заказа;
- просматривать информацию о заказах, отмечать готовность позиций и изменять статус заказа;
- просматривать состав, КБЖУ и доступность позиций меню;
- просматривать технологические карты блюд, напитков, полуфабрикатов и готовых товаров;
- создавать задачи по заготовкам
- указывать дату приготовления полуфабриката;
- помещать и удалять позиции из стоп-листа;
- формировать и просматривать акты списания.

3. Клиенты должны иметь возможность:

- Авторизоваться и регистрироваться в приложении;
- просматривать актуальное меню с подробной пищевой информацией и доступностью каждой позиции;
- формировать корзину с выбором количества, добавок и модификаторов;
- оформлять заказ с выбором типа заказа, времени самовывоза и комментарием;
- получать скидку по категории клиента в системе лояльности;
- просматривать профиль, историю заказов и отменять заказы в допустимом статусе.

1.3 Системные требования

Системные требования описывают программно-аппаратное окружение, архитектуру системы, используемые модули и принципы взаимодействия между ними:

- Модуль авторизации и разграничения доступа. Обеспечивает регистрацию клиентов, вход клиентов и сотрудников, создание защищенной сессии и предоставление функций в соответствии с ролью пользователя: администратор, повар или бариста.
- Модуль управления меню и категориями. Отвечает за хранение и редактирование блюд, напитков, добавок, модификаторов и категорий, а также за отображение состава, КБЖУ, стоимости и актуальной доступности позиций.
- Модуль заказов и клиентского взаимодействия. Обеспечивает формирование корзины, оформление заказа с выбором типа заказа, времени самовывоза, комментария и скидки, а также просмотр истории и отмену заказов в допустимом статусе.
- Производственный модуль. Поддерживает работу с очередью заказов, изменением статусов, технологическими картами, заготовками, задачами, стоп-листом и отметкой готовности отдельных позиций заказа.
- Складской модуль. Обеспечивает учет сырья и полуфабрикатов, управление их категориями, контроль остатков, пополнение склада, списание продуктов и связь складских данных с технологическими картами.
- Административно-аналитический модуль. Предоставляет инструменты управления клиентами, программой лояльности и персоналом, а также отчеты по продажам, популярности позиций, ABC-анализу и инвентаризации.

Программная система спроектирована как самостоятельное решение для автоматизации работы кофейни Garden Room и предусматривает возможность дальнейшего масштабирования на сеть заведений. Все модули взаимосвязаны и работают в единой архитектуре с централизованной базой данных и синхронизацией данных в реальном времени.

1.4 Требования к пользовательскому интерфейсу

Требования к пользовательскому интерфейсу определяют визуальный стиль, принципы взаимодействия с системой и особенности отображения данных. Навигация должна быть логична и линейна: один экран – одна задача. Все кнопки имеют понятные подписи («Добавить в заказ», «Готово», «Оплатить»). Особое внимание уделяется удобству работы на сенсорных экранах, так как POS используется сотрудниками кофейни в условиях высокой нагрузки и ограниченного времени на выполнение операций.

Интерфейс системы выполнен в спокойной природной цветовой гамме, отражающей концепцию здорового питания и натуральности. Основной фон приложения – белый (#FFFFFF) и светло-серый (#F5F5F5), для карточек используются светлые оттенки (#F5F7F1, #EDF1E5). Основной акцентный цвет – зелёный (#91A56E), для активных элементов применяется тёмно-зелёный (#606E52), а в веб-интерфейсе клиента – насыщенный зелёный (#1E5928). Ошибки и удаление выделяются красно-коричневым цветом (#742C27) и красным (#E22D2D).

Все основные текстовые элементы выполнены в графитовом (#474747) и тёмно-сером (#333333) цветах, что обеспечивает контраст и удобочитаемость на светлом фоне. Для второстепенных подписей применяются серые оттенки (#555555, #666666, #777777), а для успешных действий – зелёные оттенки (#1E5928, #2DE24B). Используются шрифты без засечек, текст выравнивается по сетке, межстрочный интервал увеличен, чтобы снизить визуальную нагрузку.

POS-интерфейс разработан с приоритетом скорости: элементы управления имеют минимальный размер 44×44 рх, что соответствует рекомендациям Apple HIG и позволяет уверенно работать пальцами на сенсорном экране. Для оформления заказа требуется не более трёх действий: выбор позиции → подтверждение → выдача чека. Категории меню отображаются плитками с реальными фотографиями блюд и напитков, а пищевые характеристики (калорийность, БЖУ, аллергены) видны сразу под названием позиции.

Веб-приложение для клиентов ориентировано на визуальный просмотр меню. Фотография позиции всегда крупная, под ней – краткая информация о составе и пищевой ценности. В интерфейсе исключены скрытые элементы: пользователь видит ключевую информацию без прокрутки.

1.5 План-график выполнения ВКР

План-график выполнения выпускной квалификационной работы определяет последовательность шагов, необходимых для достижения цели проекта, а также сроки выполнения каждого этапа. Он обеспечивает прозрачность процесса разработки и позволяет

своевременно контролировать прогресс. Разбиение на этапы способствует более точному распределению времени, выявлению критических точек и снижению рисков задержек. План строится с учетом необходимости выполнения как технической части (разработка), так и аналитико-документальной. План-график представлен в таблице 1.5.1.

Таблица 1.5.1 – План-график выполнения ВКР

№	Этап работы	Срок выполнения
1	Анализ предметной области, сбор и формализация требований	10.11.2025 – 30.11.2025
2	Разработка технического задания и архитектуры системы	01.12.2025 – 20.12.2025
3	Проектирование базы данных и API	21.12.2025 – 10.01.2026
4	Разработка административного веб-интерфейса (модули меню, заказов, склада)	11.01.2026 – 15.02.2026
5	Разработка браузерного клиентского приложения (iOS/Android или кроссплатформенного)	01.02.2026 – 01.03.2026
6	Реализация модуля аналитики и CRM	16.02.2026 – 10.03.2026
7	Интеграция с платёжными системами и push-сервисами	11.03.2026 – 20.03.2026
8	Тестирование функциональности, исправление ошибок	21.03.2026 – 10.04.2026
9	Написание пользовательской документации и инструкций	11.04.2026 – 20.04.2026
10	Подготовка к защите ВКР, демонстрация системы, оформление пояснительной записки	21.04.2026 – 25.05.2026

Данный план позволяет контролировать выполнение проекта на всех его этапах и своевременно корректировать работу при изменении требований или появлении новых данных. Четкое распределение задач по временным промежуткам снижает риск несоблюдения сроков и обеспечивает системный подход к разработке программного решения. Реализация плана гарантирует завершение проекта в намеченные сроки и достижение требуемого уровня качества программного комплекса.

2 АНАЛИЗ ТРЕБОВАНИЙ И ОПРЕДЕЛЕНИЕ СПЕЦИФИКАЦИЙ

2.1 Описание предметной области задачи ВКР

2.1.1 Информационные объекты предметной области и взаимосвязи между ними

Современная кофейня, работающая по концепции здорового питания, оперирует большим количеством данных, связанных с меню, ингредиентами и управлением заказами. Для корректной работы организации важно выделить ключевые этапы бизнес-процесса, определить информационные объекты и взаимосвязи между ними, обеспечив основу для последующей автоматизации. Анализ процесса работы кофейни позволяет структурировать его как последовательность связанных действий и определить набор данных, формируемых на каждом этапе. [3]

В рамках процесса работы управляющий формирует меню, определяя список позиций и распределяя их по категориям. Для каждой позиции указывается состав, включающий перечень ингредиентов, их массу и возможные варианты замены. После этого производится закупка сырья и подготовка полуфабрикатов, а также вычисляется пищевая ценность блюд и напитков, включая калории, белки, жиры и углеводы. Далее согласовываются технология приготовления и оформление подачи, после чего меню утверждается и становится доступным для клиентов и сотрудников.

Клиент выбирает необходимые позиции из меню и оформляет заказ в заведении или на вынос. Сформированный заказ передается сотрудникам в работу, после чего начинается приготовление позиций с одновременным автоматическим списанием ингредиентов со склада. После завершения приготовления готовый заказ передается клиенту. Информация о заказе сохраняется в аналитической подсистеме для формирования статистики продаж и контроля себестоимости. На основе полученных данных управляющий анализирует показатели деятельности заведения и принимает решения о корректировке меню и планировании дальнейших закупок.

Исходя из анализа процесса, выделяются информационные объекты предметной области:

- Объект «Позиция» представляет конечный продукт, доступный клиенту для заказа. Позиция объединяет рецептуру, технологию приготовления, данные о пищевой ценности и информацию о стоимости. Структура позиции включает: тип (блюдо, напиток или добавка), состав (перечень ингредиентов), параметры порции и возможные варианты модификации (без сахара, растительное молоко и т. д.). Именно объект «Позиция» связывает клиента, ингредиенты и производственный процесс приготовления.

- Объект «Добавка» описывает добавленные клиентом к основной позиции заказа сырье и/или полуфабрикат. Добавка содержит в себе информацию о массе продукта и цене. Объект «Добавка» осуществляет связь между основными позициями и отдельными предпочтениями клиента.
- Объект «Ингредиент» описывает единицу сырья или полуфабрикат, необходимые для приготовления позиции. Каждый ингредиент имеет свои характеристики: срок годности, массу, остаток на складе, себестоимость и пищевую ценность. Через объект «Ингредиент» осуществляется связь между складским учётом, рецептурой и процессом списания при приготовлении заказа.
- Объект «Заказ» объединяет запрос клиента, выбранные позиции и процесс приготовления. Заказ фиксирует момент оформления, состав и статус исполнения, связывая клиента и сотрудников. Он является источником данных для аналитики и списания ингредиентов. Объект «Заказ» позволяет отслеживать весь путь выполнения – от выбора позиций до передачи клиенту.
- Объект «Клиент» описывает человека, приобретающего позиции. Он содержит контактные данные, историю заказов и его категорию в системе лояльности. «Клиент» связывает заказы и систему лояльности, позволяя персонализировать обслуживание.
- Объект «Сотрудник» определяет участников процессов кофейни: бариста, повара и управляющего. Каждый сотрудник имеет свою роль и набор операций в бизнес-процессе. Через «Сотрудник» осуществляется учёт ответственности: кто принимает заказ, кто готовит, кто контролирует остатки ингредиентов.

2.1.2 Информационные потребности пользователей

Информационные потребности определяются задачами пользователей в рамках бизнес-процесса без привязки к реализации программного средства. Каждый участник процесса работает с определённым набором данных, связанным с его функциями, которые определяются его ролью в системе. Для выполнения своих функций каждый участник процесса обладает собственными потребностями, которые напрямую связаны с его обязанностями, возможностями и уровнем ответственности в рамках рассматриваемого бизнес-процесса.

Такие потребности определяют, какие данные необходимы сотруднику для корректного выполнения повседневных операций, взаимодействия с другими участниками и соблюдения установленного порядка работы. На их основе формируется перечень информации, используемой для принятия обоснованных решений, контроля результатов деятельности и своевременного выявления возможных отклонений в процессе. Для каждой роли пользователей были выделены следующие информационные потребности:

1. Для бариста/повара:

- перечень реализуемых блюд, напитков и добавок с указанием их стоимости;
- сведения о позициях меню, временно недоступных для приготовления или продажи;
- данные о текущих заказах, стадии их выполнения и особых пожеланиях клиентов;
- очередность приготовления заказов с учётом времени оформления и выдачи;
- рецептурный состав позиций меню и технологические операции их приготовления;
- сведения о наличии сырья и полуфабрикатов
- для производственного процесса;
- перечень заготовок, требующих приготовления для обеспечения работы кофейни;
- информация о расходе сырья при приготовлении заказов и выполнении производственных операций.

2. Для управляющего кофейни:

- сведения о реализуемых позициях меню, их себестоимости и отпускной цене;
- информация о позициях меню, временно исключённых из продажи;
- сведения о технологических картах, рецептурном составе и нормах расхода сырья;
- данные о сотрудниках, их должностных ролях и уровне доступа к рабочим процессам;
- сведения о наличии полуфабрикатов, используемых в производственном процессе;
- сведения о текущих остатках сырья на складе;
- информация о сырье, требующем пополнения для обеспечения бесперебойной работы кофейни;
- сведения о клиентах, их категориях и истории заказов;
- данные о предполагаемом расходе ингредиентов на основе прошлых продаж;
- сведения о востребованности позиций меню и динамике выручки за выбранные периоды.

3. Для клиента:

- сведения о доступных позициях меню и их отпускной цене;
- информация о составе выбранной позиции и наличии потенциальных аллергенов;
- данные о пищевой ценности позиции: калорийности, белках, жирах и углеводах;
- сведения о доступности позиции для заказа в текущий момент;
- информация об итоговой стоимости сформированного заказа;
- сведения о категории клиента и применяемых условиях программы лояльности;
- данные о текущем состоянии выполнения оформленного заказа.

Таким образом, каждая категория пользователей имеет доступ только к той информации, которая необходима ей в рамках бизнес-процесса.

2.1.3 Методы работы с информационными объектами предметной области

2.1.3.1 Способы хранения информации об объектах предметной области

В результате анализа предметной области было выявлено, что хранению подлежат сведения о меню кофейни, позициях блюд и напитков, ингредиентах, технологических картах, заказах, клиентах и сотрудниках. Отдельно фиксируются данные о складских остатках, стоп-листе, актах списания, категориях клиентов и скидках программы лояльности. Такой набор информации отражает ключевые процессы кофейни: прием и приготовление заказов, учет рецептов, контроль запасов, обслуживание клиентов и управленческий анализ.

Для указанной предметной области целесообразно использовать реляционную модель хранения данных. Она позволяет представить объекты в виде связанных таблиц, где каждая таблица отвечает за отдельную логическую сущность, а взаимосвязи между сущностями описываются через первичные и внешние ключи. Благодаря этому данные о рецептурах, заказах, клиентах и складских остатках хранятся согласованно и не дублируются в разных частях системы.

Реляционная модель особенно подходит для учета объектов кофейни, так как большинство данных имеют устойчивую структуру и выраженные связи. Один заказ может включать несколько позиций, одна позиция меню может состоять из набора ингредиентов, а один ингредиент может использоваться в нескольких технологических картах. Представление таких отношений в виде таблиц упрощает контроль целостности и снижает вероятность логических ошибок при добавлении, изменении и удалении информации.

Использование реляционного подхода также обеспечивает основу для последующей обработки данных: расчета себестоимости и пищевой ценности, формирования истории заказов, анализа продаж и контроля складских остатков.

2.1.3.2 Алгоритм расчета пищевой ценности и себестоимости позиций меню

В предприятиях общественного питания для описания блюд и напитков применяются расчеты пищевой ценности, себестоимости и конечной цены позиции.

Фактический вклад ингредиента в калорийность позиции определяется пропорционально его массе в рецептуре. Если значение калорийности ингредиента задано на 100 г, то для массы m используется формула:

$$\text{Калории_ингр} = (\text{Ккал_ингр} / 100) \cdot m, \quad (1)$$

где Ккал_ингр – калорийность ингредиента на 100 г; m – масса ингредиента в рецептуре, г.

Аналогично рассчитываются белки, жиры и углеводы. Общая пищевая ценность позиции определяется суммированием вкладов всех ингредиентов:

$$\text{КБЖУ_позиции} = \sum (\text{КБЖУ_ингр_i} \cdot m_i / 100), \quad (2)$$

где m_i – масса i -го ингредиента, г; КБЖУ_ингр_i – значение соответствующего показателя для i -го ингредиента на 100 г.

Себестоимость позиции рассчитывается как сумма стоимости ингредиентов, входящих в рецептуру:

$$C = \sum (\text{Цена_ингр_i} \cdot m_i), \quad (3)$$

где Цена_ингр_i – стоимость единицы массы i -го ингредиента; m_i – масса ингредиента в рецептуре.

Конечная цена позиции может определяться через коэффициент наценки:

$$\text{Цена_позиции} = C \cdot (1 + K_наценки), \quad (4)$$

где $K_наценки$ – коэффициент наценки, применяемый при формировании цены продажи.

При оформлении заказа стоимость и калорийность рассчитываются по каждой строке с учетом количества основной позиции и выбранных добавок. Такой подход позволяет автоматически учитывать итоговое влияние каждой позиции заказа на общую сумму и КБЖУ.

$$\text{Sum_добавок} = \sum (\text{PriceRub_добавки} \cdot Q_добавки \cdot Q_позиции), \quad (5)$$

$$S_строки = \text{PriceRub_позиции} \cdot Q_позиции + \text{Sum_добавок}, \quad (6)$$

$$\text{Kcal_добавок} = \sum (\text{CaloriesKcal_добавки} \cdot Q_добавки \cdot Q_позиции), \quad (7)$$

$$K_строки = \text{CaloriesKcal_позиции} \cdot Q_позиции + \text{Kcal_добавок}, \quad (8)$$

где $S_строки$ – стоимость строки заказа; $K_строки$ – калорийность строки заказа; $Q_позиции$ – количество основной позиции; $Q_добавки$ – количество добавки, выбранное для одной основной позиции.

Итоговые стоимость и калорийность заказа определяются суммированием рассчитанных строк заказа:

$$S_{\text{заказа}} = \text{round2} (\Sigma S_{\text{строки}} \cdot (1 - \text{DiscountPercent} / 100)), \quad (9)$$

$$K_{\text{заказа}} = \text{round2}(\Sigma K_{\text{строки}}), \quad (10)$$

где $S_{\text{заказа}}$ – итоговая стоимость заказа с учетом скидки; $K_{\text{заказа}}$ – итоговая калорийность заказа; DiscountPercent – процент скидки

Для контроля складских остатков рассчитывается отклонение фактического количества сырья от расчетного расхода и его денежное выражение:

$$\Delta = \text{StockFact} - \text{Required}, \quad (11)$$

$$\Delta_{\text{cost}} = \Delta \cdot \text{CostRub}, \quad (12)$$

где Δ – отклонение фактического остатка от расчетного расхода; StockFact – фактический остаток сырья; Required – расчетный расход сырья; Δ_{cost} – стоимостное отклонение; CostRub – стоимость единицы сырья.

2.1.3.3 Способы интерпретации и визуального представления информации

Для эффективной работы с данными в предметной области важно не только корректное хранение и обработка информации, но и её удобное представление пользователю. От того, насколько понятно отображается информация о позициях меню, заказах, ингредиентах и складских остатках, зависит скорость принятия решений сотрудниками и качество обслуживания клиентов. Интерфейс должен обеспечивать наглядность, снижение когнитивной нагрузки на пользователя и быстрый доступ к ключевым данным.

В разрабатываемой системе способы представления информации подбираются с учётом роли пользователя:

- Клиенту важно удобно просматривать меню, состав позиций, стоимость и пищевую ценность.
- Сотрудникам необходимо быстро ориентироваться в заказах, технологических картах, стоп-листе, остатках ингредиентов и текущем состоянии рабочих процессов.
- Администратор использует интерфейс для контроля меню, складских данных, заказов, клиентов, аналитической информации и общего состояния системы.

Информация в предметной области представляется несколькими способами:

- в виде списков – список позиций меню, заказов, ингредиентов, клиентов и сотрудников;
- в виде карточек – карточки блюд, напитков и добавок для клиентов;
- в виде таблиц – для отображения остатков, состава позиций и истории заказов;
- в виде диаграмм – для отображения статистических данных;
- в виде уведомлений – о критических остатках ингредиентов и недоступности отдельных позиций.

Принципы интерпретации:

- позиции отображаются с составом, ценой и рассчитанной пищевой ценностью, включая КБЖУ;
- остатки ингредиентов визуализируются с цветовым индикатором состояния: достаточно, мало или критично;
- заказы отображаются в списке очередности с выделением текущего статуса: новый → готовится → готов;
- технологические карты представляются в структурированном виде, чтобы сотрудник мог быстро определить состав и порядок приготовления позиции;
- клиенты отображаются с ФИО, категорией и номером телефона, для удобной навигации администратора;
- аналитические данные отображаются в обобщённом виде и позволяют оценивать популярность позиций и состояние продаж.

2.1.3.4 Технологии получения и передачи информации

Передача и получение информации являются важной частью функционирования информационной системы общественного питания «Garden Nook». Обмен данными между пользовательскими интерфейсами, серверной частью и базой данных обеспечивает актуальное отображение меню, оформление заказов, работу со складом, ведение технологических карт, обработку списаний, управление клиентами, сотрудниками и отчётами. Такая организация позволяет разделить функции системы на логические блоки.

В системе используется клиент-серверная архитектура. Клиентская часть представлена веб-приложением для посетителей и WPF-приложением для сотрудников и администратора, а серверная часть реализована в виде Web API.

Взаимодействие между компонентами выполняется через HTTP/HTTPS-запросы, а данные передаются в структурированном формате JSON. Это позволяет однозначно описывать состав заказа, сведения о блюдах и напитках, данные клиента, складские остатки и результаты выполнения операций.

Взаимодействие компонентов системы построено по принципам REST API. Для работы с ресурсами используются стандартные HTTP-методы:

- GET – получение данных, например, списка блюд, напитков, заказов, клиентов, складских остатков или отчётов;
- POST – создание новых записей, например, оформление заказа, регистрация клиента, добавление ингредиента, сотрудника или позиции меню;
- PUT – изменение существующих данных, например, редактирование заказа, обновление сведений о клиенте, изменение технологической карты или параметров блюда;
- DELETE – удаление записей, если это допускается бизнес-логикой системы.

Для защиты данных используется механизм аутентификации и разграничения доступа по ролям. После успешной авторизации сервер создаёт cookie-аутентификацию, благодаря чему пользователь или сотрудник может выполнять последующие запросы без повторной передачи логина и пароля. Права доступа позволяют отделять действия клиента от административных операций, например, управление меню, складом, сотрудниками, скидками, категориями товаров и отчётностью доступно только пользователям с соответствующей ролью.

Работа с хранилищем данных выполняется через реляционную базу данных SQL Server с использованием Entity Framework Core. Серверная часть обращается к связанным таблицам, содержащим сведения о заказах, клиентах, сотрудниках, блюдах, напитках, ингредиентах, технологических картах, заготовках, списаниях и скидках. При обработке запросов выполняется выборка, добавление, изменение и удаление данных с сохранением связей между сущностями и контролем целостности информации.

Таким образом, технологии получения и передачи информации в системе «GardenNook» обеспечивают своевременную передачу данных от клиента к серверу, корректную обработку операций, безопасный доступ к функциям системы, сохранение изменений в базе данных и возврат результата в понятном для пользовательского интерфейса виде.

2.1.4 Обзор существующих программных реализаций решения задачи

На рынке представлено множество программных систем, направленных на автоматизацию заведений общественного питания, однако большинство из них ориентированы на учёт заказов и интеграцию с кассовым оборудованием. Для кофейни здорового питания критически важны функции, связанные с расчётом пищевой ценности позиций, контролем состава и работы со складом, а также визуальным представлением состава позиции клиенту. При анализе предметной области были рассмотрены несколько существующих решений, частично или полностью пересекающихся с задачей проекта. Ниже приведён обзор аналогов, определены их достоинства и недостатки, которые повлияли на формирование требований к системе:

1. Аналог 1: iiko – распространённая ресторанный POS-система, включающая модули для кассовых операций, учёта заказов, инвентаризации, хранения технологических карт, а также интеграцию с программами лояльности. Решение используется в сетевых заведениях, ресторанах и кафе. Интерфейс представлен на рисунке 2.1.4.1.

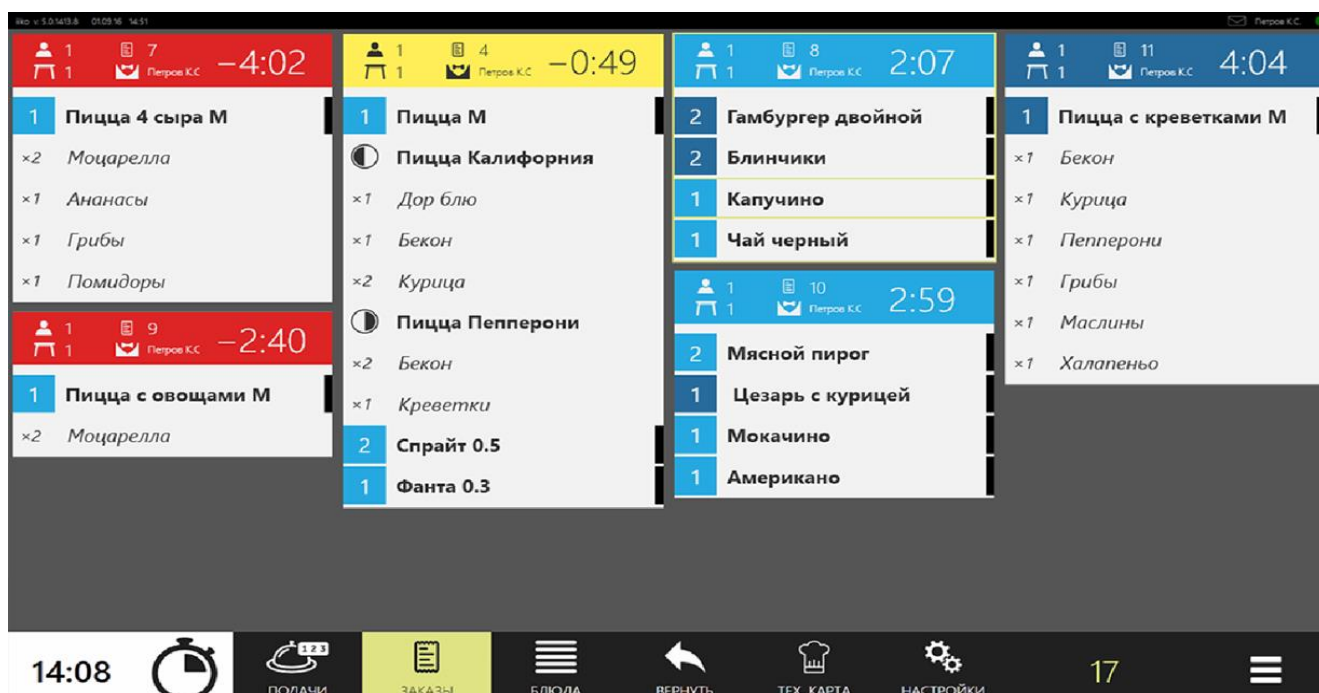


Рисунок 2.1.4.1 – Интерфейс iiko

Преимущества:

- автоматизация всех базовых операций ресторана (касса, склад, персонал);
- поддержка учёта себестоимости и технологических карт блюд;
- развитые отчёты по продажам и аналитика.

Недостатки:

- нет отображения пищевой ценности блюд (КБЖУ) для клиентов;
- система сложна в настройке и требует обучения персонала;
- высокая стоимость внедрения, особенно для небольших кофе-точек.

2. Аналог 2: r_keeper – профессиональная система автоматизации ресторанного бизнеса, ориентированная на средние и крупные предприятия. Предоставляет гибкие инструменты для управления заказами, складом, персоналом и финансовыми потоками. Интерфейс представлен на рисунке 2.1.4.2.

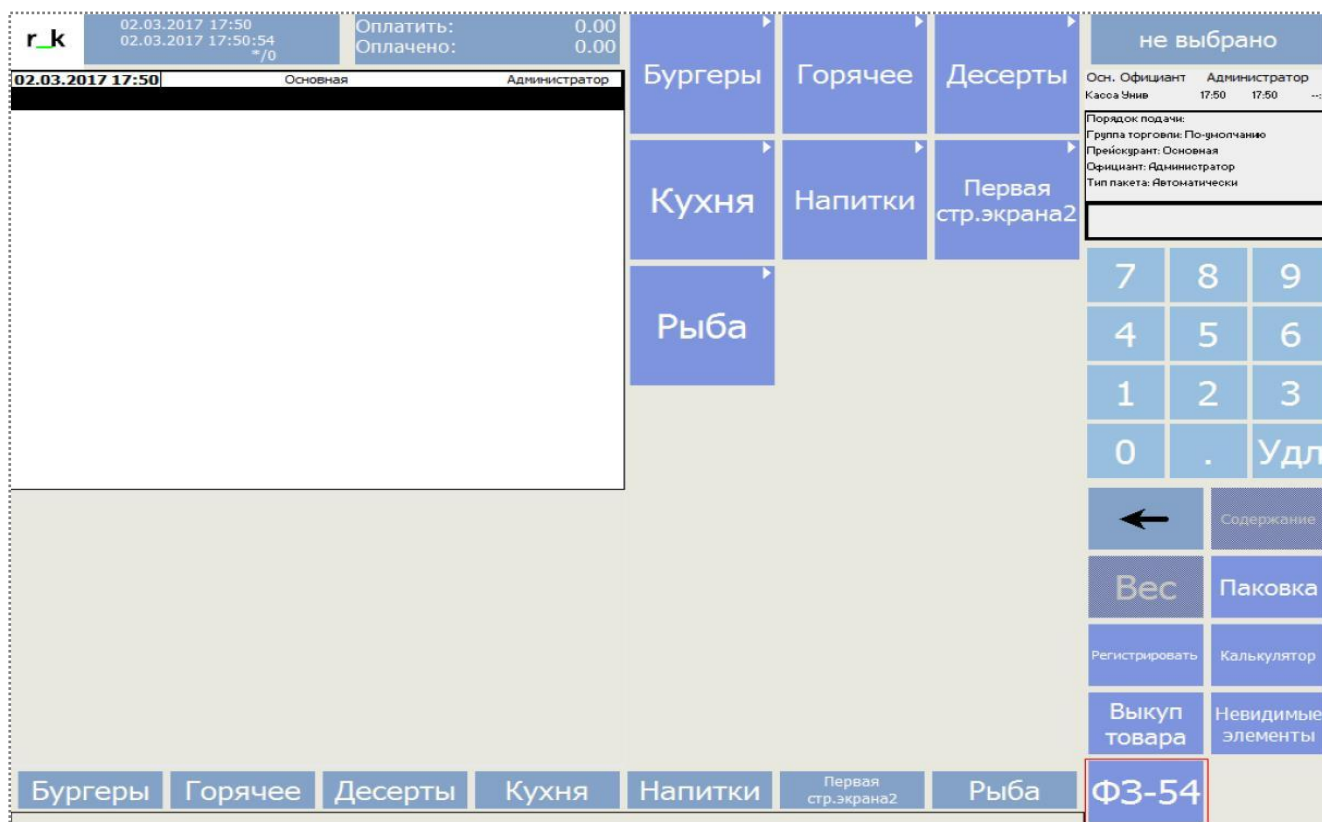


Рисунок 2.1.4.2 – Интерфейс r_keeper

Преимущества:

- комплексное управление рестораном «под ключ»;
- широкие возможности конфигурации под разные форматы заведений;
- поддержка сетевой архитектуры и распределённых точек.

Недостатки:

- отсутствует функционал отображения состава и пищевой ценности блюд клиенту;
- система ориентирована на внутренние операционные процессы, а не на работу с потребителем;
- высокие расходы на обслуживание и поддержку.

Анализ показал, что существующие решения хорошо закрывают задачи ресторана, связанные с оформлением и обработкой заказов, учётом продаж и интеграцией с кассовым оборудованием. Однако ни одна из систем не ориентирована на прозрачное предоставление клиенту информации о составе позиций, пищевой ценности (КБЖУ), аллергенах и работе с концепцией здорового питания. Кроме того, отсутствуют возможности автоматического расчёта КБЖУ на основании ингредиентов и автоматического списания сырья по рецептуре. Таким образом, разработка собственной информационной системы Garden Nook имеет обоснованную новизну: она ориентирована не только на процесс приёма заказов, но и на управление здоровым меню, прозрачную информацию о составе позиций и складскую аналитику.

2.1.5 Концептуальное обоснование разработки

Анализ предметной области показал, что работа кофейни здорового питания включает множество взаимосвязанных процессов: формирование меню, расчёт пищевой ценности, учёт рецептур, управление заказами и складом, анализ продаж и планирование закупок. Большая часть этих действий выполняется вручную: данные фиксируются в таблицах, рецептуры хранятся в разных источниках, расчёты КБЖУ и себестоимости проводятся отдельно.

Информационные потребности пользователей подтвердили необходимость автоматизации. Бариста и повару требуется доступ к актуальным рецептурам и заказам, управляющему – аналитика продаж, данные о складских остатках, информация о минимальном количестве ингредиентов и прогнозы потребления, клиенту – прозрачные сведения о составе позиций и пищевой ценности. Эти потребности не перекрываются, но опираются на одни и те же данные, что усиливает необходимость единого хранилища информации.

В результате были выделены ключевые информационные объекты системы: позиция, ингредиент, заказ, клиент, сотрудник. Выбор реляционной базы данных MS SQL Server обеспечил структурированное хранение данных и строгую связность между объектами. Построение взаимосвязей через первичные и внешние ключи позволяет исключить дублирование информации и гарантировать корректность данных при выполнении операций (например, списание ингредиентов при создании заказа).

Анализ аналогов (iiko, r_keeper, YCLIENTS) показал, что существующие решения решают стандартные задачи автоматизации заведений, но не предоставляют возможности работы с концепцией здорового меню – расчётом пищевой ценности позиций, учётом аллергенов и прозрачным отображением состава для клиента. Это подтверждает новизну разработки.

Таким образом, необходимость создания программного комплекса Garden Nook обусловлена реальной потребностью бизнеса: объединить разрозненные процессы, обеспечить достоверность данных и ускорить работу персонала. Система позволит исключить ручные расчёты, повысить прозрачность работы кофейни и улучшить качество обслуживания клиентов.

2.2 Классы и характеристики пользователей

Пользователи программного комплекса Garden Nook разделяются на функциональные группы в соответствии с их ролью в бизнес-процессах кофейни. Такое разделение позволяет определить информационные потребности каждой группы, ограничить доступ к служебным данным и связать пользовательские действия с ответственностью за конкретный этап обслуживания. Дополнительно классификация пользователей помогает выстроить логику интерфейса, разграничить доступные функции системы и обеспечить более удобную работу каждого участника процесса. Классы пользователей подробно описаны в таблице 2.2.1.

Таблица 2.2.1 – Классы и характеристики пользователей

Роль пользователя	Характеристика	Информационные потребности	Основные операции
Клиент	Внешний пользователь системы, оформляющий заказ через веб-интерфейс.	Просмотр актуального меню, состава, КБЖУ, цены и статуса заказа.	Регистрация, авторизация, формирование корзины, оформление и отмена собственного заказа, просмотр профиля и истории.
Бариста-кассир	Сотрудник фронт-зоны, принимающий и сопровождающий заказы.	Данные о меню, доступности позиций, комментариях клиента, времени подачи и статусах заказов.	Создание заказа, изменение статуса, просмотр техкарт напитков, работа со стоп-листом и заготовками.
Повар	Сотрудник производственной зоны, отвечающий за приготовление блюд.	Очередь заказов, технологические карты, состав рецептур, остатки сырья и полуфабрикатов.	Просмотр заказов, отметка готовности, контроль техкарт, работа с заготовками, стоп-листом и актами списания.
Управляющий/администратор	Административный пользователь, контролирующий ассортимент, склад и аналитику.	Данные о продажах, клиентах, сотрудниках, остатках, скидках, популярных позициях и списаниях.	Управление справочниками, заказами, клиентами, персоналом, складом, программой лояльности и отчетами.

Итоговая ролевая модель показывает, что каждая группа пользователей работает только с тем набором данных, который необходим для выполнения ее задач. Клиентский контур отделен от служебных справочников и аналитики, а сотрудники получают доступ к операциям в соответствии со своими должностными обязанностями. Это повышает удобство работы, снижает риск ошибок и поддерживает целостность данных.

2.3 Функциональные требования

2.3.1 Определение функциональных возможностей

Функциональные требования программного средства «Garden Nook» описывают перечень действий, доступных авторизованным пользователям и соответствующих их роли и потребностям. Они были сформированы опираясь на информацию из технического задания из Приложения А. Реализация этих функций необходима для автоматизированной и удобной поддержки ключевых бизнес-процессов кофейни.

Функциональные требования клиента: [4]

- Регистрация новой учетной записи;
- авторизация с созданием защищенной сессии;
- получение и просмотр меню (блюда, напитки, добавки, модификаторы) с учетом актуальной доступности позиций;
- формирование корзины, настройка количества, добавок и модификаторов для позиций;
- оформление заказа с автоматическим расчетом итоговой стоимости, калорийности, скидки по категории клиента, указанием типа заказа, комментария и времени самовывоза;
- просмотр личного профиля и истории заказов;
- отмена заказа при допустимом статусе.

Функциональные требования повара: [5]

- Авторизация в системе;
- просмотр актуального списка заказов, обновляемого в реальном времени, с прилегающими комментариями и временем подачи;
- отметка готовности отдельных позиций заказа и изменение статуса заказа;
- просмотр технологических карт блюд, включающих список ингредиентов и технологию приготовления;
- просмотр и изменение количества сырья и полуфабрикатов, готовых к использованию;
- добавление элемента в список задач;
- указание даты приготовления кулинарного полуфабриката;
- просмотр, добавление и удаление из списка блюд, находящихся в стоп-листе;
- формирование актов списания.

Для клиента и повара наглядно демонстрируются процессы взаимодействия с программным комплексом на рисунках 2.3.1.1 и 2.3.1.2.

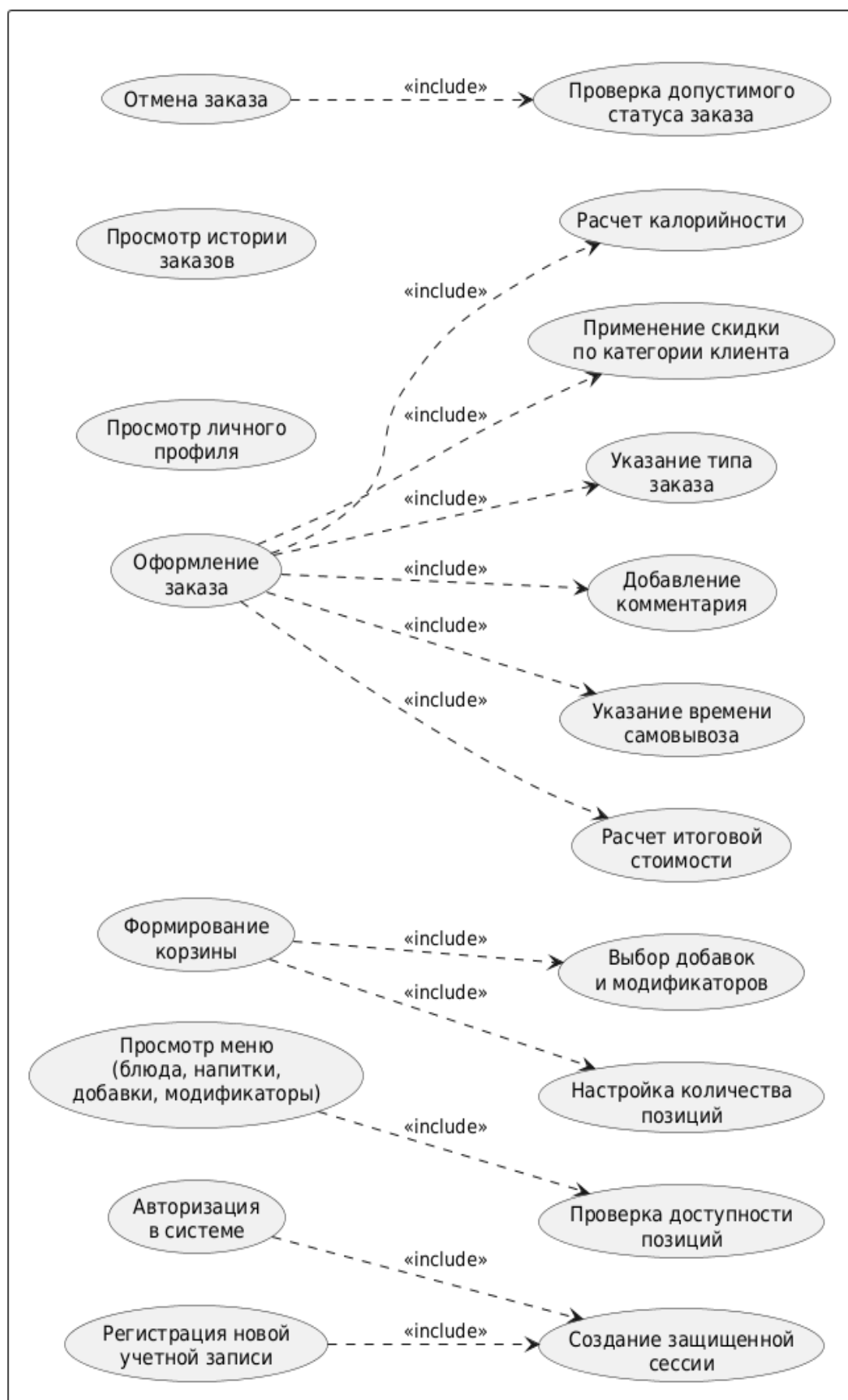
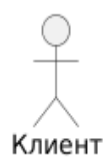


Рисунок 2.3.1.1 – Диаграмма вариантов использования для клиента

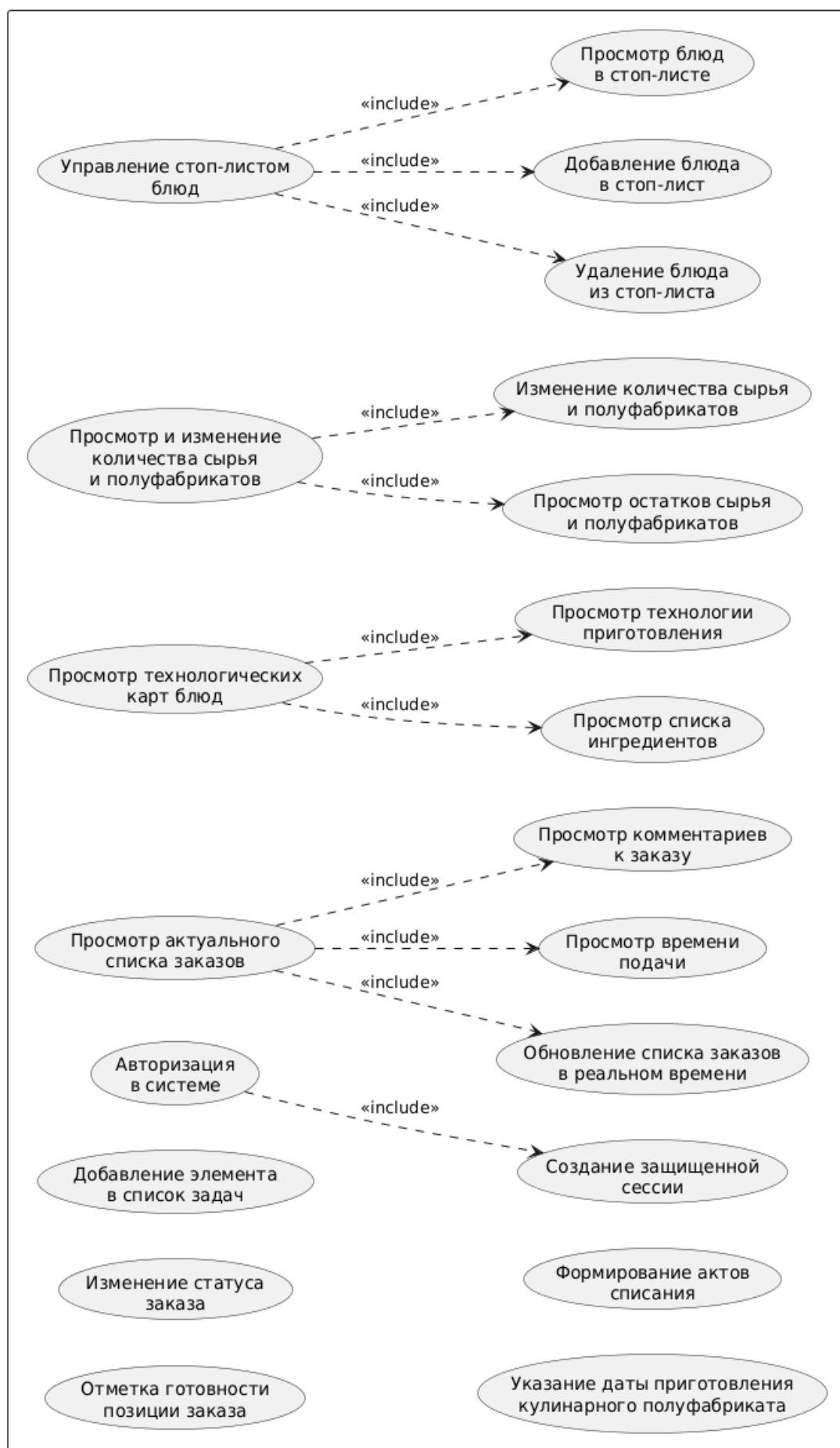
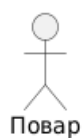


Рисунок 2.3.1.2 – Диаграмма вариантов использования для повара

Функциональные требования для администратора: [3]

- Авторизация в системе;
- просмотр и добавление, удаление и редактирование (далее – управление) позиций меню;
- просмотр и управление категориями меню;
- просмотр и управление списком сырья и полуфабрикатов (далее – ингредиентов);
- просмотр и управление категориями сырья и полуфабрикатов;
- контроль остатка ингредиентов и выполнение операций пополнения склада;
- просмотр и управление техническими картами;
- управление заготовками и задачами по приготовлению полуфабрикатов;
- просмотр, добавление и удаление позиций стоп-листа;
- формирование, просмотр и удаление актов списания;
- просмотр списка заказов за все время;
- добавление, удаление и редактирование заказов;
- изменение статуса заказа;
- просмотр деталей заказа;
- просмотр списка клиентов;
- управление данными клиентов, включая изменение категории и просмотр истории заказов;
- просмотр аналитики по продажам, ABC-анализа и данных инвентаризации;
- просмотр статистики популярных позиций;
- управление программой лояльности;
- управление составом персонала и правами доступа сотрудников.

Диаграмма вариантов использования для администратора представлена в Приложении Б.

Функциональные требования бариста-кассира: [6]

- Авторизация в системе;
- просмотр меню (блюда, напитки, добавки, модификаторы) с учетом актуальной доступности позиций;
- формирование корзины, настройка количества, добавок и модификаторов для позиций;
- оформление заказа с автоматическим расчетом итоговой стоимости, калорийности и указанием комментария, времени подачи и скидки;
- просмотр актуального списка заказов, обновляемого в реальном времени, с прилегающими комментариями и временем подачи;

- просмотр технологических карт напитков, включающих список ингредиентов и технологию приготовления;
- работа с заготовками при наличии соответствующих задач;
- просмотр, добавление и удаление из списка напитков и добавок, находящихся в стоп-листе;
- изменение статуса заказа

Визуализация взаимодействий бариста-кассира с программным средством представлена на рисунке 2.3.1.3.

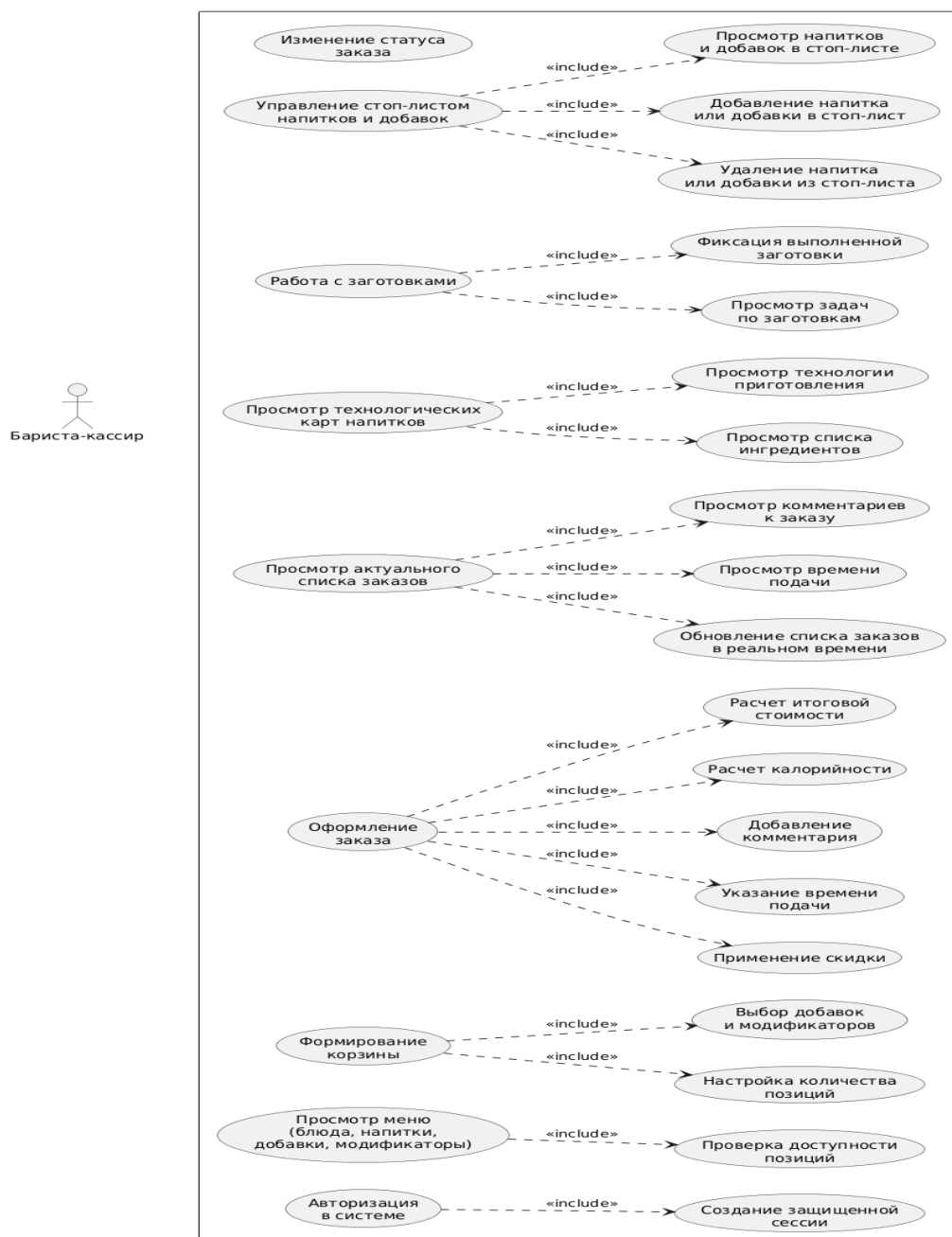


Рисунок 2.3.1.3 – Диаграмма вариантов использования для бариста-кассира

2.3.2 Описание прецедентов

Детализация функциональных требований в рамках разрабатываемой системы представлена в виде описания ключевых прецедентов, отражающих последовательность действий актеров и реакцию программного комплекса на их запросы. В таблицах 2.3.2.1-2.3.2.3 описаны основные прецеденты взаимодействия пользователя с программным продуктом.

Таблица 2.3.2.1 – Описание прецедента «Авторизация клиента»

Характеристика	Описание
Название прецедента	Авторизация клиента
Краткое описание	Процесс входа клиента в учетную запись для получения доступа к защищенным функциям системы.
Актер	Неавторизованный клиент.
Предусловия	Клиент имеет зарегистрированную учетную запись и доступ к форме авторизации веб-приложения.
Основной успешный сценарий	1. Клиент открывает форму авторизации в веб-приложении. 2. Клиент вводит номер телефона и пароль. 3. Клиент инициирует вход нажатием кнопки «Войти». 4. Система передает учетные данные на сервер авторизации. 5. Сервер проверяет корректность формата и соответствие данных учетной записи клиента. 6. При успешной проверке система формирует набор авторизационных признаков пользователя. 7. Система создает авторизационную cookie-сессию клиента. 8. Клиент получает подтверждение успешного входа и переход к функционалу оформления заказов.
Постусловие	В системе зарегистрирована активная авторизационная сессия клиента; доступ к защищенным разделам предоставлен.

Таблица 2.3.2.2 – Описание прецедента «Создание заказа клиентом»

Характеристика	Описание
Название прецедента	Создание заказа клиентом
Краткое описание	Процесс формирования и фиксации клиентского заказа с последующей постановкой в производственную очередь.
Актер	Авторизованный клиент.
Предусловия	Клиент авторизован в системе; в корзину добавлена минимум одна позиция; выбран допустимый тип заказа.
Основной успешный сценарий	1. Клиент открывает корзину и проверяет состав выбранных позиций. 2. Клиент при необходимости корректирует количество и набор добавок. 3. Клиент выбирает тип заказа и указывает комментарий к заказу. 4. Клиент отправляет запрос на создание заказа. 5. Система передает данные на серверный endpoint оформления заказа. 6. Сервер выполняет валидацию структуры запроса, идентификаторов позиций и типа заказа. 7. Сервер рассчитывает итоговые показатели заказа, применяет клиентскую скидку и проверяет доступность заготовок. 8. При успешной проверке система сохраняет заказ и связанные позиции в базе данных в рамках транзакции. 9. Система присваивает заказу уникальный номер и статус «В процессе». 10. Клиент получает подтверждение оформления заказа с номером, статусом и итоговыми параметрами.
Постусловие	Заказ сохранен в системе, помещен в очередь кухни и доступен клиенту для последующего отслеживания.

Таблица 2.3.2.3 – Описание прецедента «Обработка нового заказа поваром»

Характеристика	Описание
Название прецедента	Обработка нового заказа поваром
Краткое описание	Процесс принятия нового заказа в работу на кухне, приготовления и фиксации результата обработки.
Актер	Повар.
Предусловия	Повар авторизован в рабочем интерфейсе; в системе присутствуют новые заказы со статусом «В процессе».
Основной успешный сценарий	1. Повар открывает раздел обработки заказов в интерфейсе кухни. 2. Система загружает и отображает очередь новых заказов в порядке поступления. 3. Повар выбирает новый заказ и анализирует состав позиций, добавок и комментариев клиента. 4. Повар принимает заказ в работу и выполняет приготовление в соответствии с технологическими требованиями. 5. По завершении приготовления повар инициирует подтверждение готовности заказа. 6. Система фиксирует завершение обработки заказа. 7. Система переводит заказ в статус «Готов». 8. Обновленный статус заказа становится доступным в связанных пользовательских и служебных интерфейсах.
Постусловие	Заказ обработан сотрудником кухни и переведен в статус «Готов»; состояние заказа актуализировано в системе.

2.4 Нефункциональные требования

Нефункциональные требования определяют качественные характеристики программной системы Garden Nook и регламентируют условия ее стабильной эксплуатации в контуре кофейни здорового питания. С учетом принятой архитектуры решения (веб-клиент для клиентов, WPF-клиент для сотрудников, серверный API на ASP.NET Core и база данных MS SQL Server) требования сгруппированы по ключевым атрибутам качества: производительность, надежность, безопасность, совместимость, сопровождаемость и удобство использования.

2.4.1 Производительность и масштабируемость

Система должна обеспечивать нормативное время отклика при типовой нагрузке: загрузка меню, корзины и профиля клиента – не более 2 секунд для 95% запросов; оформление заказа через API – не более 3 секунд для 95% запросов с учетом серверной валидации, расчетов и транзакционной записи в базу данных; получение производственной очереди кухни – не более 2 секунд. Механизм обновления заказов на стороне рабочего интерфейса кухни должен поддерживать актуализацию данных в режиме, близком к реальному времени, с интервалом обновления не более 10 секунд. Архитектура должна допускать масштабирование серверного слоя без переработки клиентских приложений за счет оптимизации SQL-запросов, индексации, кэширования неизменяемых справочных данных и выделения ресурсоемких операций в отдельные сервисы.

2.4.2 Надежность и доступность

Критические бизнес-операции (создание заказа, списание заготовок, изменение статусов) должны выполняться атомарно и согласованно, исключая частичную фиксацию данных при сбоях. Серверная часть должна обеспечивать корректную обработку исключительных ситуаций с возвратом контролируемых кодов и сообщений об ошибках без нарушения целостности данных. Система должна обеспечивать доступность не ниже 99% в часы производственной эксплуатации и поддерживать регламентное восстановление работоспособности после отказа с сохранением накопленной информации о заказах и клиентах. Для повышения ремонтпригодности должна быть обеспечена централизованная регистрация событий и ошибок с возможностью последующего анализа инцидентов.

2.4.3 Безопасность

Передача данных между клиентскими приложениями и сервером должна выполняться исключительно по защищенному протоколу HTTPS (TLS 1.2 и выше). Доступ к защищенным методам API должен предоставляться только авторизованным пользователям на основе cookie-аутентификации и серверной проверки прав доступа в соответствии с ролью пользователя (клиент, сотрудник, администратор). Входные данные, поступающие от клиентов, должны проходить обязательную валидацию на сервере (корректность идентификаторов, формат контактных данных, допустимость количественных параметров заказа). Система должна исключать несанкционированный доступ к персональным и операционным данным, а учетные данные пользователей должны храниться в защищенном виде с применением криптографического хеширования и соли. Журналы безопасности должны фиксировать события входа, выхода и критических изменений состояния заказов для последующего аудита.

2.4.4 Переносимость и совместимость

Веб-клиент должен корректно функционировать в актуальных версиях браузеров Google Chrome, Microsoft Edge и Mozilla Firefox на настольных и мобильных устройствах. Рабочее приложение сотрудников (WPF) должно стабильно функционировать в операционных системах семейства Windows 10/11 при установленной платформе .NET 8. Серверный API должен быть совместим со стандартной инфраструктурой разворачивания ASP.NET Core и обеспечивать корректный обмен данными в формате JSON (UTF-8) по REST-интерфейсам. Подсистема хранения данных должна поддерживать работу с MS SQL Server и обеспечивать миграцию схемы без потери существующих данных.

2.4.5 Сопровождаемость и расширяемость

Программный код должен иметь модульную структуру с разделением на слои представления, бизнес-логики, доступа к данным и контрактов обмена, что снижает связность и упрощает сопровождение. Расширение функциональности (новые роли, дополнительные статусы заказов, новые категории меню, дополнительные отчеты) должно выполняться без нарушения базовых сценариев работы системы. Изменения структуры базы данных должны вноситься через управляемые миграции с контролем версий и возможностью воспроизводимого развертывания. Техническая документация, именование сущностей и правила обработки ошибок должны поддерживать единый стандарт, достаточный для передачи проекта на сопровождение другому разработчику.

2.4.6 Удобство использования и локализация

Интерфейсы клиентского и служебного контуров должны обеспечивать интуитивную навигацию и выполнение основных операций с минимальным числом действий пользователя. Критичные сценарии (авторизация, формирование заказа, подтверждение оформления, просмотр очереди кухни) должны сопровождаться однозначной визуальной обратной связью и понятными диагностическими сообщениями. Текстовые элементы интерфейса, форматы даты и времени, а также отображение денежных значений должны соответствовать русскоязычной локализации и принятым стандартам отображения для валюты рубль. Пользовательский интерфейс должен сохранять читаемость и функциональную доступность при различных разрешениях экранов и в условиях интенсивной операционной работы персонала. Сформулированные нефункциональные требования задают ограничения, которые необходимо учитывать при выборе программных средств и проектировании архитектуры. Они связывают качество реализации с условиями эксплуатации системы в реальной кофейне: быстрым обслуживанием, защитой данных, устойчивостью операций и удобством ежедневной работы персонала.

3 ВЫБОР ПРОГРАММНЫХ СРЕД И СРЕДСТВ РАЗРАБОТКИ

3.1 Сравнительный анализ имеющихся возможностей по выбору средств разработки

Для выбора технологического стека Garden Nook выполнено сопоставление альтернатив по критериям интеграции с текущей архитектурой, надежности, производительности, удобства сопровождения и стоимости владения. Дополнительно учитывались совместимость с уже используемыми компонентами системы и сложность дальнейшего развития проекта. Сводная информация предоставлена в таблицах 3.1.1-3.1.5.

Таблица 3.1.1 – Сравнение серверных платформ

Кандидат	Плюсы	Минусы
ASP.NET Core	Нативная интеграция с C# и EF Core, высокая производительность, удобная реализация REST API и middleware.	Ориентация на экосистему Microsoft, требует устойчивой .NET-экспертизы.
Spring Boot	Зрелая экосистема Java, большое количество библиотек.	Дополнительная сложность интеграции с текущим .NET-стеком, повышенные затраты на сопровождение смешанной архитектуры.
Node.js (NestJS)	Быстрая разработка API, единый язык с фронтендом.	Менее предсказуемая модель для сложной транзакционной логики, дополнительные риски при сопровождении многомодульной системы.

По результатам сравнения в качестве серверной платформы выбран ASP.NET Core. Он обеспечивает встроенную поддержку Web API, удобную организацию маршрутов, механизмов авторизации и обработки ошибок. Дополнительным преимуществом является согласованность платформы с языком C# и существующей структурой проекта Garden Nook.

Таблица 3.1.2 – Сравнение технологий desktop-интерфейса для сотрудника

Кандидат	Плюсы	Минусы
WPF	Развитый data binding, гибкая стилизация интерфейса, удобная работа со сложными экранами.	Платформенная зависимость от Windows.
WinForms	Низкий порог входа, быстрый старт разработки.	Ограниченные средства современного UI, сложнее поддерживать масштабируемую визуальную архитектуру.
.NET MAUI	Потенциальная кроссплатформенность.	Избыточность для задач текущего Windows-контура, дополнительные риски для стабильности desktop-сценариев.

Для служебного интерфейса выбран WPF, так как приложение сотрудников должно стабильно работать на рабочих станциях Windows и поддерживать насыщенный настольный интерфейс. Технология позволяет реализовать таблицы, модальные окна, статусы заказов и быстрые операции без зависимости от браузера. Кроме того, WPF уже используется в проекте, что снижает трудоемкость сопровождения.

Таблица 3.1.3 – Сравнение подходов к доступу к данным

Кандидат	Плюсы	Минусы
Entity Framework Core	Единая объектная модель, поддержка миграций, интеграция с ASP.NET Core.	Более сложный SQL-контроль без профилирования.
Dapper	Высокая скорость выполнения запросов, явный контроль SQL.	Повышенный объем ручного кода, отсутствие встроенной модели миграций.
NHibernate	Богатые ORM-возможности.	Более высокий порог сопровождения, избыточность для текущего масштаба проекта.

В качестве механизма доступа к данным выбран Entity Framework Core. Он позволяет связать объектную модель C# с реляционной структурой базы данных, описывать связи между сущностями и управлять изменениями схемы через миграции. Такой подход уменьшает объем ручного SQL-кода и повышает воспроизводимость развертывания.

Таблица 3.1.4 – Сравнение систем управления базами данных

Кандидат	Плюсы	Минусы
Microsoft SQL Server	Надежная транзакционная модель, интеграция с EF Core SqlServer, развитые инструменты администрирования.	Ограничения бесплатной редакции, привязка к экосистеме Microsoft.
PostgreSQL	Высокая надежность, широкие возможности расширения.	Дополнительные затраты на переадаптацию текущей конфигурации проекта.
SQLite	Простота развертывания.	Ограничения в многопользовательских и нагруженных сценариях, неподходящая для центральной БД производственной системы.

В качестве СУБД выбран Microsoft SQL Server. Данное средство поддерживает транзакции, ограничения целостности, индексацию и надежное хранение связанных данных, что соответствует требованиям к заказам, рецептурам и складскому учету. Выбор также согласуется с экосистемой .NET и средствами разработки проекта.

Таблица 3.1.5 – Сравнение сред разработки

Кандидат	Плюсы	Минусы
Visual Studio Community	Полный цикл разработки .NET-решения, удобная отладка API, MVC и WPF, интеграция с миграциями EF Core.	Повышенные требования к ресурсам рабочей станции.
Visual Studio Code	Легковесность, гибкость расширений.	Меньшая интеграция для сложного WPF-сценария, необходимость дополнительной настройки инструментов.
JetBrains Rider	Сильная поддержка C#, высокая производительность IDE.	Платная лицензия, меньшая встроенная поддержка визуального проектирования XAML.

Основной средой разработки выбрана Visual Studio Community. Она предоставляет инструменты для разработки, отладки и сборки проектов ASP.NET Core, WPF и библиотек классов в одном рабочем пространстве. Это упрощает сопровождение решения, проверку ошибок и работу с зависимостями.

3.2 Характеристика выбранных программных сред и средств

В соответствии с результатами сравнительного анализа для реализации Garden Nook принят набор программных средств, согласованный с функциональными и нефункциональными требованиями проекта.

Язык программирования C# и платформа .NET 8 используются как единая технологическая основа решения. Данный выбор обеспечивает строгую типизацию, развитые средства асинхронной обработки, удобную работу с объектной моделью предметной области и стабильную поддержку со стороны экосистемы Microsoft.

ASP.NET Core Web API применяется для реализации серверных endpoint-ов, обслуживающих операции авторизации, формирования заказов, получения профиля клиента и загрузки очереди кухни. Фреймворк обеспечивает расширяемую архитектуру HTTP-конвейера и формализованное управление кодами состояния и ответами API. [7]

ASP.NET Core MVC (Razor) используется для веб-интерфейса клиентского контура. Такой подход позволяет совместить серверный рендеринг страниц с интерактивными сценариями на JavaScript (работа с корзиной, профиль, асинхронные запросы к API) без избыточного усложнения фронтенд-инфраструктуры.

WPF-приложение на .NET 8 применяется в служебном контуре для отображения производственной очереди и работы персонала кухни. Технология предоставляет необходимые механизмы построения насыщенного интерфейса, управления визуальными состояниями и адаптации под эксплуатацию на рабочих станциях Windows. [5], [8], [9]

Entity Framework Core реализует слой доступа к данным и связывает серверную бизнес-логику с реляционной моделью базы. Использование миграций обеспечивает контролируемое развитие схемы хранения и воспроизводимость развертывания на разных экземплярах среды. [10]

Microsoft SQL Server выступает централизованным хранилищем операционных данных системы: учетных записей, каталога меню, заказов, статусов, ингредиентов и складских сущностей. СУБД гарантирует транзакционную целостность и поддерживает инструменты администрирования, необходимые для сопровождения системы.

Swashbuckle (Swagger) используется для автоматизированного документирования REST-интерфейсов и оперативной проверки серверных методов в процессе разработки и тестирования.

В качестве основной среды разработки применена Visual Studio Community, обеспечивающая единый цикл проектирования, отладки и сборки проектов API, веб-клиента и WPF-модуля. Для проектирования пользовательских макетов и визуальной композиции интерфейсов используется Figma.

Таким образом, выбранные программные среды и средства соответствуют архитектуре Garden Nook, обеспечивают требуемый уровень надежности и создают основу для дальнейшего развития программного комплекса в рамках последующих этапов ВКР.

4 АЛГОРИТМЫ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ

4.1 Этапы реализации

Разработка программного комплекса Garden Nook включает несколько ключевых этапов, каждый из которых направлен на создание удобного инструмента для клиентов кофейни и сотрудников, отвечающих за прием заказов, производство, складской учет и управленческую аналитику:

- Проектирование базы данных: описание сущностей меню, заказов, клиентов, сотрудников, скидок, ингредиентов, технологических карт, складских остатков, заготовок и актов списания, создание ER-диаграммы.
- Разработка базы данных: создание сущностей, указание первичных и внешних ключей, импорт данных, реализация миграций и добавление ограничений целостности.
- Разработка серверной части: реализация контуров авторизации, управления сессией и ролевого разграничения доступа, реализация операции по созданию заказов, расчету итоговых параметров и управлению статусами заказов, реализация методов клиентского профиля, истории заказов и отмены заказов в допустимых состояниях, реализация endpoint-ов кухонной очереди для получения актуального списка заказов и их параметров, реализация складских и служебных операции, связанных с доступностью позиций, аналитикой, заготовками и внутренними процессами обслуживания.
- Разработка веб-интерфейса клиента: реализация пользовательских сценариев регистрации и входа в систему, реализация модулей просмотра меню, управления корзиной, добавками и модификаторами позиций, реализация оформления заказа, отображение результатов расчета и сопровождение заказа в личном профиле клиента.
- Разработка WPF-приложения сотрудников: реализация модулей авторизации сотрудника и отображения очереди заказов с периодическим обновлением, реализация модулей повара и бариста-кассира: технологические карты, управление заготовками с фиксацией даты приготовления, ведение стоп-листа, формирование актов списания, изменение статусов заказов, реализация модулей администратора: управление товарами, заказами, категориями и ингредиентами, просмотр клиентской базы, аналитика продаж и популярных позиций, элементы управления программой лояльности.

4.2 Пользовательский интерфейс

4.2.1 Описание взаимодействия с пользователем

Программный комплекс предназначен для оформления заказов клиентами кофейни Garden Nook и последующей обработки этих заказов сотрудниками, обеспечивая полную поддержку основных этапов работы – от выбора позиции меню до приготовления, складского учета и анализа результатов деятельности. Для основных сценариев взаимодействия пользователей с системой были разработаны UML-диаграммы активности, представленные на рисунках 4.2.1.1 - 4.2.1.3.

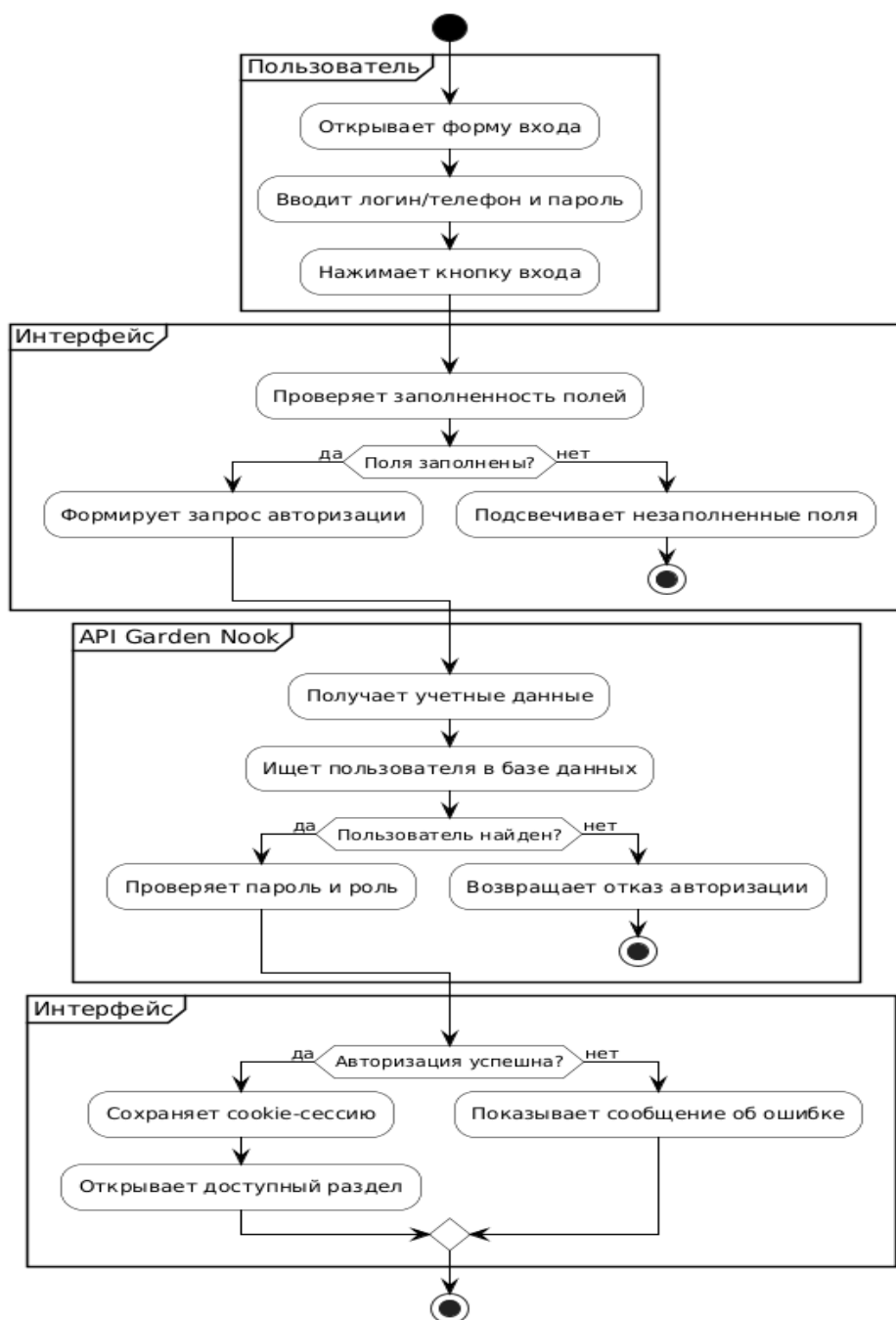


Рисунок 4.2.1.1 – Алгоритм действий по авторизации пользователя

На диаграмме представлена последовательность действий при авторизации пользователя в системе Garden Nook. Процесс разделён на несколько зон ответственности: «Пользователь», «Интерфейс» и «API Garden Nook», что позволяет показать взаимодействие между клиентской частью приложения и серверной логикой. Такое разделение также помогает наглядно определить, на каком этапе выполняется ввод данных, их предварительная проверка и последующая обработка на сервере.

Процесс начинается с действия пользователя, который открывает форму входа, вводит логин или номер телефона и пароль, после чего нажимает кнопку входа. Далее управление передаётся интерфейсу системы. На этом этапе выполняется первичная проверка заполненности обязательных полей. Если одно или несколько полей не заполнены, интерфейс подсвечивает незаполненные поля, и процесс авторизации завершается без отправки запроса на сервер.

Если все необходимые поля заполнены, интерфейс формирует запрос авторизации и передаёт введённые учётные данные в API Garden Nook. Серверная часть получает данные, после чего выполняет поиск пользователя в базе данных. Если пользователь с указанным логином или телефоном не найден, API возвращает отказ в авторизации, и процесс завершается неуспешно. Это позволяет исключить дальнейшую проверку пароля для несуществующей учётной записи.

Если пользователь найден, сервер проверяет корректность введённого пароля, а также роль пользователя. По результатам проверки API возвращает интерфейсу ответ об успешной или неуспешной авторизации. На стороне интерфейса выполняется анализ полученного результата. При успешной авторизации система сохраняет cookie-сессию и открывает пользователю доступный раздел приложения в соответствии с его ролью и правами доступа. За счёт этого пользователь получает доступ только к тем функциям системы, которые предусмотрены для его учётной записи.

Если авторизация не была подтверждена, интерфейс отображает сообщение об ошибке. После этого процесс завершается без предоставления доступа к защищённым разделам системы. Такой подход обеспечивает базовую защиту приложения от некорректного входа и предотвращает переход к рабочим разделам без успешной проверки учётных данных. Таким образом, диаграмма отражает полный сценарий входа в систему: от ввода пользователем учётных данных до проверки данных на сервере и принятия решения о предоставлении доступа.

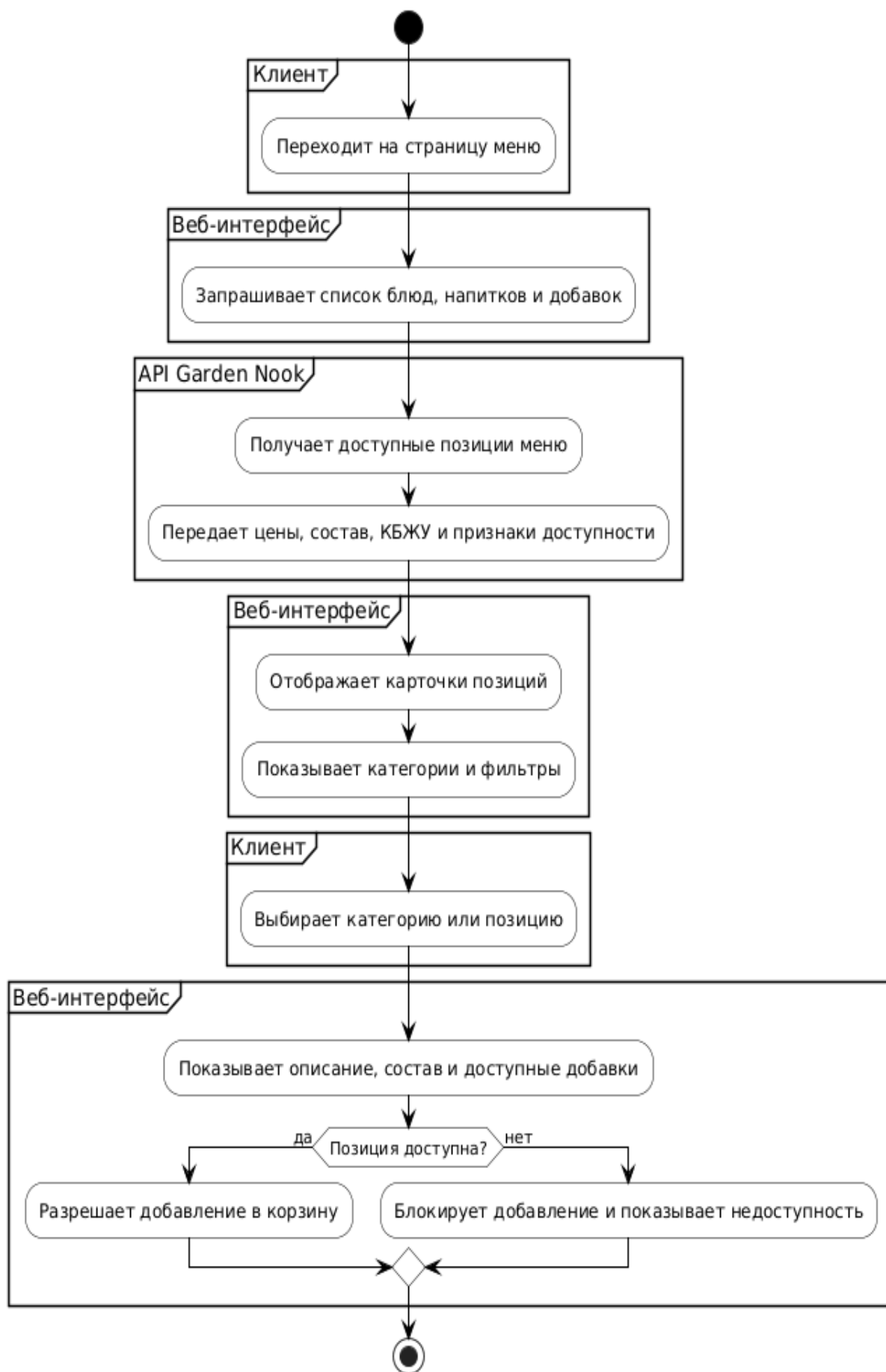


Рисунок 4.2.1.2 – Алгоритм действий по просмотру меню клиентом

На диаграмме представлен процесс просмотра меню клиентом в веб-интерфейсе системы Garden Nook. Сценарий отражает последовательность взаимодействия пользователя с клиентской частью приложения и серверным API при получении информации о блюдах, напитках и добавках. Диаграмма разделена на зоны ответственности: «Клиент», «Веб-интерфейс» и «API Garden Nook», что позволяет показать, какие действия выполняет пользователь, какие операции обрабатывает интерфейс, а какие относятся к серверной части системы.

Процесс начинается с перехода клиента на страницу меню. После открытия страницы веб-интерфейс формирует запрос к API Garden Nook для получения актуального перечня позиций. В запрос входит получение списка блюд, напитков и добавок, которые должны быть отображены пользователю. Такой подход позволяет не хранить меню только на клиентской стороне, а получать данные из единого источника, где могут обновляться цены, состав, доступность и пищевая ценность товаров.

API Garden Nook принимает запрос и получает доступные позиции меню. После этого серверная часть передаёт в веб-интерфейс основные сведения о каждой позиции: цену, состав, показатели КБЖУ и признак доступности. Признак доступности необходим для корректной работы интерфейса, так как отдельные позиции могут временно отсутствовать, быть отключены администратором или быть недоступными из-за нехватки ингредиентов.

После получения ответа от API веб-интерфейс отображает карточки позиций меню. В карточках клиент видит основную информацию, необходимую для первичного выбора товара. Дополнительно интерфейс показывает категории и фильтры, которые упрощают навигацию по меню. С помощью категорий пользователь может перейти к нужному типу продукции, например, к блюдам, напиткам или добавкам, а фильтры помогают быстрее найти подходящую позицию.

Далее клиент выбирает интересующую категорию или конкретную позицию меню. После выбора веб-интерфейс отображает подробную информацию: описание, состав и доступные добавки. Это позволяет пользователю ознакомиться с характеристиками товара до добавления его в корзину, уточнить состав, пищевую ценность и при необходимости выбрать дополнительные компоненты.

На завершающем этапе система проверяет доступность выбранной позиции. Если позиция доступна, веб-интерфейс разрешает добавление товара в корзину, после чего клиент может продолжить оформление заказа. Если позиция недоступна, интерфейс блокирует добавление и показывает сообщение о недоступности. Так, диаграмма описывает процесс получения меню, выбора позиции и проверки возможности её добавления в корзину.

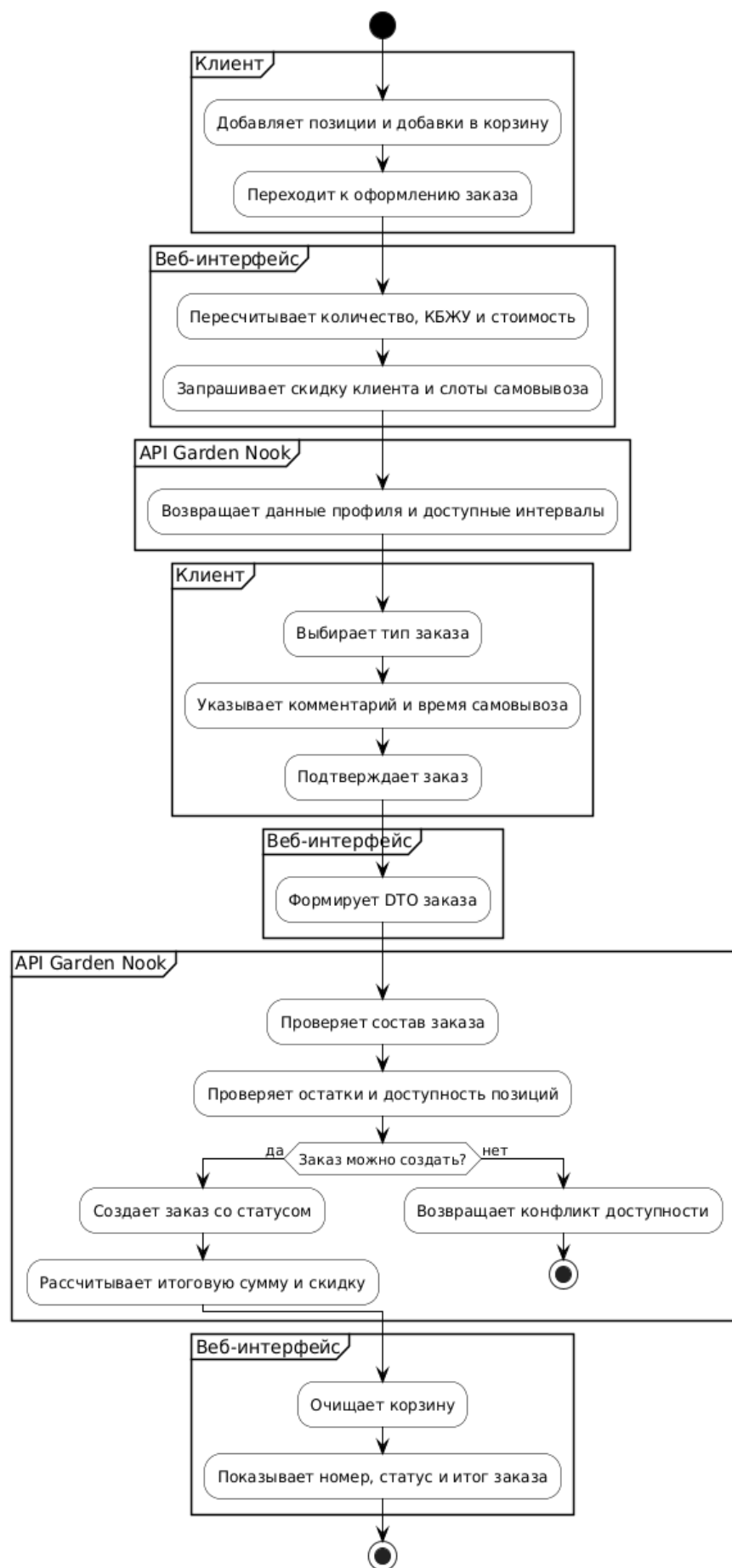


Рисунок 4.2.1.3 – Алгоритм действий по оформлению заказа клиентом

На диаграмме представлен процесс оформления заказа клиентом в веб-интерфейсе системы Garden Nook. Сценарий отражает последовательность действий от добавления позиций и добавок в корзину до создания заказа на стороне API и отображения результата пользователю. Процесс разделён на зоны ответственности «Клиент», «Веб-интерфейс» и «API Garden Nook», что позволяет показать распределение операций между пользователем, клиентской частью приложения и серверной логикой.

Процесс начинается с того, что клиент добавляет выбранные позиции и дополнительные добавки в корзину, после чего переходит к оформлению заказа. На стороне веб-интерфейса выполняется пересчёт количества выбранных товаров, показателей КБЖУ и общей стоимости заказа. Затем интерфейс запрашивает у API Garden Nook сведения, необходимые для оформления: данные профиля клиента, возможную скидку и доступные интервалы самовывоза.

API Garden Nook возвращает в веб-интерфейс данные профиля и список доступных временных интервалов. После этого клиент выбирает тип заказа, указывает комментарий и время самовывоза, а затем подтверждает заказ. На основании введённых данных веб-интерфейс формирует DTO заказа, в котором содержатся выбранные позиции, добавки, количество, параметры самовывоза и дополнительная информация от клиента.

Далее сформированные данные передаются в API Garden Nook. Серверная часть проверяет состав заказа, а также остатки и доступность выбранных позиций. На этом этапе система определяет, можно ли создать заказ с указанными товарами и выбранными параметрами. Дополнительно проверяется корректность переданных данных, чтобы исключить создание заказа с ошибочными позициями, неверным количеством или недоступным временным интервалом.

Если все позиции доступны, API создаёт заказ с начальным статусом и рассчитывает итоговую сумму с учётом скидки клиента. Если во время проверки выявляется конфликт доступности, например, позиция стала недоступной или её недостаточно для выполнения заказа, API возвращает соответствующий отказ. В этом случае заказ не создаётся, а пользователь должен скорректировать состав корзины или выбрать другое время самовывоза.

Такая проверка необходима для того, чтобы не допустить оформления заказа, который невозможно выполнить фактически. Она также обеспечивает согласованность данных между корзиной пользователя и актуальным состоянием меню на сервере.

При успешном создании заказа веб-интерфейс очищает корзину и показывает клиенту номер заказа, его текущий статус и итоговую стоимость. Это позволяет пользователю сразу убедиться, что заказ был принят системой и передан в дальнейшую обработку.

Дополнительно клиент получает актуальную информацию о состоянии заказа без необходимости повторно вводить данные или оформлять покупку заново. Таким образом, диаграмма описывает полный цикл оформления заказа от выбора позиций до подтверждения результата.

4.2.2 Определение операций пользователей и составление функциональных блоков

В рамках использования программного комплекса пользователи выполняют операции, связанные с оформлением заказа, обработкой производственной очереди, управлением меню, складом, технологическими картами, клиентами, персоналом и отчетностью. Для каждого раздела системы выделены операции пользователей:

В веб-интерфейсе клиента:

1. Авторизация клиента.
2. Регистрация клиента.
3. Просмотр меню клиента.
4. Открытие карточки выбранной позиции.
5. Выбор добавок или модификаторов позиции.
6. Добавление позиции в корзину.
7. Изменение количества позиции в корзине.
8. Выбор типа заказа.
9. Выбор времени самовывоза.
10. Ввод комментария к заказу.
11. Подтверждение оформления заказа.
12. Просмотр личного кабинета клиента.
13. Просмотр истории и статусов заказов.
14. Отмена активного заказа.

В WPF-приложении сотрудника:

15. Авторизация сотрудника.
16. Просмотр производственной очереди заказов.
17. Фильтрация заказов по статусу.
18. Открытие деталей заказа.
19. Просмотр технологической карты позиции.
20. Отметка готовности позиции заказа.
21. Завершение заказа.
22. Создание заказа сотрудником.
23. Добавление позиции меню.
24. Редактирование позиции меню.
25. Удаление позиции меню.
26. Создание категории меню.
27. Редактирование категории меню.
28. Удаление категории меню.

29. Прием поставки сырья.
30. Редактирование складской позиции.
31. Добавление позиции в стоп-лист.
32. Удаление позиции из стоп-листа.
33. Создание акта списания.
34. Выполнение задачи заготовки.
35. Просмотр списка клиентов.
36. Изменение категории клиента.
37. Управление персоналом.
38. Просмотр аналитических отчетов.

Общее навигационное действие:

39. Возвращение в главное меню.

Проанализировав операции пользователей, выделили основные функциональные блоки:

Блок 1 – Авторизация и управление пользователями. Блок обеспечивает регистрацию и вход клиентов, а также авторизацию сотрудников во внутреннем WPF-приложении. После входа пользователю открываются функции в соответствии с его ролью.

Блок 2 – Работа с меню и карточками позиций. Блок предназначен для просмотра и управления ассортиментом заведения. Клиенты просматривают меню и выбирают добавки, а сотрудники добавляют, редактируют и удаляют позиции и категории.

Блок 3 – Оформление и управление заказами. Блок отвечает за создание заказа: клиент добавляет позиции в корзину, выбирает тип заказа, время самовывоза и комментарий. Также клиент может просматривать историю, отслеживать статусы и отменять активные заказы.

Блок 4 – Производственная очередь и выполнение заказов. Блок используется сотрудниками для обработки поступивших заказов. Сотрудник просматривает очередь, открывает детали заказа, отмечает готовность позиций и завершает заказ.

Блок 5 – Складской учет и движение сырья. Блок предназначен для контроля складских остатков. Сотрудники редактируют складские позиции и создают акты списания.

Блок 6 – Стоп-лист и доступность позиций. Блок отвечает за временное ограничение продажи недоступных позиций меню. Сотрудник добавляет позиции в стоп-лист или удаляет их после восстановления доступности.

Блок 7 – Управление клиентами и персоналом. Блок объединяет административные функции по работе с клиентами и сотрудниками. В нем можно просматривать клиентов, изменять их категории и управлять данными персонала.

Блок 8 – Отчетность, аналитика и навигация. Блок предназначен для просмотра отчетов и анализа работы заведения. Навигационные функции обеспечивают быстрый переход между разделами системы.

4.2.3 Проектирование структуры экранов и схемы навигации

В процессе технического проектирования пользовательского интерфейса программного комплекса Garden Nook разрабатываются навигационные схемы, которые отражают структуру экранов и взаимосвязи между основными функциональными модулями. Эти схема создается на основе анализа пользовательских сценариев, ролей и задач, выполняемых в рамках клиентского и служебного контуров приложения. На диаграммах выделены функциональные блоки и номерами обозначены операции пользователей из раздела 4.2.2.

На рисунке 4.2.3.1 представлена навигационная схема, разработанная для WPF-приложения сотрудников. Она демонстрирует структуру экранов, используемых администратором, поваром и бариста при работе с заказами, меню, складом и отчетностью.

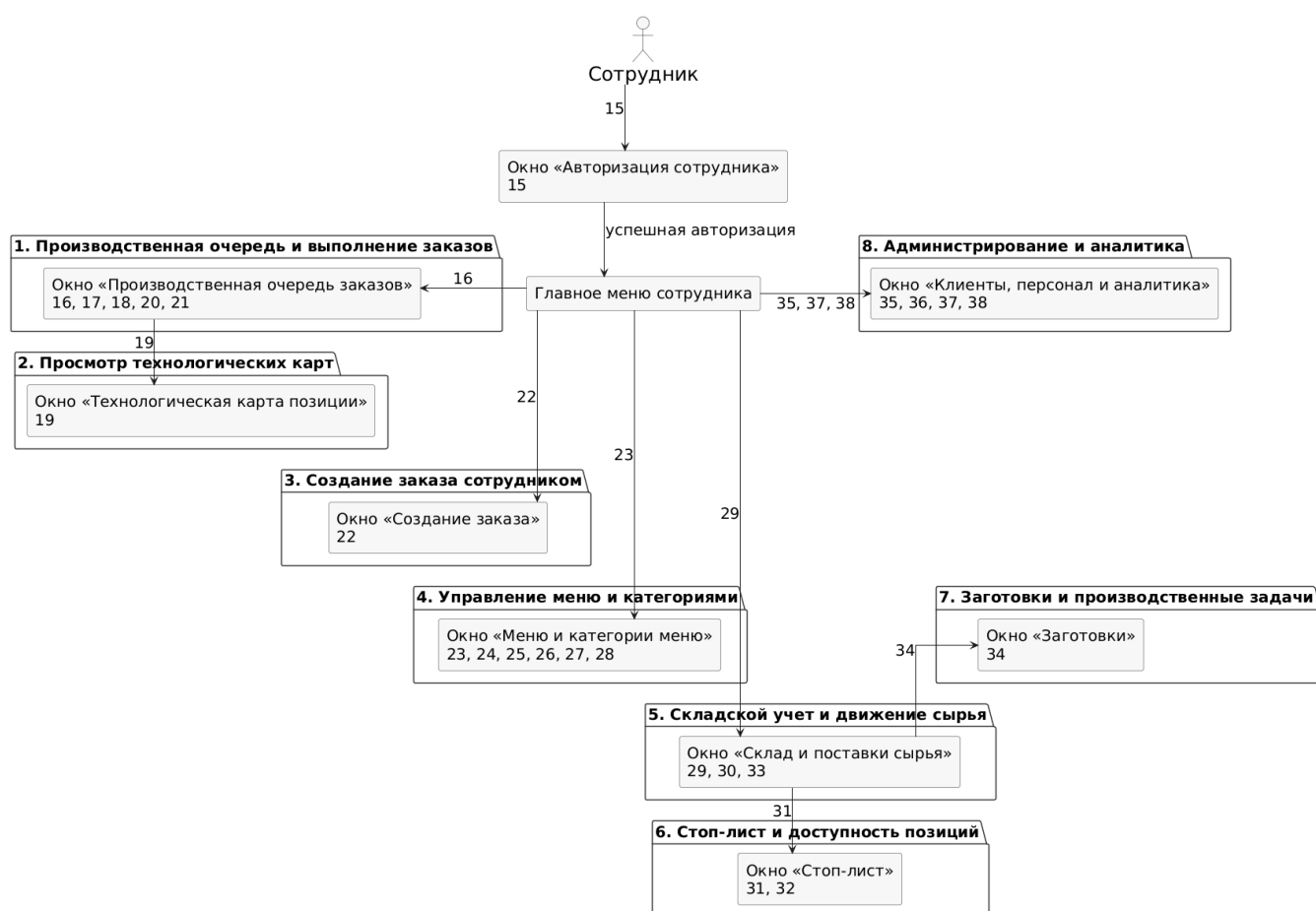


Рисунок 4.2.3.1 – Навигационная схема WPF-приложения сотрудников

Схема показывает работу сотрудника во внутреннем WPF-приложении после успешной авторизации. Из главного меню сотрудник переходит к основным разделам системы: производственной очереди, технологическим картам, созданию заказов, управлению меню, складу, стоп-листу, заготовкам, персоналу и аналитике. Таким образом, схема отражает навигацию сотрудника между функциональными окнами программного комплекса и связь каждого раздела с соответствующими задачами.

Для клиентского веб-интерфейса программного средства также создается навигационная схема, отражающая последовательность переходов между авторизацией, меню, корзиной, оформлением заказа и личным кабинетом. Навигационная схема, представленная на рисунке 4.2.3.2, демонстрирует распределение основных экранов клиента и переходы, выполняемые при оформлении заказа.

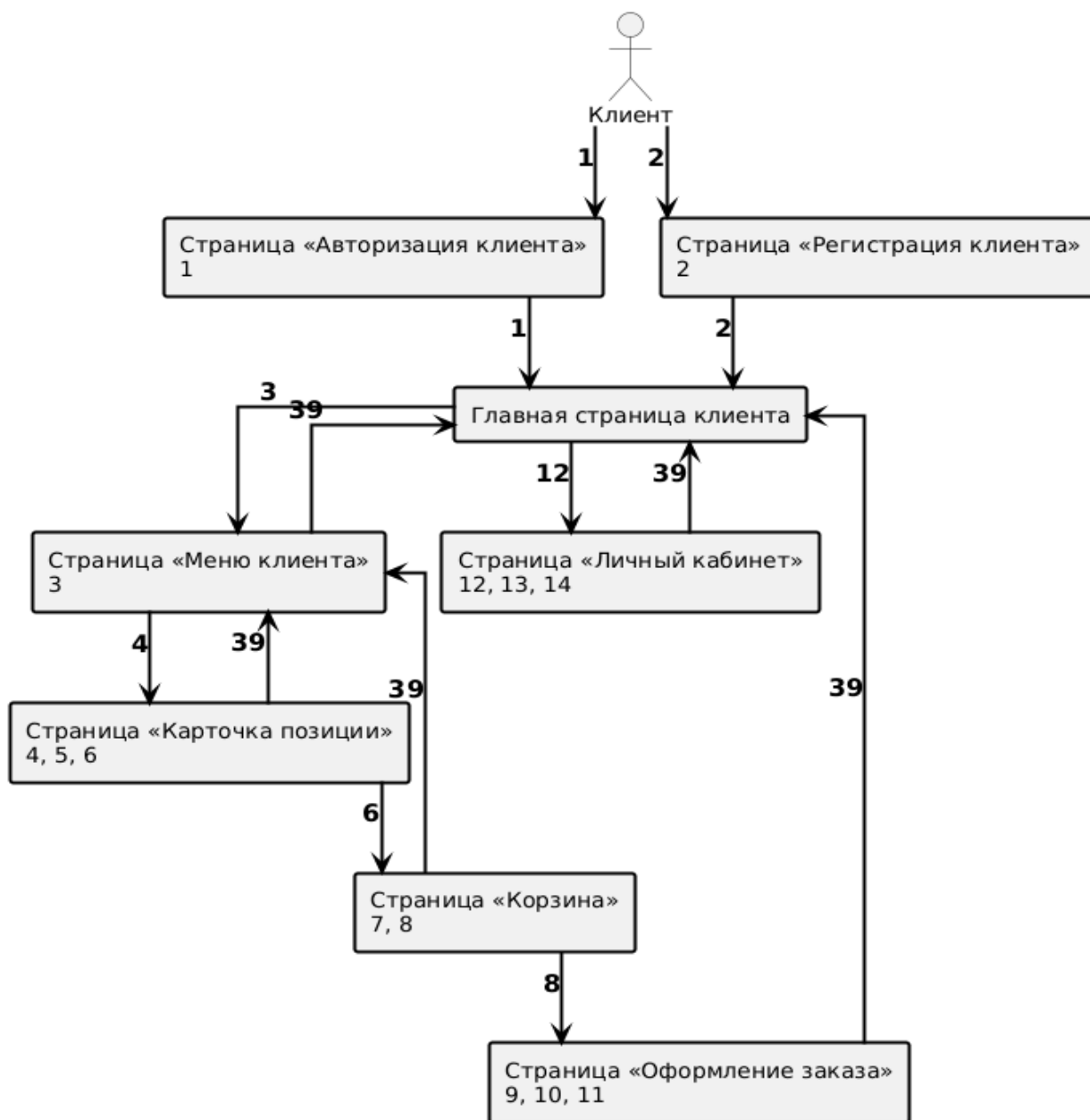


Рисунок 4.2.3.2 – Навигационная схема веб-интерфейса клиента

Схема показывает навигацию клиента в веб-части программного комплекса. Клиент может авторизоваться или зарегистрироваться, после чего переходит на главную страницу, откуда доступны меню, карточки позиций, корзина, оформление заказа и личный кабинет. Таким образом, схема отражает основной путь клиента: просмотр ассортимента, выбор позиции, добавление в корзину, оформление заказа и возврат к главным разделам сайта.

4.2.4 Разработка дизайна интерфейса

В ходе разработки программного комплекса проектируется два интерфейса – веб-интерфейс клиента и WPF-приложение сотрудника, каждый из которых включает окна с различными функциями.

Для обеспечения визуальной целостности и комфорта при длительной работе используется спокойная природная цветовая палитра, читаемые шрифты и контрастные элементы управления:

Основные цвета:

- Оливково-зеленый – #91A56E (панель навигации, основные акценты и кнопки).
- Темно-зеленый – #606E52 и #1E5928 (границы, активные состояния и подтверждающие действия).
- Белый – #FFFFFF (основной фон модальных окон и карточек).
- Светло-серый – #F5F5F5 и #F0F0F0 (фон клиентских страниц и неактивные области).
- Темно-серый – #474747 (основной цвет текста).
- Темно-красный – #742C27 (удаление, отмена и сообщения об ошибках).

Шрифты:

- Веб-интерфейс – Segoe UI для меню и Arial для экранов авторизации и регистрации.
- WPF-интерфейс – стандартный Segoe UI, крупные заголовки 30 px, кнопки 18-22 px, текст карточек и таблиц 17-18 px.

Все элементы интерфейса (кнопки, поля ввода, карточки, таблицы, статусы и модальные окна) выполнены в едином визуальном стиле. Это обеспечивает простоту восприятия, визуальную согласованность и удобство взаимодействия на всех этапах работы. Макеты окон программного комплекса показаны на рисунках 4.2.4.1 – 4.2.4.4.



Рисунок 4.2.4.1 – Макет главного экрана WPF-приложения

На главном окне WPF-приложения слева расположен перечень разделов, доступных сотруднику в соответствии с его ролью, а справа – основной функционал программного средства. Пользователь может выбрать необходимый модуль для перехода к соответствующему функционалу.

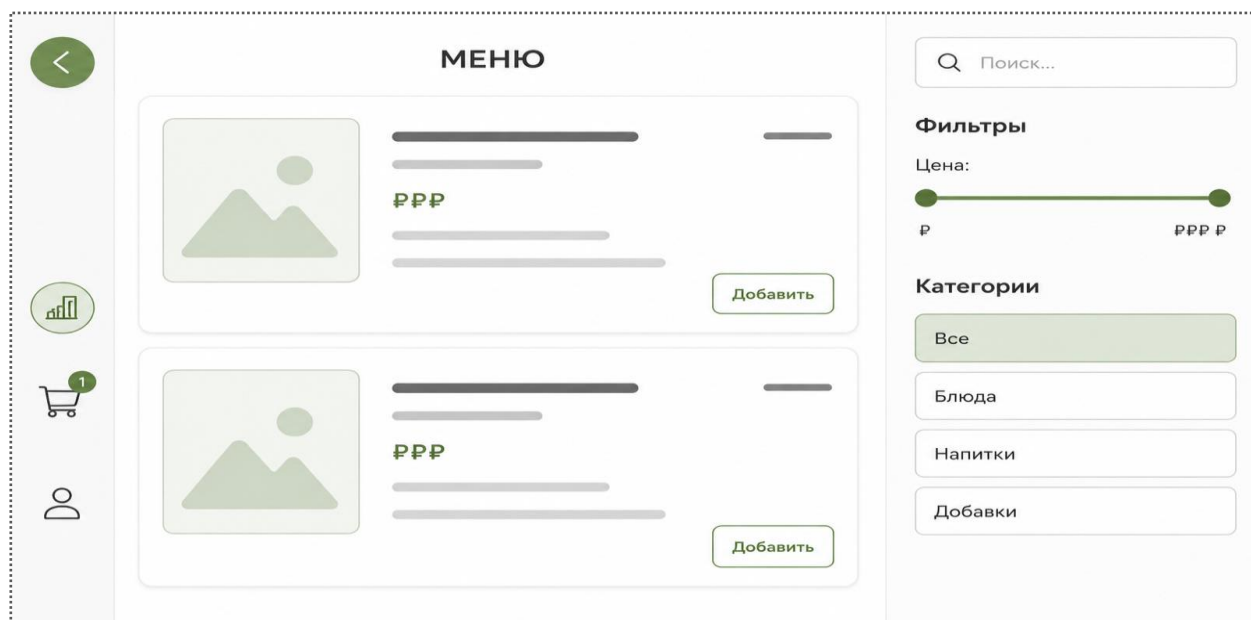


Рисунок 4.2.4.2 – Макет веб-интерфейса клиента с карточками меню

Веб-интерфейс клиента предназначен для выбора позиций меню, просмотра состава, цены, пищевой ценности и доступных добавок. Карточная структура позволяет быстро собрать заказ и перейти в корзину.

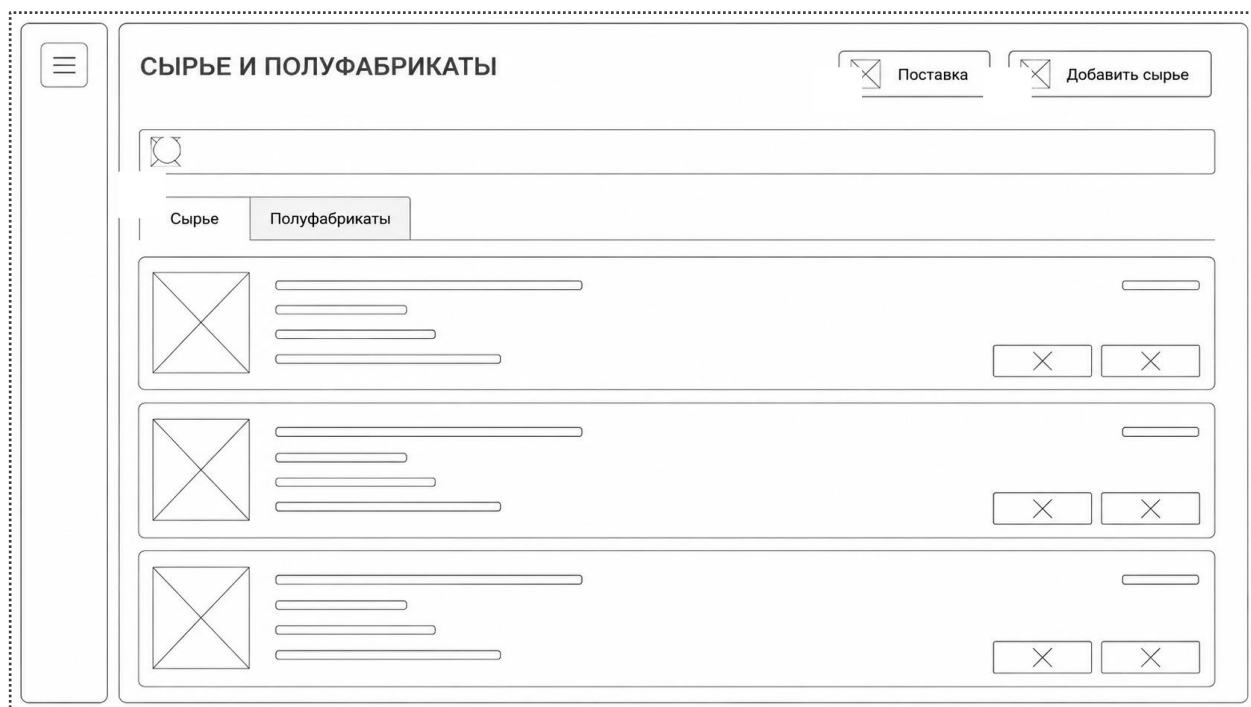


Рисунок 4.2.4.3 – Макет окна управления данными

Окно управления данными предназначено для просмотра, добавления, редактирования и удаления записей в выбранном разделе базы данных. В рабочей области отображаются таблицы или карточки записей, а рядом располагаются основные действия пользователя:

- «Добавить» – для создания новой записи.
- «Изменить» – для редактирования выбранной записи.
- «Удалить» – для удаления выбранной записи.
- «Обновить» или автоматическая перезагрузка списка – для получения актуальных данных с сервера.

Для удобства работы с большими объемами данных в разделах используются поиск, фильтрация по статусам или категориям, а также визуальное выделение важных показателей.

Макет окна «Редактировать сотрудника» представляет собой веб-интерфейс с заголовком «Редактировать сотрудника» и кнопкой закрытия. Основная область содержит поля для ввода: «ФИО сотрудника», «Логин», «Роль» (выпадающий список) и «Новый пароль». Под полем «Новый пароль» находится подсказка: «При редактировании оставьте поле пустым, чтобы сохранить текущий пароль». В нижней части окна расположены кнопки «Сохранить» и «Отмена».

Рисунок 4.2.4.4 – Макет окна изменения или добавления данных

Окно изменения или добавления записи предназначено для ввода и проверки данных перед сохранением. В данном окне пользователь изменяет значения полей, выбирает элементы справочников и подтверждает действие. В нижней части окна расположены кнопки «Сохранить» или «Добавить» для подтверждения изменений и «Отмена» для закрытия окна без сохранения.

4.3 Входные, выходные и промежуточные данные

4.3.1 Входные данные

Входные данные формируются пользователями в веб-интерфейсе и WPF-приложении, а также поступают из справочников системы при выполнении операций. Перед отправкой на сервер они проходят проверку обязательных полей, допустимых значений и прав пользователя.

Таблица 4.3.1 – Структура и форматы входных данных

Категория данных	Структура	Формат	Назначение
Учетные данные	Логин, пароль, роль пользователя	JSON / HTTP POST	Авторизация клиента или сотрудника.
Данные заказа	Позиции, количество, добавки, модификаторы, комментариев, тип заказа, время самовывоза	JSON / HTTP POST	Создание заказа и передача его в производственную очередь.
Справочники меню	Название, категория, цена, состав, доступность, КБЖУ	JSON / формы WPF	Создание и редактирование позиций меню.
Ингредиенты и техкарты	Сырье, масса, единицы измерения, пищевая ценность, технология приготовления	JSON / формы WPF	Расчет КБЖУ, себестоимости и списание сырья.
Складские операции	Поставка, списание, стоп-лист, причина операции	JSON / формы WPF	Контроль остатков и доступности позиций.

4.3.2 Выходные данные

Выходные данные формируются серверной частью и отображаются в интерфейсах пользователей. Они используются для подтверждения выполненных операций, отображения статусов, формирования отчетов и обновления рабочих экранов.

Таблица 4.3.2 – Структура и форматы выходных данных

Категория данных	Структура	Формат	Назначение
Ответ авторизации	Статус входа, cookie-сессия, роль пользователя	HTTP response / cookie	Открытие соответствующего контура системы.
Результат оформления заказа	Номер заказа, сумма, скидка, статус, состав заказа	JSON	Подтверждение заказа клиенту и передача в очередь кухни.
Очередь заказов	Список заказов, позиции, комментарии, время подачи, статусы	JSON	Работа повара и бариста-кассира.
Справочные списки	Меню, ингредиенты, категории, клиенты, сотрудники	JSON / таблицы WPF	Заполнение экранов управления и форм редактирования.
Отчеты и аналитика	Продажи, популярность позиций, ABC-анализ, инвентаризация	JSON / графики / таблицы	Поддержка управленческих решений.

4.3.3 Промежуточные данные

Промежуточные данные существуют во время выполнения пользовательских сценариев и не всегда сохраняются как самостоятельные сущности. Они нужны для удержания состояния интерфейса, проверки операций и подготовки итоговых запросов к API.

Таблица 4.3.3 – Структура промежуточных данных

Категория	Состав	Место хранения	Назначение
Корзина клиента	Состав заказа до подтверждения, количество, добавки, комментарий	localStorage / состояние страницы	Сохранение выбранных позиций до отправки заказа.
DTO-модели	OrderRequest, OrderResponse, ProfileResponse и другие контракты	Память приложения / JSON	Передача данных между клиентом и сервером.
Расчетные показатели	КБЖУ, себестоимость, цена, скидка, итоговая сумма	Память серверного приложения	Формирование результата заказа и карточек позиций.
Состояния интерфейса	Выбранный раздел, фильтры, поиск, статус загрузки и ошибки	Память клиента	Обеспечение удобной работы без лишних перезагрузок.
Транзакционные данные	Временный набор изменений заказа и складских остатков	Транзакция БД	Атомарное сохранение критичных операций.

4.4 Реализация используемых методов хранения, обработки и передачи информации об объектах предметной области

4.4.1 Методы хранения данных

Для оптимального хранения и управления данными в программном комплексе используется реляционная база данных Microsoft SQL Server. Работа с базой данных выполняется через Entity Framework Core, благодаря чему приложение взаимодействует с данными на уровне объектов, без необходимости вручную писать SQL-запросы. Основные таблицы отражают устойчивые сущности предметной области: клиентов, сотрудников, меню, ингредиенты, технологические карты, заказы, складские операции, заготовки, списания и справочники. Связи между таблицами обеспечивают целостность рецептов, заказов и складского учета. В таблицах 4.4.1.1 – 4.4.1.37 приведено подробное описание каждой таблицы базы данных. [11], [10]

Таблица 4.4.1.1 – Таблица «ClientCategories»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(50)	Наименование записи

Таблица 4.4.1.2 – Таблица «Clients»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
FullName	nvarchar(255)	ФИО пользователя или сотрудника
PhoneNumber	nvarchar(20)	Номер телефона клиента
Password	nvarchar(100)	Пароль учетной записи
ClientCategoryId	int	Внешний ключ к таблице «ClientCategories»
OrderCount	int	Количество заказов клиента

Таблица 4.4.1.3 – Таблица «Discounts»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(50)	Наименование записи
DiscountPercent	decimal(5, 2)	Процент скидки

Таблица 4.4.1.4 – Таблица «DishCategories»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(50)	Наименование записи

Таблица 4.4.1.5 – Таблица «Dishes»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(255)	Наименование записи
CategoryId	int	Внешний ключ к таблице «DishCategories»
UnitOfMeasureId	int	Внешний ключ к таблице «UnitsOfMeasure»
CostRub	decimal(10, 2)	Себестоимость в рублях
MarkupPercent	decimal(10, 2)	Процент наценки
PriceRub	decimal(10, 2)	Цена продажи в рублях
CostPercent	decimal(5, 2)	Доля себестоимости в цене
TechnicalCardId	int	Внешний ключ к таблице «TechnicalCards»
FatsG	decimal(8, 2)	Количество жиров в граммах
ProteinsG	decimal(8, 2)	Количество белков в граммах
CarbsG	decimal(8, 2)	Количество углеводов в граммах
CaloriesKcal	decimal(8, 2)	Калорийность в килокалориях
Kilojoules	decimal(8, 2)	Энергетическая ценность в килоджоулях
ImageUrl	nvarchar(500)	Ссылка на изображение позиции
IsAvailable	bit	Признак доступности позиции для заказа

Таблица 4.4.1.6 – Таблица «DishToppings»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
ToppingId	int	Внешний ключ к таблице «ToppingsAndSyrups»
OrderDishItemId	int	Внешний ключ к таблице «OrderDishItems»
Quantity	decimal(8, 2)	Количество позиции
FinalPrice	decimal(10, 2)	Итоговая цена позиции

Таблица 4.4.1.7 – Таблица «DrinkCategories»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(50)	Наименование записи

Таблица 4.4.1.8 – Таблица «Drinks»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(255)	Наименование записи
Quantity	decimal(8, 2)	Количество позиции
UnitOfMeasureId	int	Внешний ключ к таблице «UnitsOfMeasure»
CategoryId	int	Внешний ключ к таблице «DrinkCategories»
CostRub	decimal(10, 2)	Себестоимость в рублях
MarkupPercent	decimal(10, 2)	Процент наценки
PriceRub	decimal(10, 2)	Цена продажи в рублях
CostPercent	decimal(5, 2)	Доля себестоимости в цене
TechnicalCardId	int	Внешний ключ к таблице «TechnicalCards»
FatsG	decimal(8, 2)	Количество жиров в граммах
ProteinsG	decimal(8, 2)	Количество белков в граммах
CarbsG	decimal(8, 2)	Количество углеводов в граммах
CaloriesKcal	decimal(8, 2)	Калорийность в килокалориях
Kilojoules	decimal(8, 2)	Энергетическая ценность в килоджоулях
ImageUrl	nvarchar(500)	Ссылка на изображение позиции
IsAvailable	bit	Признак доступности позиции для заказа

Таблица 4.4.1.9 – Таблица «DrinkToppings»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
ToppingId	int	Внешний ключ к таблице «ToppingsAndSyrups»
OrderDrinkItemId	int	Внешний ключ к таблице «OrderDrinkItems»
Quantity	decimal(8, 2)	Количество позиции
FinalPrice	decimal(10, 2)	Итоговая цена позиции

Таблица 4.4.1.10 – Таблица «IngredientCategories»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(50)	Наименование записи

Таблица 4.4.1.11 – Таблица «Ingredients»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(255)	Наименование записи
Stock	decimal(10, 2)	Текущий остаток ингредиента
UnitOfMeasureId	int	Внешний ключ к таблице «UnitsOfMeasure»
CostRub	decimal(10, 2)	Себестоимость в рублях
CategoryId	int	Внешний ключ к таблице «IngredientCategories»

Таблица 4.4.1.12 – Таблица «IngredientSupplyActItems»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
SupplyActId	int	Внешний ключ к таблице «IngredientSupplyActs»
IngredientId	int	Внешний ключ к таблице «Ingredients»
Quantity	decimal(10, 2)	Количество позиции
UnitOfMeasureId	int	Внешний ключ к таблице «UnitsOfMeasure»

Таблица 4.4.1.13 – Таблица «IngredientSupplyActs»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Date	datetime2(7)	Дата операции или документа

Таблица 4.4.1.14 – Таблица «IngredientWriteOffActItems»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
WriteOffActId	int	Внешний ключ к таблице «WriteOffActs»
IngredientId	int	Внешний ключ к таблице «Ingredients»
Quantity	decimal(10, 2)	Количество позиции
UnitOfMeasureId	int	Внешний ключ к таблице «UnitsOfMeasure»
WriteOffTypeId	int	Внешний ключ к таблице «WriteOffTypes»

Таблица 4.4.1.15 – Таблица «MenuItemPortionLimits»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
ItemType	nvarchar(16)	Тип позиции меню
ItemId	int	Идентификатор позиции меню выбранного типа
RemainingPortions	decimal (10, 2)	Оставшееся количество доступных порций
CreatedAt	datetime2(7)	Дата и время создания записи
UpdatedAt	datetime2(7)	Дата и время последнего обновления записи

Таблица 4.4.1.16 – Таблица «OrderDishItems»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
OrderId	int	Внешний ключ к таблице «Orders»
DishId	int	Внешний ключ к таблице «Dishes»
Quantity	decimal (8, 2)	Количество позиции
FinalPrice	decimal (10, 2)	Итоговая цена позиции
IsCompleted	bit	Признак выполнения позиции заказа

Таблица 4.4.1.17 – Таблица «OrderDrinkItemModifiers»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
OrderDrinkItemId	int	Внешний ключ к таблице «OrderDrinkItems»
MilkIngredientId	int	Внешний ключ к таблице «Ingredients»
CoffeeIngredientId	int	Внешний ключ к таблице «Ingredients»

Таблица 4.4.1.18 – Таблица «OrderDrinkItems»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
OrderId	int	Внешний ключ к таблице «Orders»
DrinkId	int	Внешний ключ к таблице «Drinks»
Quantity	decimal(8, 2)	Количество позиции
FinalPrice	decimal(10, 2)	Итоговая цена позиции
IsCompleted	bit	Признак выполнения позиции заказа

Таблица 4.4.1.19 – Таблица «Orders»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
CreatedAt	datetime2(7)	Дата и время создания записи
ClientId	int	Внешний ключ к таблице «Clients»
OrderTypeId	int	Внешний ключ к таблице «OrderTypes»
Comment	nvarchar(max)	Комментарий к записи или документу
TotalCalories	decimal(10, 2)	Итоговая калорийность заказа
DiscountId	int	Внешний ключ к таблице «Discounts»
TotalPrice	decimal(10, 2)	Итоговая стоимость
StatusID	int	Внешний ключ к таблице «OrderStatuses»
PickupAt	datetime2(7)	Плановое время получения заказа

Таблица 4.4.1.20 – Таблица «OrderStatuses»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(50)	Наименование записи

Таблица 4.4.1.21 – Таблица «OrderToppingItems»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
OrderId	int	Внешний ключ к таблице «Orders»
ToppingId	int	Внешний ключ к таблице «ToppingsAndSyrups»
Quantity	int	Количество позиции
TotalPrice	decimal(18, 2)	Итоговая стоимость
IsCompleted	bit	Признак выполнения позиции заказа

Таблица 4.4.1.22 – Таблица «OrderTypes»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(50)	Наименование записи

Таблица 4.4.1.23 – Таблица «Preparations»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(255)	Наименование записи
SemiFinishedId	int	Внешний ключ к таблице «SemiFinished»
StockGrams	decimal(10, 2)	Остаток заготовки в граммах
ProductionDate	date	Дата изготовления заготовки

Таблица 4.4.1.24 – Таблица «PreparationTasks»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
SemiFinishedId	int	Внешний ключ к таблице «SemiFinished»
Comment	nvarchar(500)	Комментарий к записи или документу
CreatedAt	datetime2(7)	Дата и время создания записи
TaskText	nvarchar(255)	Текст задачи по приготовлению заготовки

Таблица 4.4.1.25 – Таблица «SemiFinished»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(255)	Наименование записи
CostRub	decimal(10, 2)	Себестоимость в рублях
CategoryId	int	Внешний ключ к таблице «SemiFinishedCategories»
UnitOfMeasureId	int	Внешний ключ к таблице «UnitsOfMeasure»
TechnicalCardId	int	Внешний ключ к таблице «TechnicalCards»
FatsG	decimal(8, 2)	Количество жиров в граммах
ProteinsG	decimal(8, 2)	Количество белков в граммах
CarbsG	decimal(8, 2)	Количество углеводов в граммах
CaloriesKcal	decimal(8, 2)	Калорийность в килокалориях
Kilojoules	decimal(8, 2)	Энергетическая ценность в килоджоулях

Таблица 4.4.1.26 – Таблица «SemiFinishedCategories»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(50)	Наименование записи

Таблица 4.4.1.27 – Таблица «SemiFinishedWriteOffActItems»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
WriteOffActId	int	Внешний ключ к таблице «WriteOffActs»
SemiFinishedId	int	Внешний ключ к таблице «SemiFinished»
Quantity	decimal(10, 2)	Количество позиции
UnitOfMeasureId	int	Внешний ключ к таблице «UnitsOfMeasure»
WriteOffTypeId	int	Внешний ключ к таблице «WriteOffTypes»

Таблица 4.4.1.28 – Таблица «Staff»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
RoleId	int	Внешний ключ к таблице «StaffRoles»
FullName	nvarchar(255)	ФИО пользователя или сотрудника
Login	nvarchar(100)	Логин сотрудника для входа в систему
Password	nvarchar(100)	Пароль учетной записи

Таблица 4.4.1.29 – Таблица «StaffRoles»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(50)	Наименование записи

Таблица 4.4.1.30 – Таблица «TechnicalCardIngredientComposition»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
TechnicalCardId	int	Внешний ключ к таблице «TechnicalCards»
IngredientId	int	Внешний ключ к таблице «Ingredients»
UnitOfMeasureId	int	Внешний ключ к таблице «UnitsOfMeasure»
GrossWeight	decimal(10, 6)	Масса брутто по технологической карте
ColdLossPercent	decimal(10, 2)	Процент потерь при холодной обработке
NetWeight	decimal(10, 6)	Масса нетто после холодной обработки
HotLossPercent	decimal(10, 2)	Процент потерь при тепловой обработке
OutputWeight	decimal(10, 6)	Итоговый выход продукта

Таблица 4.4.1.31 – Таблица «TechnicalCards»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(255)	Наименование записи
Description	nvarchar(max)	Описание записи

Таблица 4.4.1.32 – Таблица «TechnicalCardSemiFinishedComposition»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
TechnicalCardId	int	Внешний ключ к таблице «TechnicalCards»
SemiFinishedId	int	Внешний ключ к таблице «SemiFinished»
UnitOfMeasureId	int	Внешний ключ к таблице «UnitsOfMeasure»
GrossWeight	decimal(10, 6)	Масса брутто по технологической карте
ColdLossPercent	decimal(10, 2)	Процент потерь при холодной обработке
NetWeight	decimal(10, 6)	Масса нетто после холодной обработки
HotLossPercent	decimal(10, 2)	Процент потерь при тепловой обработке
OutputWeight	decimal(10, 6)	Итоговый выход продукта

Таблица 4.4.1.33 – Таблица «ToppingCategories»

Имя	Тип	Описание
id	int IDENTITY(1,1)	Идентификатор записи
name	nvarchar(max)	Наименование записи

Таблица 4.4.1.34 – Таблица «ToppingsAndSyrups»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(255)	Наименование записи
Quantity	decimal(8, 2)	Количество позиции
UnitOfMeasureId	int	Внешний ключ к таблице «UnitsOfMeasure»
CostRub	decimal(10, 2)	Себестоимость в рублях
MarkupPercent	decimal(10, 2)	Процент наценки
PriceRub	decimal(10, 2)	Цена продажи в рублях
CostPercent	decimal(5, 2)	Доля себестоимости в цене
TechnicalCardId	int	Внешний ключ к таблице «TechnicalCards»
FatsG	decimal(8, 2)	Количество жиров в граммах
ProteinsG	decimal(8, 2)	Количество белков в граммах
CarbsG	decimal(8, 2)	Количество углеводов в граммах
CaloriesKcal	decimal(8, 2)	Калорийность в килокалориях
Kilojoules	decimal(8, 2)	Энергетическая ценность в килоджоулях
CategoryID	int	Внешний ключ к таблице «ToppingCategories»
IsAvailable	bit	Признак доступности позиции для заказа

Таблица 4.4.1.35 – Таблица «UnitsOfMeasure»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(50)	Наименование записи

Таблица 4.4.1.36 – Таблица «WriteOffActs»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Date	datetime2(7)	Дата операции или документа
Comment	nvarchar(500)	Комментарий к записи или документу
StaffId	int	Внешний ключ к таблице «Staff»

Таблица 4.4.1.37 – Таблица «WriteOffTypes»»

Имя	Тип	Описание
Id	int IDENTITY(1,1)	Идентификатор записи
Name	nvarchar(200)	Наименование записи

Основные таблицы базы данных и связи один ко многим между ними представлены на схеме, отражающей структуру хранения данных и логические отношения между сущностями предметной области. Схема на рисунке 4.4.1.1 показывает, как организованы данные в системе для обеспечения целостности, удобного доступа и эффективной работы программного комплекса. Полная ER-диаграмма представлена в Приложении Г.

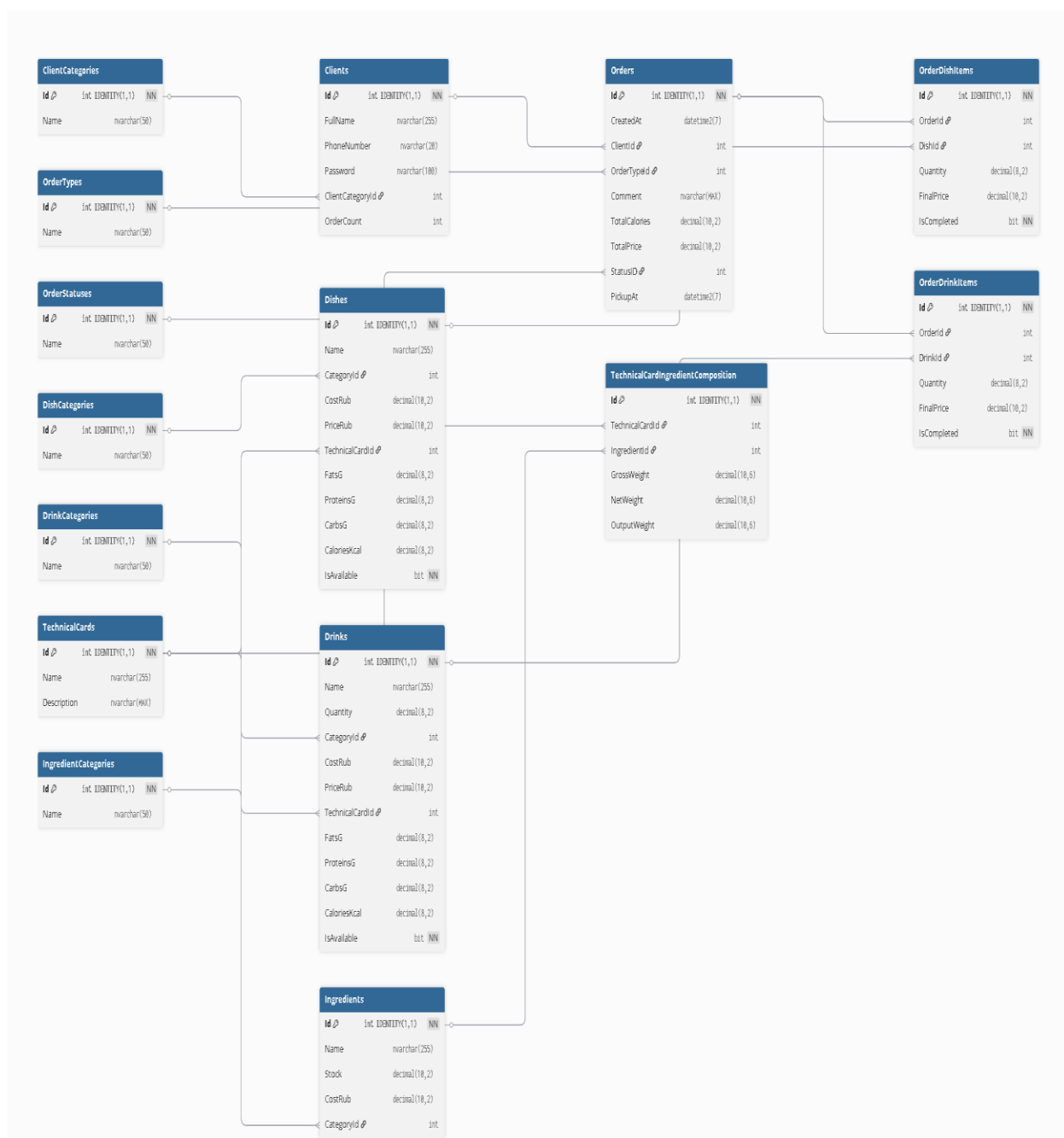


Рисунок 4.4.1.1 – ER-диаграмма базы данных Garden Nook

4.4.2 Алгоритмы реализации используемых формул расчета

В программной реализации Garden Nook расчетные алгоритмы применяются при формировании карточек меню, оформлении заказа, определении расчетного веса компонентов технологической карты и построении отчетов по складским остаткам. Основная логика находится на стороне серверного приложения GardenNookApi, прежде всего в OrdersController, а также в сервисах IPreparationStockService и IPickupSchedulingService. Данные передаются через DTO-модели проекта TransferModels, например, OrderRequest, OrderResponse, DiscountDto, а сохранение выполняется через Entity Framework Core и контекст ApplicationDbContext.

Исходными данными для расчетов являются поля PriceRub, CaloriesKcal, Quantity, OutputWeight, NetWeight, GrossWeight, CostRub, сведения о скидках и складских остатках. Они загружаются из сущностей Dish, Drink, ToppingsAndSyrup, Ingredient, SemiFinishedProduct, Order, OrderDishItem и OrderDrinkItem. Для денежных и весовых значений используется тип decimal, а при отсутствии данных применяются безопасные значения по умолчанию, например, (dish.PriceRub ?? 0m).

4.4.2.1 Алгоритм определения рабочего веса компонента технологической карты

Определение рабочего веса компонента технологической карты выполняется по приоритету: сначала используется OutputWeight, затем NetWeight, затем GrossWeight. Такой порядок позволяет брать наиболее точное значение, но не останавливать расчет при частично заполненной рецептуре. Данная логика применяется при анализе состава блюд, расчете выхода и складских операциях, связанных с методами TryConsumeForOrderAsync и RestoreConsumedForOrderAsync. На рисунке 4.4.2.1.1 представлен код с применением алгоритма. Алгоритм описан на схеме, представленной на рисунке 4.4.2.1.2.

```
private static decimal PickComponentWeight(decimal? outputWeight, decimal? netWeight, decimal? grossWeight)
{
    if (outputWeight.HasValue && outputWeight.Value > 0)
    {
        return outputWeight.Value;
    }

    if (netWeight.HasValue && netWeight.Value > 0)
    {
        return netWeight.Value;
    }

    if (grossWeight.HasValue && grossWeight.Value > 0)
    {
        return grossWeight.Value;
    }

    return 0m;
}
```

Рисунок 4.4.2.1.1 – Код алгоритма определения веса компонента

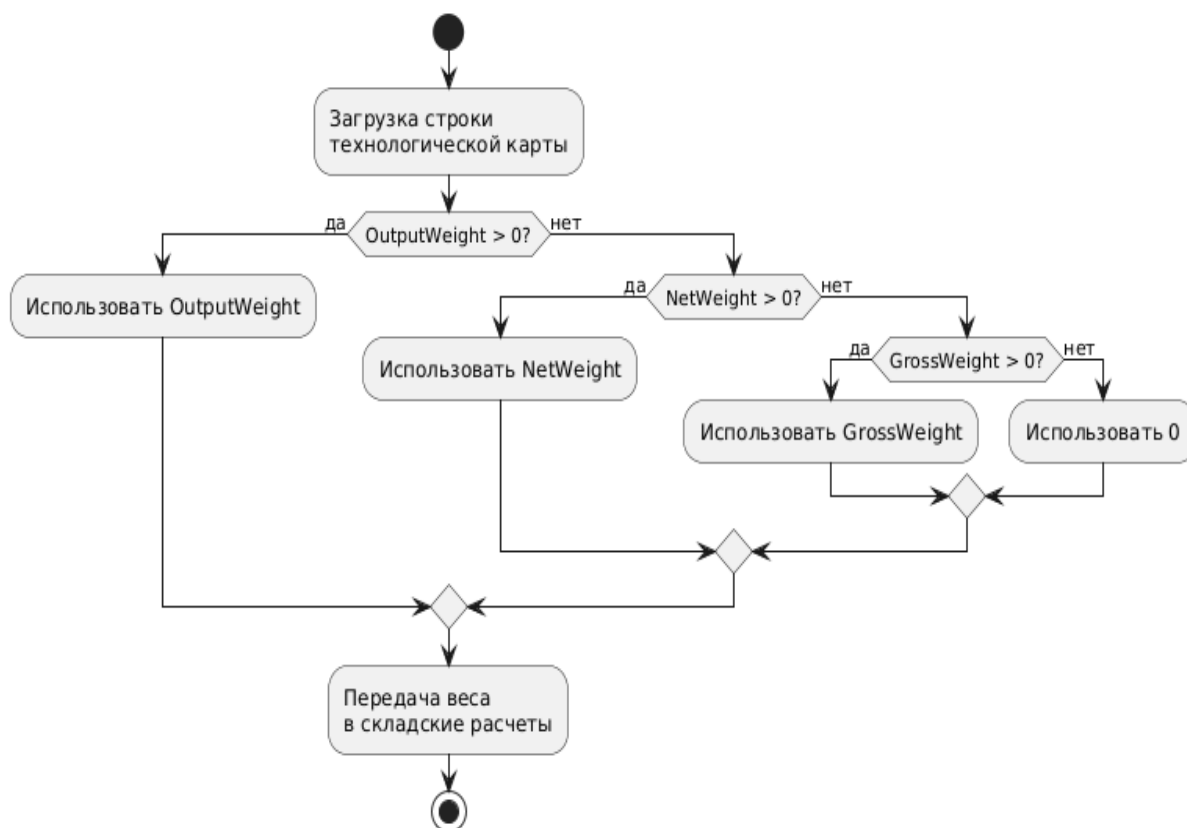


Рисунок 4.4.2.1.2 – Алгоритм определения веса компонента технологической карты

4.4.2.2 Алгоритм расчета складских остатков

Складские расчеты связаны с методами TryConsumeForOrderAsync, RefreshMenuAvailabilityAsync, TryConsumePortionLimitsAsync и RestorePortionLimitsAsync. Система определяет расчетный расход сырья по заказам, технологическим картам, списаниям и полуфабрикатам, затем сравнивает его с фактически остатками. Учет поля CostRub позволяет получать не только количественное, но и денежное выражение отклонений. Метод TryConsumeForOrderAsync выполняет проверку доступности требуемого сырья на складе на основе технологических карт и компонентов заказа. После успешного резервирования сырья запускается RefreshMenuAvailabilityAsync, которая пересчитывает доступность каждой позиции меню с учетом текущих остатков и новых резервирований. При ошибке выполнения заказа метод RestorePortionLimitsAsync автоматически восстанавливает все остатки порций и сырья. Таким образом, алгоритмы Create, ApplyDiscount, RebuildOrderCompositionAsync и складские сервисы связывают меню, заказ, технологические карты и складской учет в единую расчетную цепочку. На рисунке 4.4.2.2.1 представлен код метода RestorePortionLimitsAsync (), который восстанавливает остаток порций позиции при отмене заказа. Алгоритм описан на схеме, представленной на рисунке 4.4.2.2.2.

```

private async Task RestorePortionLimitsAsync(OrderRequest savedOrder)
{
    var requirements = BuildPortionLimitRequirements(savedOrder);
    if (requirements.Count == 0)
    {
        return;
    }

    var keys = requirements.Keys.ToList();
    var itemTypes = keys.Select(x => x.ItemType).Distinct().ToList();
    var itemIds = keys.Select(x => x.ItemId).Distinct().ToList();
    var rows = await _db.MenuItemPortionLimits
        .Where(x => itemTypes.Contains(x.ItemType) && itemIds.Contains(x.ItemId))
        .ToListAsync();
    var rowsByKey = rows.ToDictionary(x => new MenuItemLimitKey(x.ItemType.Trim().ToLowerInvariant(), x.ItemId), x => x);
    var now = DateTime.Now;

    foreach (var pair in requirements)
    {
        if (rowsByKey.TryGetValue(pair.Key, out var row))
        {
            row.RemainingPortions = Round2(Math.Max(0m, row.RemainingPortions) + pair.Value);
            row.UpdatedAt = now;
        }
    }
}

```

Рисунок 4.4.2.2.1 – Код метода RestorePortionLimitsAsync ()

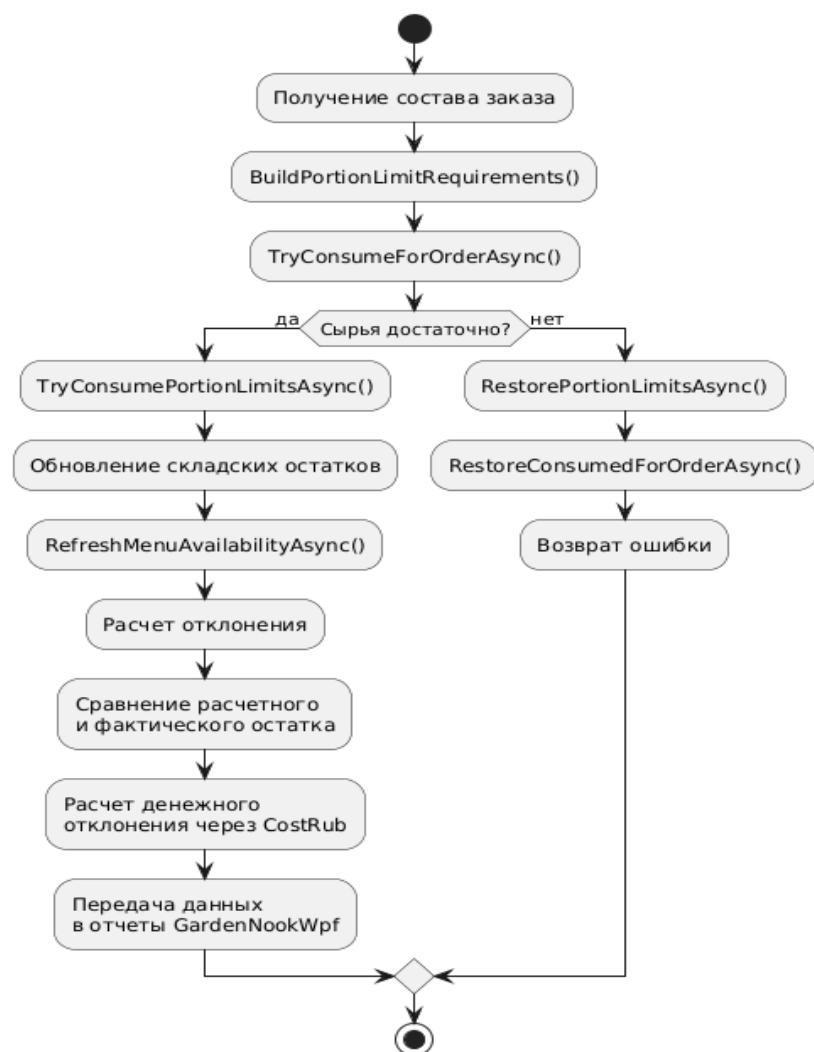


Рисунок 4.4.2.2.2 – Алгоритм расчета складских остатков

4.4.2.3 Алгоритм расчета заказа

Расчет заказа выполняется в методе Create контроллера OrdersController. Фрагмент кода метода Create представлен на рисунке 4.4.2.3.1 Сервер получает OrderRequest, проверяет количество позиций, загружает блюда, напитки и добавки через Entity Framework Core, а затем применяет методы ApplyDefaultDrinkModifiersAsync, ValidateDrinkModifiersAsync и BuildPortionLimitRequirements. Для каждой позиции в циклах foreach рассчитываются стоимость и калорийность: цена и КБЖУ умножаются на Quantity, а добавки дополнительно пересчитываются с учетом количества основной позиции.

После обработки всех строк заказа система суммирует значения в переменных totalPriceBeforeDiscount и totalCalories. Если для заказа предусмотрена скидка, она определяется методом ResolveDiscountForOrderAsync и применяется через ApplyDiscount. Итоговая стоимость округляется методом Round2 и сохраняется в order.TotalPrice, а калорийность в order.TotalCalories. При изменении заказа аналогичные действия выполняются в методах UpdateHistoryOrder и RebuildOrderCompositionAsync. Алгоритм описан на схеме, представленной на рисунке 4.4.2.3.2.

```
// ---- загрузим цены/ккал ----
var dishes = await _db.Dishes
    .Where(x => dishIds.Contains(x.Id))
    .Select(x => new
    {
        x.Id,
        x.Name,
        x.PriceRub,
        x.CaloriesKcal,
        CategoryName = x.Category != null ? x.Category.Name : null
    })
    .ToDictionaryAsync(x => x.Id, x => x);

var drinks = await _db.Drinks
    .Where(x => drinkIds.Contains(x.Id))
    .Select(x => new
    {
        x.Id,
        x.Name,
        x.PriceRub,
        x.CaloriesKcal,
        CategoryName = x.Category != null ? x.Category.Name : null
    })
    .ToDictionaryAsync(x => x.Id, x => x);

var toppings = await _db.ToppingsAndSyrups
    .Where(x => toppingIds.Contains(x.Id))
    .Select(x => new
    {
        x.Id,
        x.Name,
        x.PriceRub,
        x.CaloriesKcal,
        CategoryName = x.Category != null ? x.Category.Name : null
    })
    .ToDictionaryAsync(x => x.Id, x => x);
```

Рисунок 4.4.2.3.1 – Фрагмент кода метода Create ()

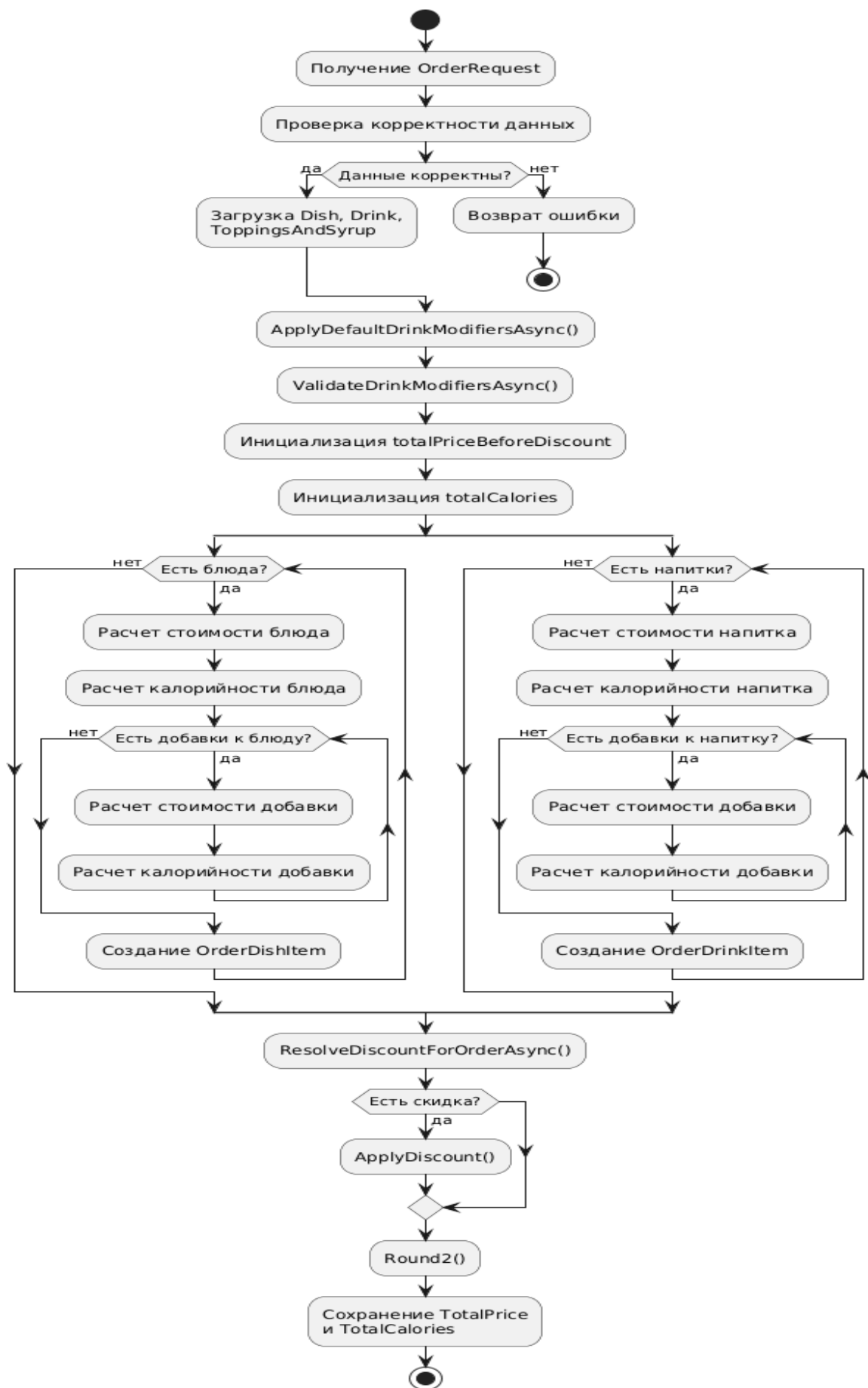


Рисунок 4.4.2.3 – Алгоритм расчета заказа

4.4.3 Алгоритмы применения методов графического анализа данных

В программной реализации Garden Nook графический способ представления и интерпретации информации используется в основном в административной части системы для анализа продаж, популярности позиций меню и состояния складских остатков. Основные данные для визуализации формируются на стороне серверного приложения GardenNookApi в контроллере ReportsController. При обращении к методу GetReports сервер вызывает внутренний метод BuildReportAsync, который собирает сведения за выбранный период и формирует набор данных для дальнейшего отображения в интерфейсе администратора.

Алгоритм подготовки данных для графического анализа выполняется поэтапно. Сначала пользователь выбирает отчетный период, после чего сервер определяет границы выборки и получает список заказов из таблицы Orders. Далее по связанным таблицам OrderDishItems, OrderDrinkItems, OrderToppingItems, DishToppings и DrinkToppings выбираются сведения о проданных блюдах, напитках и добавках. Для каждой позиции рассчитываются количество продаж и выручка. Эти данные используются методами BuildPopularItemsAsync, BuildUnpopularItemsAsync и BuildAbcItemsAsync, которые формируют списки популярных, непопулярных и значимых по выручке позиций меню.

Отдельный этап графического анализа связан со складскими остатками. Метод BuildInventoryItemsAsync формирует данные по сырью, учитывая расход по заказам, актам списания и заготовкам. Для этого используются вспомогательные операции расчета потребления ингредиентов, после чего система сравнивает ожидаемый расход с фактическим остатком. В результате администратор получает данные о количестве расхождения и его денежном выражении через поле CostRub. Такие показатели удобно отображать в виде таблицы отклонений или диаграммы, где по вертикали указываются ингредиенты, а по горизонтали – величина недостачи или излишка. Полный алгоритм представлен на рисунке 4.4.3.1. [12]

Для данной предметной области наиболее предпочтительными являются несколько видов визуализации. Столбчатые диаграммы целесообразно использовать для сравнения количества проданных блюд и напитков, так как они позволяют быстро определить наиболее востребованные позиции меню. Таблично-графическое представление удобно для ABC-анализа, где позиции сортируются по убыванию выручки и распределяются по степени влияния на финансовый результат. Для складских отчетов предпочтительны таблицы с цветовым выделением отклонений, поскольку администратору важно видеть не только абсолютное количество сырья, но и проблемные позиции, по которым расчетный и фактический остаток расходятся.

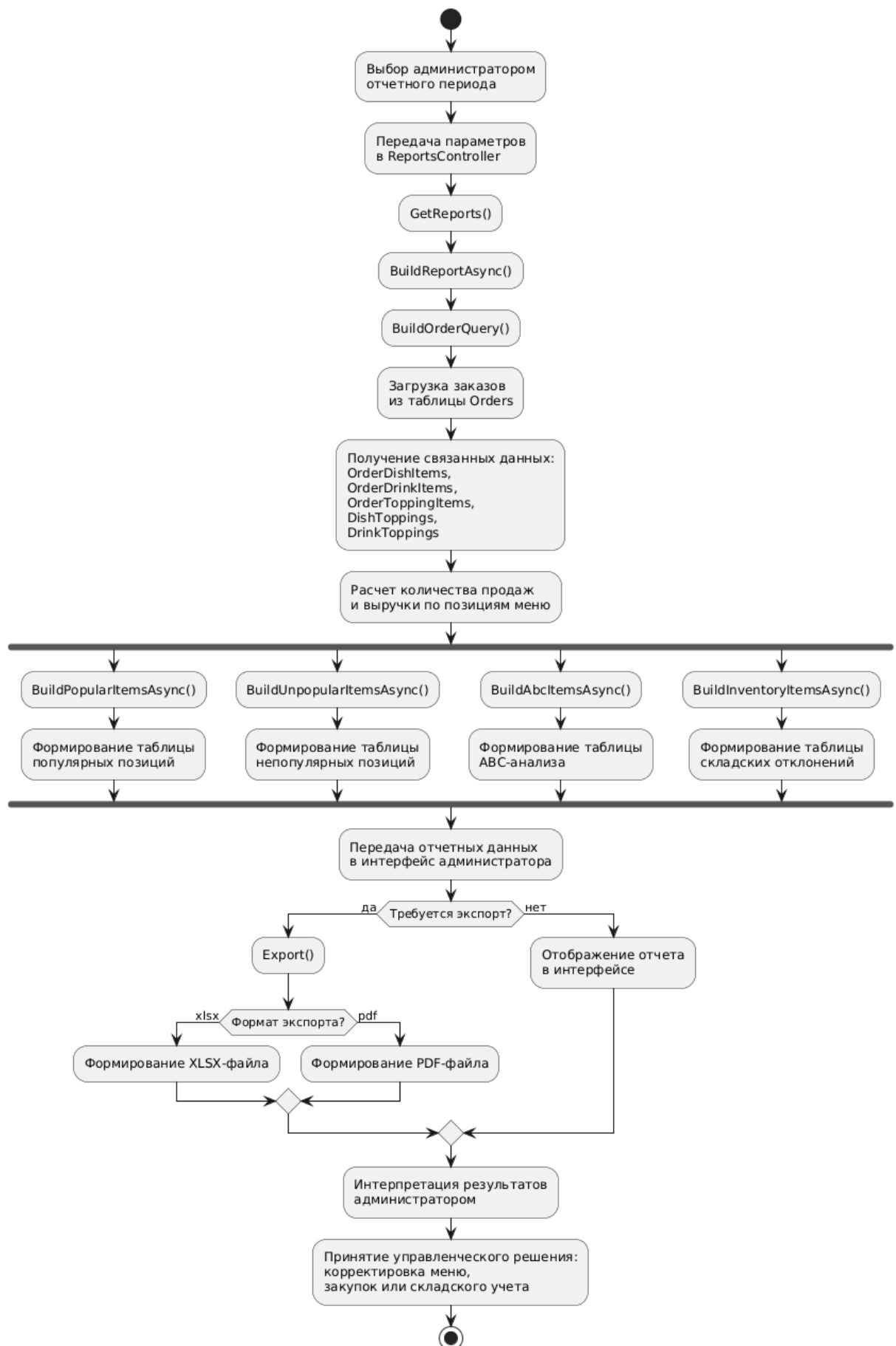


Рисунок 4.4.3.1 – Алгоритм формирования графического отчета

4.4.4 Алгоритмы использования технологий передачи данных

Для гарантированной передачи данных между компонентами системы используется клиент-серверная архитектура. Серверная часть реализована на ASP.NET Core Web API, обмен данными выполняется по HTTP/HTTPS через REST-подобные эндпоинты. Общая схема представлена на рисунке 4.4.4.1. [6], [13]

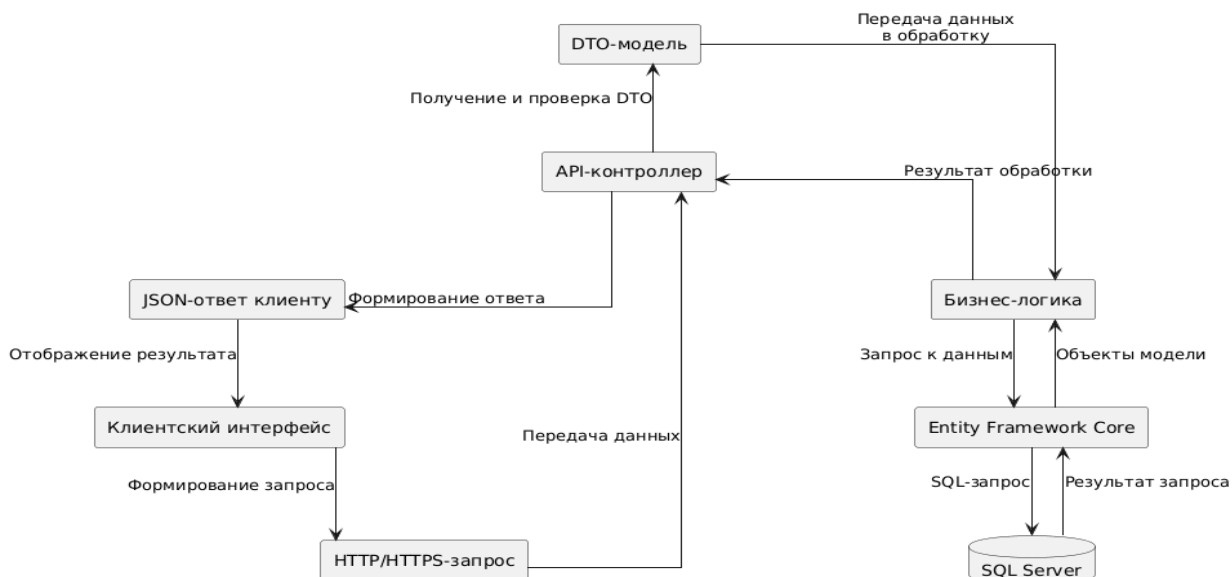


Рисунок 4.4.4.1 – Общая схема передачи данных в системе Garden Nook

Основной формат передачи данных – JSON, сформированный на основе DTO-классов из проекта TransferModels. Например, при оформлении заказа клиентская часть передает объект OrderRequest, а сервер возвращает OrderResponse с идентификатором заказа, статусом, итоговой стоимостью и калорийностью. Пример представлен на рисунке 4.4.4.2.

```
1  {
2      "orderTypeId": 1,
3      "discountId": 2,
4      "comment": "Заказ к 14:30",
5      "pickupAt": "2026-05-18T14:30:00",
6      "dishes": [
7          {
8              "dishId": 5,
9              "quantity": 2,
10             "toppings": []
11         }
12     ],
13     "drinks": [],
14     "toppings": []
15 }
```

Рисунок 4.4.4.2 – Пример JSON-файла

Серверное приложение GardenNookApi подключается к базе данных SQL Server через Entity Framework Core. Контекст базы данных используется для выполнения LINQ-запросов, которые преобразуются в SQL-запросы. Через этот механизм сервер получает и изменяет данные о пользователях, заказах, меню, ингредиентах, остатках, технологических картах и отчетах. Пример SQL-запроса представлен на рисунке 4.4.4.3.

```
1  SELECT o.Id, o.CreatedAt, o.TotalPrice, o.StatusId,  
    c.FullName AS ClientName, d.Name AS DishName,  
    odi.Quantity AS DishQuantity  
2  FROM Orders AS o  
3  LEFT JOIN Clients AS c ON o.ClientId = c.Id  
4  LEFT JOIN OrderDishItems AS odi ON o.Id = odi.OrderId  
5  LEFT JOIN Dishes AS d ON odi.DishId = d.Id  
6  WHERE o.Id = @OrderId;  
7
```

Рисунок 4.4.4.3 – Пример SQL-запроса

Для защиты передаваемых данных применяется cookie-аутентификация. После успешной авторизации сервер формирует защищенную cookie-сессию GardenNook.Auth, в которую помещаются сведения о пользователе: идентификатор, имя, логин и роль. При последующих запросах cookie автоматически передается на сервер, а доступ к функциям ограничивается через атрибуты авторизации, например, [Authorize] и [Authorize(Roles = "...")]. Пример представлен на рисунке 4.4.4.4.

```
1  # Заголовок Set-Cookie от сервера  
2  Set-Cookie: GardenNook.Auth=CfDJ8Lxk...secure-value...;  
3  Path=/;  
4  HttpOnly;  
5  Secure;  
6  SameSite=None;  
7  Expires=Mon, 18 May 2026 21:30:00 GMT  
8  
9  # Передача cookie при следующем запросе  
10 GET /api/orders HTTP/1.1  
11 Host: localhost:7235  
12 Cookie: GardenNook.Auth=CfDJ8Lxk...secure-value...  
13 Accept: application/json
```

Рисунок 4.4.4.4 – Пример использования cookie-аутентификации

4.5 Описание архитектурного решения

4.5.1 Структурная организация программной системы

Структурная организация программной системы – представление архитектуры информационной системы, демонстрирующее распределение программных компонентов по физическим и программным узлам, а также их взаимосвязь и взаимодействие. В контексте информационной системы общественного питания «GardenNook» структурная организация помогает понять, как отдельные части программного комплекса взаимодействуют для оформления заказов, управления меню, учета ингредиентов, обработки складских операций, работы сотрудников и формирования отчетности. Диаграмма развертывания разработанной информационной системы представлена на рисунке 4.5.1.1.

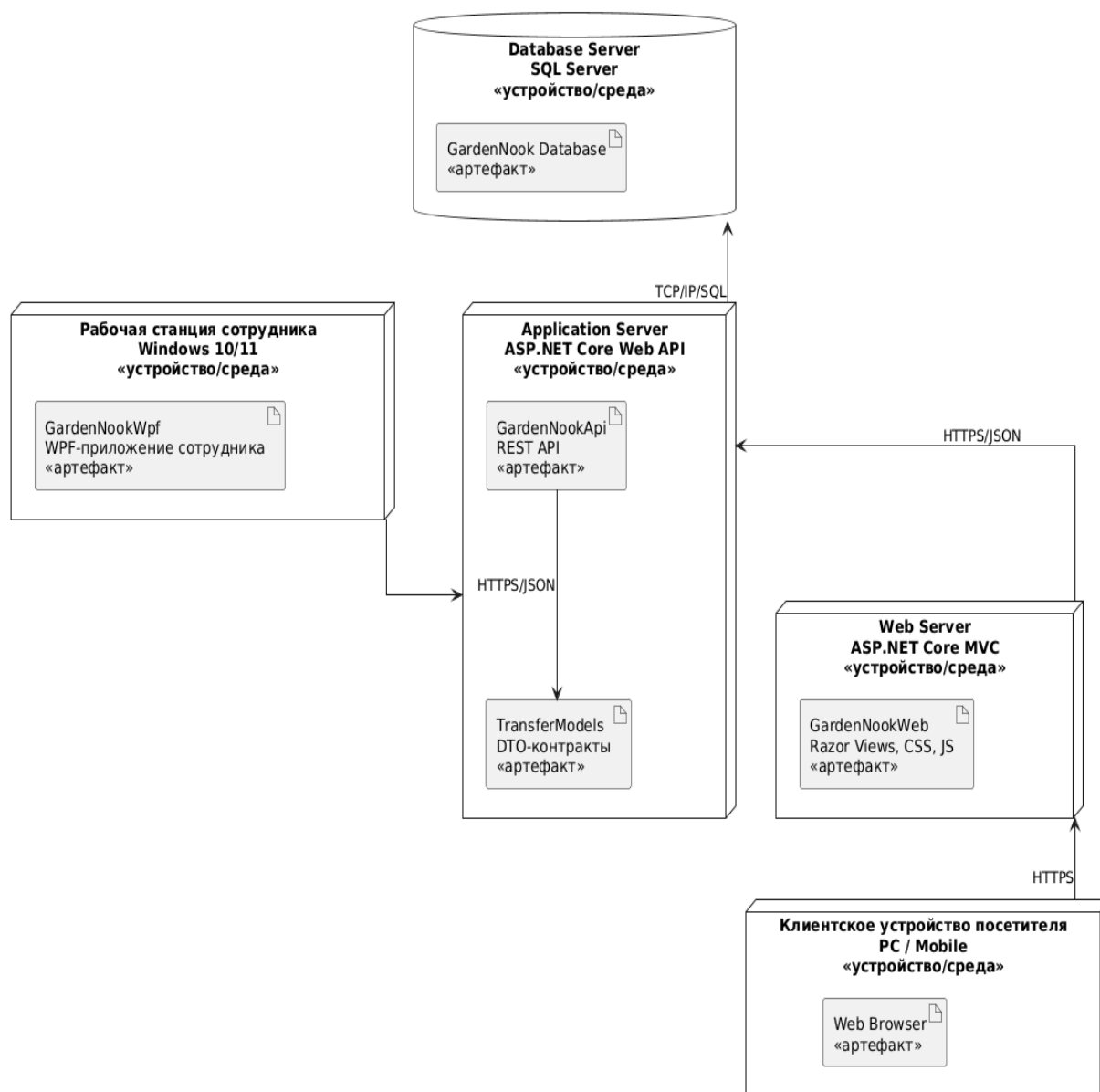


Рисунок 4.5.1.1 – Диаграмма развертывания

На данной диаграмме развертывания показана структурная организация информационной системы, включающая клиентские приложения, серверную часть и базу данных. В качестве клиентских приложений используются веб-приложение для посетителей и WPF-приложение для сотрудников кафе. Серверная часть реализована в виде Web API, а хранение данных выполняется в базе данных SQL Server.

Веб-приложение развернуто на стороне клиента и предназначено для работы посетителей кафе через браузер. Оно обеспечивает регистрацию и авторизацию пользователей, просмотр актуального меню, формирование корзины, оформление заказа, указание комментария и времени подачи, а также просмотр профиля и истории заказов. Веб-приложение передает данные на серверную часть через HTTP/HTTPS-запросы в формате JSON.

WPF-приложение устанавливается на рабочие станции сотрудников и администратора. Оно предназначено для управления внутренними процессами кафе: просмотра и обработки заказов, изменения их статусов, управления меню, категориями, ингредиентами, складскими остатками, технологическими картами, стоп-листом, списаниями и отчетами. Через данное приложение сотрудники получают доступ к функциям, необходимым для приготовления заказов, учета сырья и анализа работы заведения.

Серверная часть представлена Web API, которое является центральным узлом обработки данных. API принимает запросы от веб-приложения и WPF-приложения, выполняет проверку полученных данных, реализует бизнес-логику и обращается к базе данных через Entity Framework Core. Серверная часть обеспечивает работу с заказами, клиентами, сотрудниками, меню, складом, кухней, программой лояльности и отчетностью.

Для хранения информации используется база данных SQL Server. В ней содержатся данные о клиентах, сотрудниках, блюдах, напитках, категориях меню, ингредиентах, полуфабрикатах, заказах, составе заказов, технологических картах, складских остатках, списаниях, скидках и отчетных показателях. Пользовательские приложения не взаимодействуют с базой данных напрямую: все операции выполняются через серверную часть, что позволяет централизованно контролировать обработку данных и обеспечивать их целостность.

Для обмена данными между компонентами используется общий модуль моделей передачи данных. Он содержит DTO-модели, описывающие структуру информации, передаваемой между клиентскими приложениями и сервером. Это позволяет использовать единый формат данных при работе с меню, заказами, клиентами, сотрудниками, складом, кухней и отчетами. Для наглядности взаимодействия компонентов представлена диаграмма на рисунке 4.5.1.2.

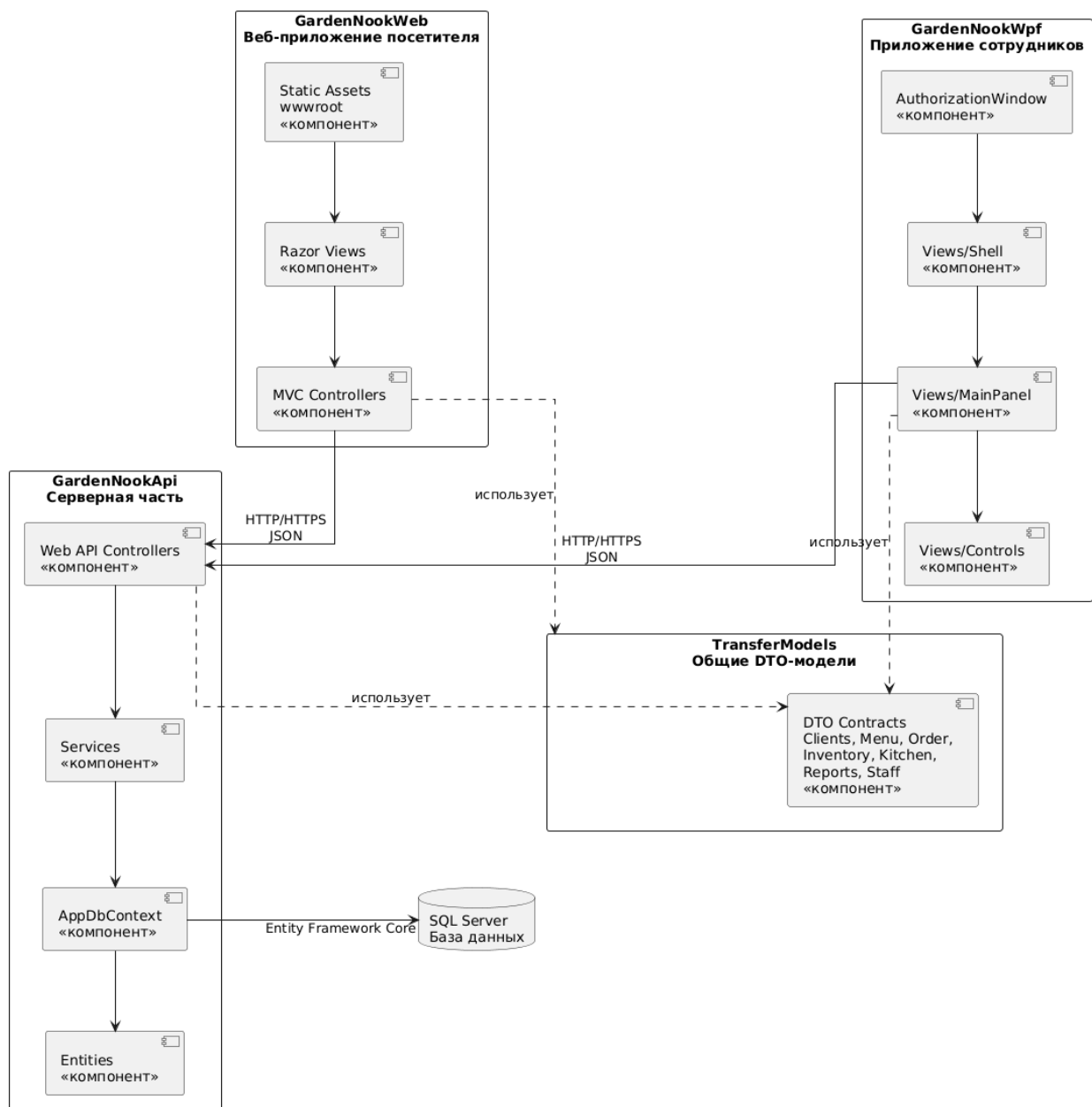


Рисунок 4.5.1.2 – Диаграмма компонентов

4.5.2 Архитектура программного кода

Программный комплекс информационной системы общественного питания «GardenNook» реализован на основе клиент-серверной архитектуры и разделён на несколько логических уровней, отражённых на диаграммах классов: диаграмма Model –показывает структуру данных и основные сущности предметной области (рисунок 4.5.2.1), диаграмма Controller, Service, DTO и пользовательских интерфейсов –показывает организацию серверной логики, обмена данными и взаимодействия с клиентскими приложениями (рисунок 4.5.2.2). [14], [15]

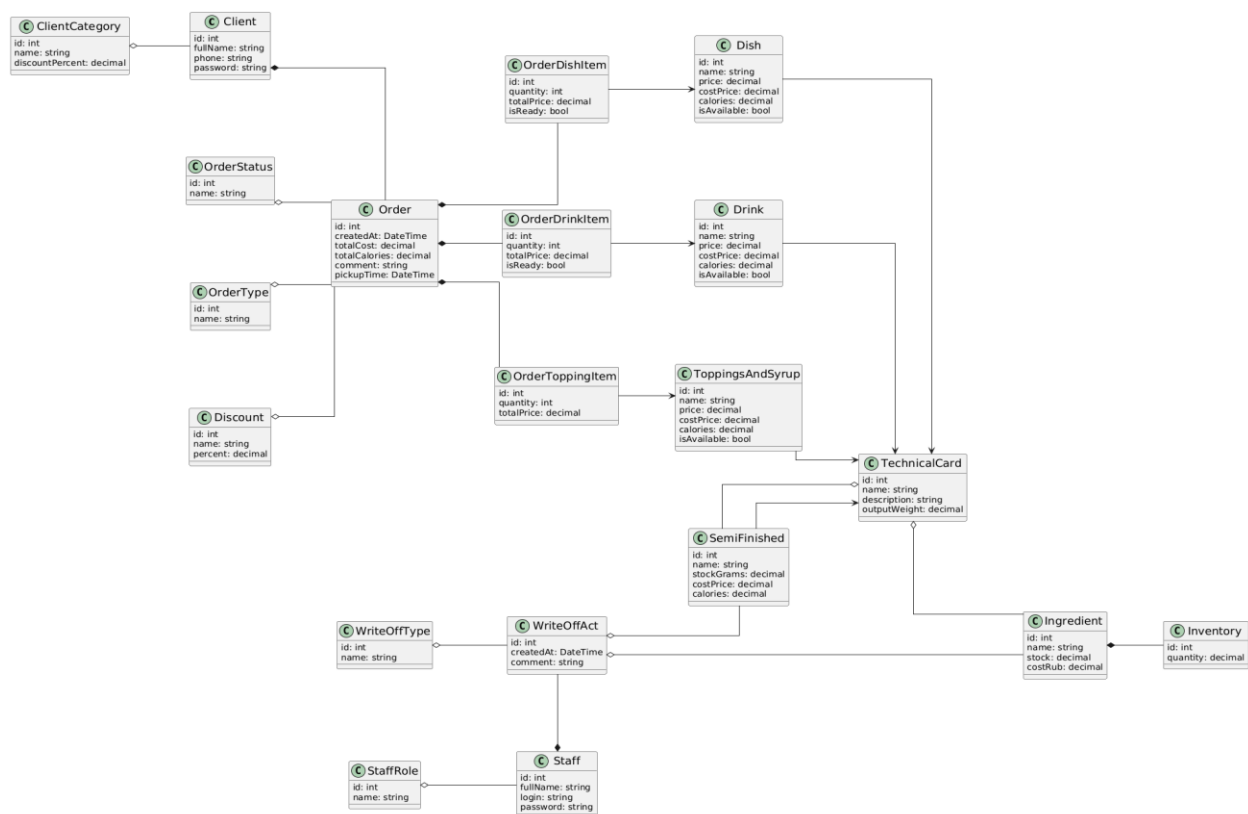


Рисунок 4.5.2.1 – Диаграмма Model

На данной диаграмме представлены основные классы модели данных информационной системы GardenNook и связи между ними. Диаграмма отражает объектное представление ключевых сущностей в программном коде приложения: клиентов, сотрудников, заказов, позиций меню, технологических карт, складских остатков и списаний.

В отличие от ER-диаграммы базы данных, данная UML-диаграмма не предназначена для детального описания структуры таблиц, первичных и внешних ключей, индексов и ограничений SQL Server. ER-диаграмма показывает физическую и логическую организацию базы данных, а диаграмма Model показывает, какие классы используются в программном коде и как основные объекты предметной области связаны между собой на уровне приложения.

Центральными элементами модели являются классы Client, Order, Dish, Drink, ToppingsAndSyrup, TechnicalCard, Ingredient, SemiFinished, Staff и WriteOffAct. Эти классы описывают основные объекты системы: клиентов, заказы, блюда, напитки, добавки, технологические карты, сырьё, полуфабрикаты, сотрудников и акты списания.

Класс Client содержит сведения о клиенте: идентификатор, ФИО, номер телефона и пароль. Класс ClientCategory используется для разделения клиентов по категориям, от которых может зависеть применяемая скидка. Между ClientCategory и Client показана связь агрегации, так как одна категория может объединять несколько клиентов.

Класс Order описывает заказ клиента. Он содержит идентификатор, дату создания, итоговую стоимость, итоговую калорийность, комментарий и время подачи. С заказом связаны

справочные классы `OrderStatus` и `OrderType`, определяющие состояние заказа и формат его получения. Класс `Discount` отражает скидку, которая может быть применена к заказу.

Состав заказа представлен классами `OrderDishItem`, `OrderDrinkItem` и `OrderToppingItem`. Эти классы описывают выбранные позиции заказа, их количество, итоговую цену и, для блюд и напитков, признак готовности. Классы `OrderDishItem`, `OrderDrinkItem` и `OrderToppingItem` связаны соответственно с `Dish`, `Drink` и `ToppingsAndSyrup`, что показывает принадлежность строки заказа к конкретной позиции меню.

Классы `Dish`, `Drink` и `ToppingsAndSyrup` описывают позиции меню. Для них указываются идентификатор, наименование, цена, себестоимость, калорийность и признак доступности. Каждая позиция меню может быть связана с технологической картой `TechnicalCard`, по которой рассчитывается состав и параметры приготовления.

Класс `TechnicalCard` описывает технологическую карту приготовления и содержит идентификатор, наименование, описание и выход готового продукта. На диаграмме показана связь технологической карты с классами `Ingredient` и `SemiFinished`, поскольку рецептура может включать как сырьё, так и полуфабрикаты.

Класс `Ingredient` описывает сырьё, используемое в приготовлении. Он содержит идентификатор, наименование, текущий остаток и стоимость. Класс `Inventory` применяется для складского учёта и фиксации количества сырья. Класс `SemiFinished` описывает полуфабрикат, его остаток в граммах, себестоимость и калорийность.

Класс `Staff` описывает сотрудника системы и содержит ФИО, логин и пароль. Класс `StaffRole` задаёт роль сотрудника: администратор, повар, бариста или кассир. Через эти классы реализуется разграничение функций между пользователями WPF-приложения.

Для оформления списаний используются классы `WriteOffAct` и `WriteOffType`. Класс `WriteOffAct` хранит дату создания и комментарий к акту списания, а `WriteOffType` определяет причину списания. На диаграмме также показано, что акт списания связан с сотрудником, оформившим операцию, а также с ингредиентами и полуфабрикатами, которые могут быть списаны со склада.

Таким образом, диаграмма `Model` показывает объектную структуру основных сущностей системы `GardenNook` и их взаимосвязи в программном коде. Она дополняет ER-диаграмму базы данных: ER-диаграмма раскрывает детальную структуру хранения данных, а UML-диаграмма `Model` описывает классы предметной области, используемые приложением для работы с этими данными.

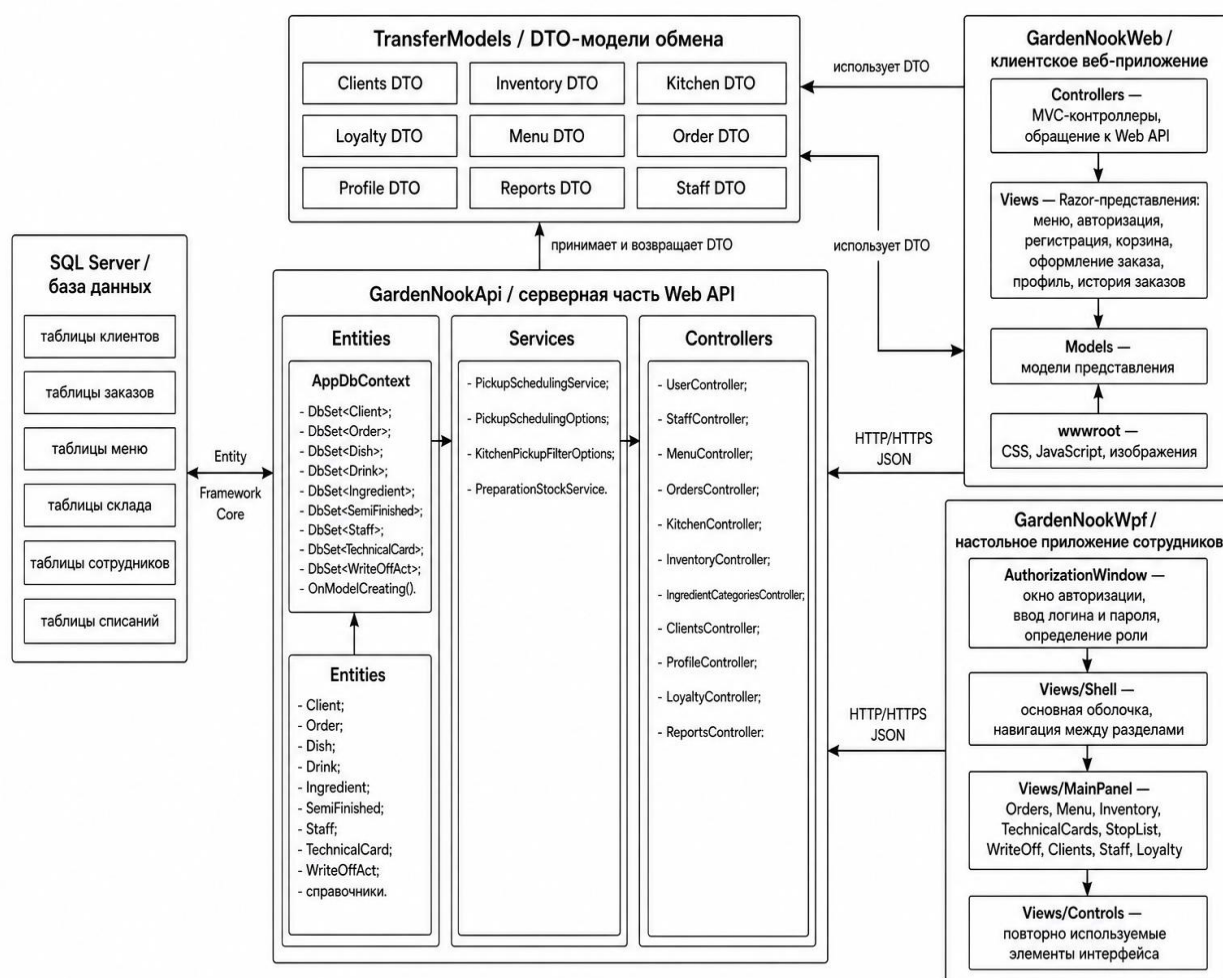


Рисунок 4.5.2.2 – Диаграмма Controller, Service, DTO, Database и UI

На данной диаграмме представлены пакеты Controllers, Services, TransferModels, Entities, GardenNookWeb и GardenNookWpf. Дополнительно на диаграмме показана база данных SQL Server, с которой серверная часть взаимодействует через Entity Framework Core. Модели данных на эту диаграмму полностью не выносятся, так как они уже показаны на отдельной диаграмме Model. Вместо этого отображается класс AppDbContext, который через DbSet<T> обеспечивает доступ к таблицам базы данных.

Controllers – классы серверной части Web API, принимающие HTTP/HTTPS-запросы от клиентских приложений и возвращающие данные в формате JSON:

- **UserController** – отвечает за регистрацию и авторизацию клиентов. Через него выполняется создание новой учётной записи, проверка номера телефона и пароля, а также получение данных авторизованного пользователя.
- **StaffController** – используется для авторизации сотрудников и получения информации о пользователях WPF-приложения. Данный контроллер обеспечивает вход администратора, повара, бариста или кассира в систему.

- MenuController – отвечает за работу с меню. Через него выполняется получение списка блюд, напитков, добавок, категорий, технологических карт и сведений об актуальной доступности позиций. Также данный контроллер используется для управления позициями меню со стороны администратора.
- OrdersController – центральный контроллер для оформления и обработки заказов. Он принимает данные корзины, рассчитывает итоговую стоимость и калорийность, применяет скидку, создаёт заказ и связанные позиции заказа. Также через него выполняется просмотр заказов, изменение статусов и отмена заказа при допустимом состоянии.
- KitchenController – предназначен для работы кухни. Он обеспечивает получение актуального списка заказов, просмотр позиций в работе, изменение готовности блюд и напитков, работу со стоп-листом и полуфабрикатами.
- InventoryController – отвечает за складской учёт. Через него выполняется просмотр и изменение остатков сырья, полуфабрикатов и других складских позиций.
- IngredientCategoriesController – используется для управления категориями ингредиентов. Он позволяет добавлять, редактировать, удалять и получать категории сырья.
- ClientsController – отвечает за просмотр и управление клиентами. Используется администратором для получения списка клиентов и анализа клиентской базы.
- ProfileController – предоставляет клиенту доступ к личному профилю и истории заказов.
- LoyaltyController – отвечает за программу лояльности, клиентские категории и скидки.
- ReportsController – используется для формирования аналитики: просмотра продаж, популярных позиций, статистики заказов и других отчётных данных.

Services – классы, реализующие отдельные элементы бизнес-логики, вынесенные из контроллеров:

- PickupSchedulingService – отвечает за расчёт и проверку времени подачи или получения заказа. Используется при оформлении заказа и позволяет учитывать ограничения по расписанию.
- PickupSchedulingOptions – хранит параметры, связанные с планированием времени выдачи заказа.
- KitchenPickupFilterOptions – задаёт параметры фильтрации заказов для кухни, чтобы сотрудники видели актуальные заказы в нужном временном диапазоне.

- PreparationStockService – используется для работы с остатками полуфабрикатов. Он обеспечивает корректное изменение количества приготовленных заготовок и их использование в производственном процессе.

TransferModels – пакет DTO-моделей, применяемых для передачи данных между клиентами и сервером. Он разделён на тематические группы: Clients, Inventory, Kitchen, Loyalty, Menu, Order, Profile, Reports и Staff. Эти модели не являются таблицами базы данных. Они описывают структуру входящих запросов и исходящих ответов, например, состав заказа, данные авторизации, сведения о меню, данные профиля, параметры отчётов и информацию для кухни.

Entities – пакет сущностей и контекста базы данных:

- ApplicationDbContext – основной класс доступа к базе данных Entity Framework Core. Он наследуется от DbContext и содержит наборы данных DbSet<T> для клиентов, заказов, блюд, напитков, ингредиентов, полуфабрикатов, сотрудников, технологических карт, списаний и справочников.
- В методе OnModelCreating() выполняется настройка таблиц, первичных ключей, внешних ключей, ограничений, типов данных, уникальных индексов и связей между сущностями.
- Через ApplicationDbContext контроллеры и сервисы получают доступ к данным, выполняют выборку, добавление, редактирование и удаление записей.

GardenNookWeb – клиентское веб-приложение для посетителей кафе:

- Controllers – MVC-контроллеры веб-приложения, которые обрабатывают действия пользователя на сайте и обращаются к Web API.
- Models – модели представления, используемые для отображения данных на страницах.
- Views – Razor-представления, формирующие интерфейс клиента: просмотр меню, авторизация, регистрация, корзина, оформление заказа, профиль и история заказов.
- wwwroot – статические ресурсы веб-приложения: CSS, JavaScript, изображения и другие файлы интерфейса.

GardenNookWpf – настольное приложение для сотрудников и администратора:

- AuthorizationWindow – окно авторизации сотрудников. Через него пользователь вводит логин и пароль, после чего система определяет роль сотрудника и открывает доступные разделы.
- Views/Shell – основная оболочка приложения, отвечающая за навигацию между разделами.
- Views/MainPanel – набор экранов для работы с основными модулями системы: клиенты, категории ингредиентов, склад, программа лояльности, меню, заказы, полуфабрикаты, сотрудники, стоп-лист, технологические карты и списания.

- Views/Controls – пользовательские элементы управления, например, компонент отображения заказов. Они используются для повторного применения интерфейсных блоков в разных разделах приложения.
- Раздел Orders используется для просмотра актуального списка заказов и изменения их статусов.
- Раздел Menu предназначен для управления блюдами, напитками, добавками и категориями меню.
- Раздел Inventory применяется для контроля остатков сырья и полуфабрикатов.
- Раздел TechnicalCards используется для просмотра и редактирования технологических карт.
- Раздел StopList отвечает за управление позициями, временно недоступными для заказа.
- Раздел WriteOff используется для формирования актов списания.
- Разделы Clients, Staff и Loyalty обеспечивают управление клиентами, сотрудниками и программой лояльности.

Таким образом, диаграмма Model показывает, какие данные хранятся в базе и как они связаны между собой, а диаграмма Controller, Service, DTO, Database и UI показывает, кто и каким образом управляет этими данными. В системе GardenNook веб-приложение используется клиентами для просмотра меню и оформления заказов, WPF-приложение используется сотрудниками для управления заказами, меню, складом и технологическими картами, а Web API выступает центральным связующим уровнем между интерфейсами и базой данных.

5 ТЕСТИРОВАНИЕ И ОПТИМИЗАЦИЯ

5.1 План тестирования

Тестирование программного комплекса GardenNook проводится с целью подтверждения корректности работы основных функциональных модулей системы, проверки соответствия реализованного программного средства требованиям технического задания, а также выявления дефектов до передачи системы в эксплуатацию. [1], [16]

С учетом специфики разработанного программного комплекса, включающего серверный REST API на ASP.NET Core, клиентское веб-приложение, WPF-приложение сотрудников, общие DTO-модели и базу данных Microsoft SQL Server, для проверки выбраны следующие виды тестирования:

- Функциональное тестирование – проверка соответствия реализованных функций требованиям технического задания. В рамках данного вида тестирования проверяются ключевые пользовательские сценарии: регистрация и авторизация клиента, вход сотрудника, просмотр меню, оформление заказа, применение скидок, изменение статусов заказа, работа кухни, управление меню, складом, технологическими картами, стоп-листом, клиентами и сотрудниками.
- Тестирование пользовательского интерфейса – проверка корректности отображения и работы элементов управления в клиентском веб-интерфейсе и WPF-приложении сотрудников. Проверяются формы ввода, таблицы, модальные окна, сообщения об ошибках, отображение статусов, доступность кнопок в зависимости от роли пользователя и корректность реакции интерфейса на действия пользователя.
- Тестирование API-взаимодействия – проверка корректности обмена данными между клиентскими приложениями и серверной частью. Проверяется формирование HTTP-запросов, соответствие JSON-данных DTO-моделям, обработка успешных и ошибочных ответов, работа JWT-аутентификации, а также корректная передача данных при оформлении заказов, изменении статусов, работе со справочниками и складскими операциями.
- Тестирование данных и бизнес-правил – проверка корректности расчетов и сохранения целостности данных в базе. Особое внимание уделяется расчету стоимости заказа, применению скидок, расчету КБЖУ, изменению складских остатков, формированию актов списания, работе стоп-листа и запрету некорректных операций, например, отрицательных количеств или изменения заказа в недопустимом статусе.
- Unit-тестирование – проверка отдельных критичных методов и участков бизнес-логики без запуска полного пользовательского интерфейса. В рамках данного вида тестирования проверяются расчеты стоимости заказа, применение скидок, обработка некорректных входных

данных, а также отдельные операции, связанные с изменением складских остатков и проверкой доступности позиций меню.

- Тестирование безопасности – проверка механизмов аутентификации и авторизации. Проверяется невозможность доступа к административным разделам без входа в систему или без соответствующей роли, корректная обработка недействительного токена, ограничение операций администратора, сотрудника кухни и клиента, а также отсутствие выдачи служебных данных при ошибочных запросах.

- Базовое тестирование производительности – оценка времени отклика системы при выполнении типовых операций: загрузка меню, открытие профиля клиента, оформление заказа, получение очереди кухни, сохранение справочников и формирование отчетов. Данный вид тестирования необходим для проверки того, что основные сценарии выполняются без заметных задержек при обычном объеме данных.

Полноценное нагрузочное тестирование не выделяется в отдельный этап, поскольку в рамках данной работы основной акцент сделан на проверке корректности бизнес-логики, пользовательских сценариев и взаимодействия клиентских приложений с серверным API. Оценка предельной пропускной способности системы, подбор серверной конфигурации и исследование поведения приложения при высокой параллельной нагрузке относятся к задачам дальнейшего развития и промышленного внедрения программного комплекса. Автоматизированное тестирование применяется выборочно для наиболее критичных расчетных и бизнес-операций; основное внимание уделяется функциональной и интеграционной проверке системы.

5.2 Результаты тестирования и оптимизация

5.2.1 Функциональное тестирование

Функциональное тестирование программного комплекса GardenNook выполнялось методом «черного ящика» через клиентский веб-интерфейс и WPF-приложение сотрудников. Проверялись основные пользовательские сценарии, предусмотренные техническим заданием: авторизация, работа с меню, оформление заказа, обработка заказов на кухне, администрирование справочников, работа с персоналом, складские операции и формирование отчетов.

Алгоритм проведения функционального тестирования включал:

- определение проверяемых модулей,
- подготовку тестовых данных,
- последовательное выполнение сценариев,
- сравнение фактического результата с ожидаемым,
- фиксацию статуса проверки.

Результаты функционального тестирования представлены в таблице 5.2.1.

Таблица 5.2.1 – Результаты функционального тестирования

№	Проверяемый модуль	Сценарий	Ожидаемый результат	Фактический результат	Статус
1	Регистрация клиента	Создание учетной записи с валидными данными	Пользователь регистрируется и может выполнить вход	Соответствует ожидаемому	Пройден
2	Авторизация клиента	Вход через клиентский веб-интерфейс	Открывается клиентский контур с меню и профилем	Соответствует ожидаемому	Пройден
3	Авторизация сотрудника	Вход сотрудника в WPF-приложение	Открывается рабочий контур согласно роли	Соответствует ожидаемому	Пройден
4	Просмотр меню	Открытие каталога блюд и напитков	Позиции меню отображаются с ценой, составом и доступностью	Соответствует ожидаемому	Пройден
5	Корзина	Добавление, удаление и изменение количества позиций	Состав корзины и итоговая стоимость пересчитываются	Соответствует ожидаемому	Пройден
6	Оформление заказа	Отправка заказа с выбранными позициями	Заказ сохраняется и появляется в очереди кухни	Соответствует ожидаемому	Пройден
7	История заказов	Просмотр клиентом ранее созданных заказов	История отображает актуальные статусы и состав заказов	Соответствует ожидаемому	Пройден
8	Очередь кухни	Просмотр заказов сотрудником кухни	Новые заказы отображаются в рабочей очереди	Соответствует ожидаемому	Пройден
9	Статусы заказа	Изменение статуса позиции и всего заказа	Статус сохраняется и отображается в клиентском и служебном контурах	Соответствует ожидаемому	Пройден

10	Управление меню	Создание и редактирование позиции меню	Данные сохраняются и отображаются в справочнике	Соответствует ожидаемому	Пройден
11	Склад	Пополнение и изменение остатков ингредиентов	Остатки обновляются без нарушения связей	Соответствует ожидаемому	Пройден
12	Технологические карты	Создание и редактирование состава позиции	Состав сохраняется и используется при расчетах	Соответствует ожидаемому	Пройден
13	Стоп-лист	Добавление позиции в список недоступных	Позиция становится недоступной для заказа	Соответствует ожидаемому	Пройден
14	Списания	Формирование акта списания	Акт сохраняется, остатки уменьшаются	Соответствует ожидаемому	Пройден
15	Отчеты	Формирование управленческого отчета	Отчет отображает продажи, расход и остатки	Соответствует ожидаемому	Пройден

По результатам функционального тестирования все основные сценарии выполнены успешно. Критических дефектов, блокирующих работу программного комплекса, не выявлено.

5.2.2 Тестирование пользовательского интерфейса

Тестирование пользовательского интерфейса было направлено на проверку корректности отображения экранов, форм, таблиц, модальных окон, сообщений об ошибках и визуальных состояний. Проверка выполнялась вручную в клиентском веб-интерфейсе и в WPF-приложении сотрудников. Дополнительно оценивалась удобность расположения элементов управления, читаемость данных в таблицах и карточках, а также корректность отображения статусов заказов, складских операций и результатов сохранения. Особое внимание уделялось тому, чтобы пользователь мог без лишних действий понять, выполнена операция успешно или требуется исправить введенные данные. Результаты UI-тестирования представлены в таблице 5.2.2.

Таблица 5.2.2 – Результаты тестирования пользовательского интерфейса

№	Проверяемый элемент	Условие проверки	Ожидаемый результат	Фактический результат	Статус
1	Веб-страница авторизации	Открытие формы входа	Поля и кнопки отображаются корректно	Соответствует ожидаемому	Пройден

2	Веб-страница меню	Просмотр списка позиций	Карточки меню читаемы, изображения и цены отображаются	Соответствует ожидаемому	Пройден
3	Веб-страница корзины	Изменение состава заказа	Элементы управления доступны, сумма пересчитывается	Соответствует ожидаемому	Пройден
4	Веб-страница профиля	Просмотр данных клиента и истории заказов	Информация отображается без визуальных искажений	Соответствует ожидаемому	Пройден
5	WPF Orders	Просмотр очереди заказов	Статусы визуально различаются, детали открываются	Соответствует ожидаемому	Пройден
6	WPF Menu	Работа с позициями меню	Таблицы, кнопки и модальные окна работают корректно	Соответствует ожидаемому	Пройден
7	WPF Inventory	Работа со складом	Формы ввода и таблицы остатков отображаются корректно	Соответствует ожидаемому	Пройден
8	WPF TechnicalCards	Редактирование технологической карты	Состав позиции отображается структурировано	Соответствует ожидаемому	Пройден
9	WPF StopList	Управление недоступными позициями	Доступность позиций понятна пользователю	Соответствует ожидаемому	Пройден
10	WPF WriteOff	Создание акта списания	Модальное окно содержит необходимые поля и валидацию	Соответствует ожидаемому	Пройден
11	WPF Clients	Просмотр и редактирование клиентов	Данные клиентов отображаются в таблице без наложений	Соответствует ожидаемому	Пройден
12	WPF Staff	Управление сотрудниками	Роли и данные сотрудников отображаются корректно	Соответствует ожидаемому	Пройден
13	WPF Loyalty	Работа со скидками и категориями	Формы сохранения и удаления работают предсказуемо	Соответствует ожидаемому	Пройден

Интерфейс программного комплекса корректно реагирует на действия пользователя. Ошибки ввода сопровождаются понятными сообщениями, а критичные операции подтверждаются через отдельные модальные окна.

5.2.3 Тестирование API-взаимодействия

Тестирование API-взаимодействия выполнялось с использованием Swagger и Postman. Проверялись корректность HTTP-методов, структура JSON-ответов, обработка валидных и невалидных данных, а также доступность защищенных маршрутов только для пользователей с соответствующими ролями. Результаты тестирования API представлены в таблице 5.2.3.

Таблица 5.2.3 – Результаты тестирования API-взаимодействия

№	Эндпоинт	Метод	Сценарий	Ожидаемый результат	Фактический результат	Статус
1	/api/auth/client	POST	Вход клиента с корректными данными	200 ОК, возвращается токен и данные пользователя	Соответствует ожидаемому	Пройден
2	/api/auth/staff	POST	Вход сотрудника с корректными данными	200 ОК, возвращается роль сотрудника	Соответствует ожидаемому	Пройден
3	/api/menu	GET	Получение клиентского меню	200 ОК, возвращается список доступных блюд и напитков	Соответствует ожидаемому	Пройден
4	/api/orders	POST	Оформление заказа	201/200 ОК, заказ создается в базе данных	Соответствует ожидаемому	Пройден
5	/api/kitchen/orders	GET	Получение очереди кухни	200 ОК, возвращается список активных заказов	Соответствует ожидаемому	Пройден
6	/api/kitchen/orders/{id}/complete	POST	Завершение заказа	Статус заказа изменяется на завершенный	Соответствует ожидаемому	Пройден
7	/api/inventory/ingredients	GET	Получение списка ингредиентов	200 ОК, возвращаются складские позиции	Соответствует ожидаемому	Пройден
8	/api/kitchen/write-off/acts	POST	Создание акта списания	Акт сохраняется, остатки изменяются	Соответствует ожидаемому	Пройден
9	/api/reports	GET	Получение отчета администратором	200 ОК, возвращаются аналитические данные	Соответствует ожидаемому	Пройден
10	/api/staff	GET	Запрос без роли администратора	Доступ запрещен	Соответствует ожидаемому	Пройден

Проверка показала, что серверные методы возвращают корректные статусы и данные. Ошибочные запросы не приводят к сохранению некорректной информации, а защищенные методы недоступны без авторизации или нужной роли.

5.2.4 Тестирование данных и бизнес-правил

Тестирование данных и бизнес-правил было направлено на проверку расчетов и сохранение целостности информации в базе данных. Особое внимание уделялось заказам, скидкам, КБЖУ, складским остаткам, стоп-листу и операциям, которые могут нарушить связи между сущностями.

Результаты проверки бизнес-правил представлены в таблице 5.2.4.

Таблица 5.2.4 – Результаты тестирования данных и бизнес-правил

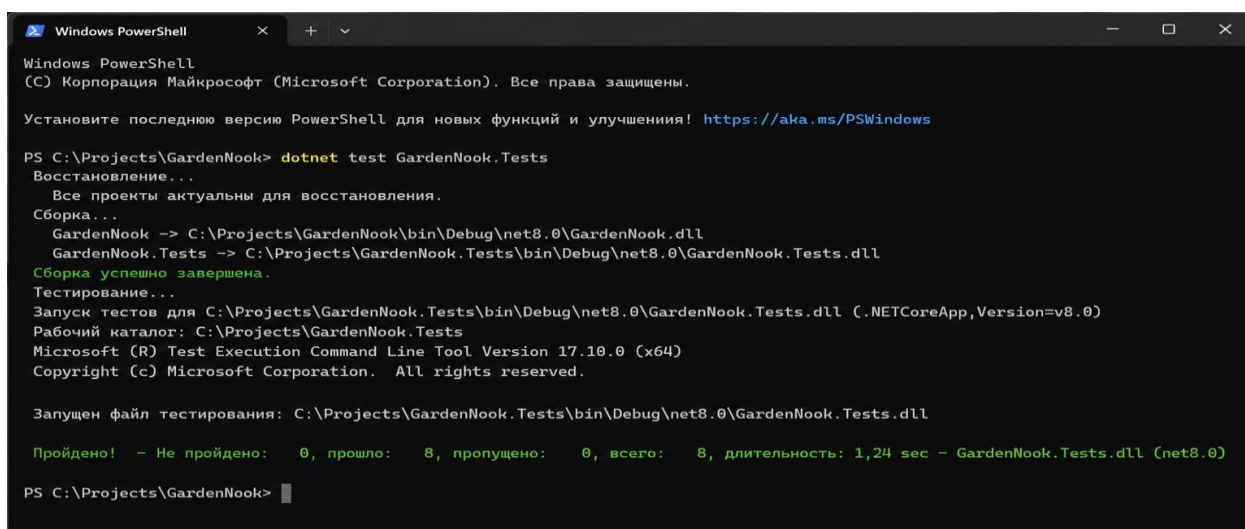
№	Проверяемое правило	Сценарий	Ожидаемый результат	Фактический результат	Статус
1	Расчет стоимости заказа	Добавление нескольких позиций и добавок	Итоговая сумма равна сумме выбранных позиций	Соответствует ожидаемому	Пройден
2	Применение скидки	Оформление заказа клиентом с категорией лояльности	Скидка применяется к итоговой стоимости	Соответствует ожидаемому	Пройден
3	Расчет КБЖУ	Просмотр пищевой ценности позиции	КБЖУ рассчитывается на основании технологической карты	Соответствует ожидаемому	Пройден
4	Изменение остатков	Оформление заказа с доступными ингредиентами	Складские остатки уменьшаются на расчетный объем	Соответствует ожидаемому	Пройден
5	Нехватка ингредиентов	Оформление заказа при недостатке заготовок	Система блокирует некорректное списание	Соответствует ожидаемому	Пройден
6	Стоп-лист	Отключение позиции вручную	Позиция недоступна для оформления заказа	Соответствует ожидаемому	Пройден
7	Отрицательные количества	Ввод отрицательного количества в форме	Данные не принимаются, выводится ошибка	Соответствует ожидаемому	Пройден
8	Удаление связанных данных	Попытка удалить запись, используемую в заказах или техкартах	Операция блокируется или требует корректной обработки связи	Соответствует ожидаемому	Пройден

Проверка подтвердила, что основные расчеты выполняются корректно, а ограничения предметной области предотвращают появление некорректных данных.

5.2.5 Unit-тестирование

Unit-тестирование применялось выборочно для проверки отдельных критичных методов и участков бизнес-логики без запуска полного пользовательского интерфейса. В автоматические проверки были включены сценарии расчета итоговой стоимости заказа, применения скидки, отклонения некорректных входных данных, изменения складских остатков и определения доступности позиций меню.

Запуск unit-тестов выполнялся командой `dotnet test` для тестового проекта `GardenNook.Tests`. Все подготовленные проверки завершились успешно, что подтверждает корректность работы наиболее важных расчетных операций. Результаты представлены на рисунках 5.2.5.1-5.2.5.2.



```
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

Установите последнюю версию PowerShell для новых функций и улучшения! https://aka.ms/PSWindows

PS C:\Projects\GardenNook> dotnet test GardenNook.Tests
Восстановление...
Все проекты актуальны для восстановления.
Сборка...
  GardenNook -> C:\Projects\GardenNook\bin\Debug\net8.0\GardenNook.dll
  GardenNook.Tests -> C:\Projects\GardenNook.Tests\bin\Debug\net8.0\GardenNook.Tests.dll
Сборка успешно завершена.
Тестирование...
Запуск тестов для C:\Projects\GardenNook.Tests\bin\Debug\net8.0\GardenNook.Tests.dll (.NETCoreApp,Version=v8.0)
Рабочий каталог: C:\Projects\GardenNook.Tests
Microsoft (R) Test Execution Command Line Tool Version 17.10.0 (x64)
Copyright (c) Microsoft Corporation. All rights reserved.

Запущен файл тестирования: C:\Projects\GardenNook.Tests\bin\Debug\net8.0\GardenNook.Tests.dll

Пройдено! - Не пройдено: 0, прошло: 8, пропущено: 0, всего: 8, длительность: 1,24 sec - GardenNook.Tests.dll (net8.0)

PS C:\Projects\GardenNook>
```

Рисунок 5.2.5.1 – Результат выполнения unit-тестов в терминале

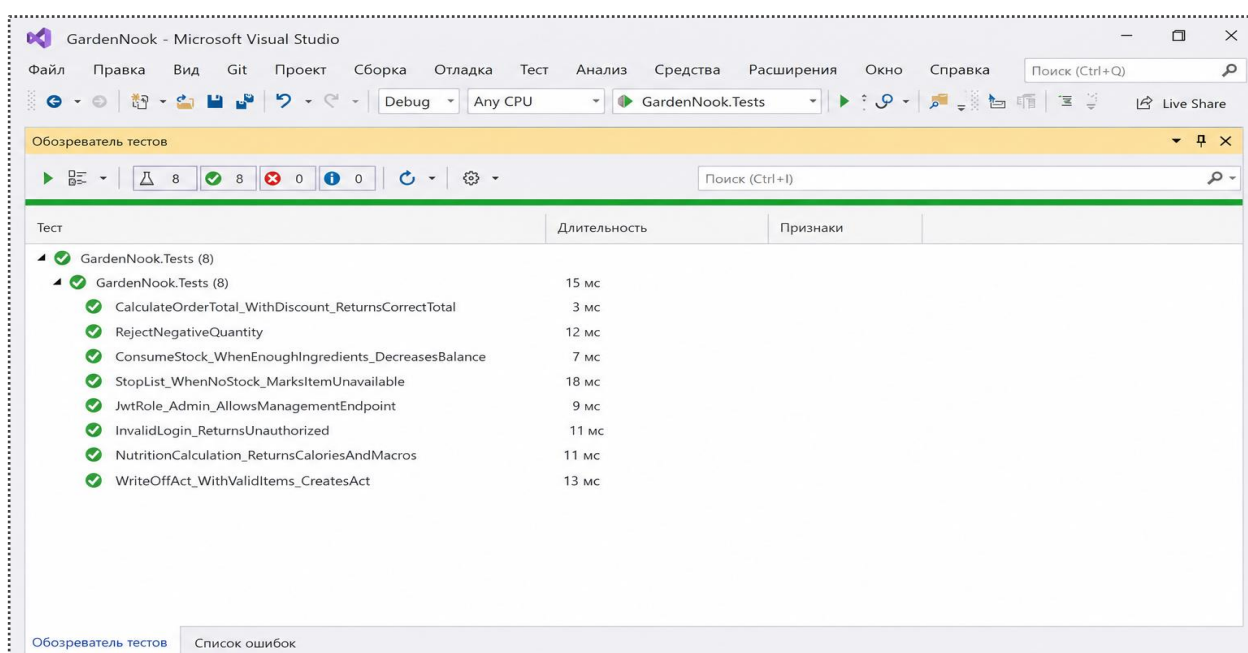


Рисунок 5.2.5.2 – Результат выполнения unit-тестов в Test Explorer

Unit-тестирование не рассматривалось как полное покрытие всего программного комплекса, а использовалось как дополнительная автоматическая проверка наиболее рискованных участков логики.

5.2.6 Тестирование безопасности

Тестирование безопасности проводилось вручную и было направлено на проверку механизмов аутентификации, JWT-авторизации и разграничения прав доступа между клиентом, сотрудником кухни и администратором. [17], [18]

Результаты тестирования безопасности представлены в таблице 5.2.5.

Таблица 5.2.5 – Результаты тестирования безопасности

№	Проверяемый аспект	Способ проверки	Ожидаемый результат	Фактический результат	Статус
1	Доступ без токена	Запрос к защищенному API без авторизации	Сервер возвращает отказ в доступе	Соответствует ожидаемому	Пройден
2	Недействительный токен	Передача измененного JWT-токена	Запрос отклоняется	Соответствует ожидаемому	Пройден
3	Роль клиента	Попытка клиента открыть административный метод	Доступ запрещен	Соответствует ожидаемому	Пройден
4	Роль сотрудника кухни	Попытка выполнить административное удаление	Операция недоступна без роли администратора	Соответствует ожидаемому	Пройден
5	Роль администратора	Открытие разделов управления меню, складом и персоналом	Операции доступны после авторизации	Соответствует ожидаемому	Пройден
6	Выход из системы	Выполнение logout и повторный запрос к API	Текущая сессия завершается	Соответствует ожидаемому	Пройден
7	XSS-строки	Ввод script-alert в текстовые поля	Скрипт не выполняется как код	Соответствует ожидаемому	Пройден
8	SQL-like строки	Ввод OR 1=1 в поля авторизации	Авторизация не выполняется, данные не раскрываются	Соответствует ожидаемому	Пройден

Механизмы авторизации и разграничения доступа работают корректно. Пользователь получает доступ только к функциям, соответствующим его роли, а некорректные или потенциально опасные входные данные не приводят к раскрытию служебной информации.

5.2.7 Базовое тестирование производительности и оптимизация

Базовое тестирование производительности проводилось для оценки времени отклика системы при выполнении типовых пользовательских операций. Проверка выполнялась в условиях локального стенда разработки с обычным объемом демонстрационных данных. Результаты базового тестирования производительности представлены в таблице 5.2.6. [19], [20]

Таблица 5.2.6 – Результаты базового тестирования производительности

№	Операция	Критерий	Фактический результат	Статус
1	Загрузка меню	Не более 2 секунд	Около 1,3 секунды	Пройден
2	Открытие профиля клиента	Не более 2 секунд	Около 1,1 секунды	Пройден
3	Оформление заказа	Не более 3 секунд	Около 2,0 секунды	Пройден
4	Получение очереди кухни	Не более 2 секунд	Около 1,2 секунды	Пройден
5	Сохранение справочника	Не более 2 секунд	Около 1,0 секунды	Пройден
6	Формирование отчета	Не более 3 секунд	Около 2,4 секунды	Пройден

По результатам тестирования были выполнены точечные оптимизационные мероприятия: уточнены сообщения об ошибках, усилена проверка обязательных полей, откорректировано отображение статусов заказов и складских операций, а также уменьшено количество лишних перезагрузок списков после сохранения данных.

Полноценное нагрузочное тестирование не выделялось в отдельный этап, поскольку в рамках данной работы основной акцент сделан на корректности бизнес-логики, пользовательских сценариев и взаимодействия клиентских приложений с серверным API. Оценка предельной пропускной способности системы, подбор серверной конфигурации и исследование поведения приложения при высокой параллельной нагрузке относятся к задачам дальнейшего развития и промышленного внедрения программного комплекса.

Проведение функционального, интерфейсного, API, бизнес-ориентированного, выборочного unit-тестирования, проверки безопасности и базовой производительности подтвердило соответствие программного комплекса GardenNook основным требованиям технического задания.

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Программный комплекс Garden Nook предназначен для автоматизации работы кофейни здорового питания и используется несколькими категориями пользователей. Клиент взаимодействует с веб-интерфейсом в браузере, а сотрудники кофейни работают в WPF-приложении. Набор доступных функций определяется ролью учетной записи, поэтому после входа система открывает только те разделы, которые необходимы конкретному пользователю.

6.1 Руководство для клиента

Клиент начинает работу с веб-интерфейсом Garden Nook со страницы авторизации. Для входа в систему клиент вводит учетные данные и подтверждает авторизацию, после чего получает доступ к меню, корзине, оформлению заказа и профилю. Форма авторизации клиента представлена на рисунке 6.1.1.

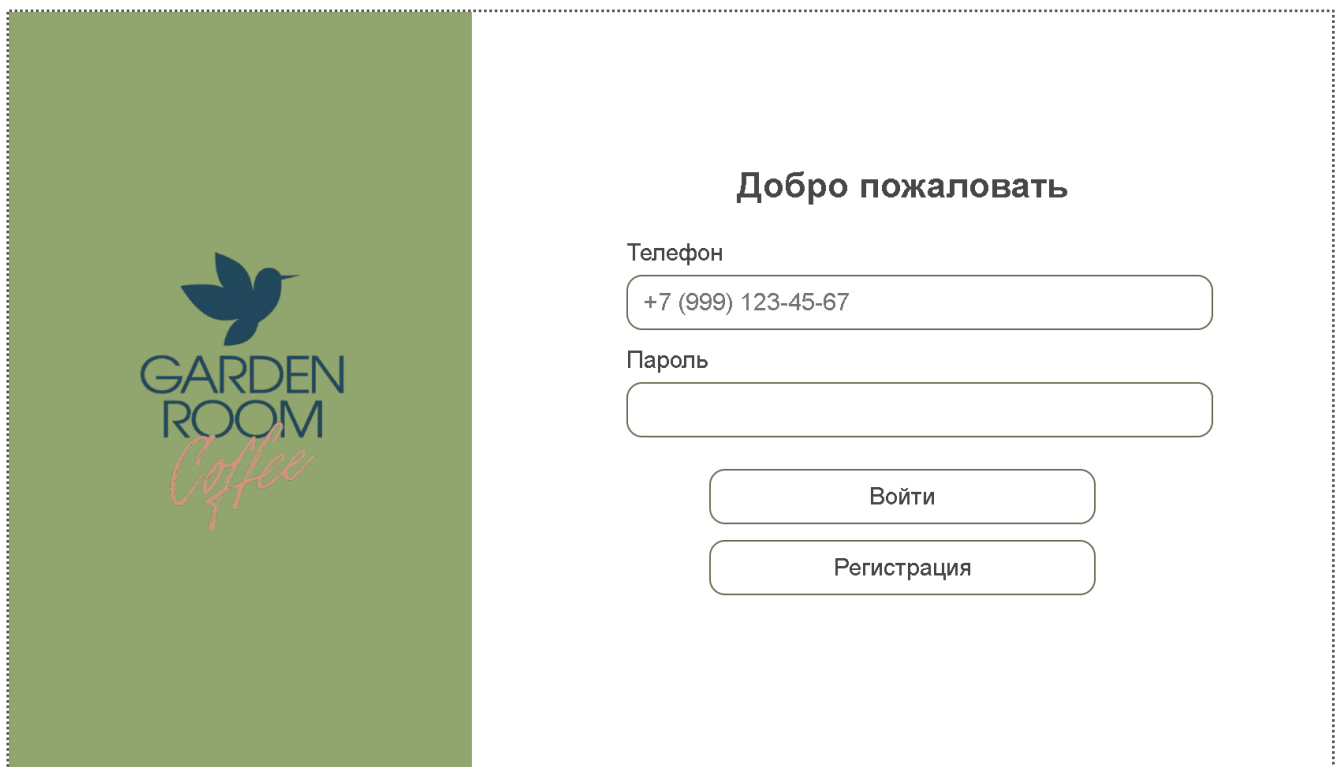
The image shows a web interface for 'Garden Room Coffee'. On the left is a green vertical banner with the company logo, which consists of a blue bird icon above the text 'GARDEN ROOM' in a sans-serif font and 'Coffee' in a red script font below it. To the right of the banner is a white login form. At the top of the form is the heading 'Добро пожаловать' in bold. Below it are two input fields: the first is labeled 'Телефон' and contains the text '+7 (999) 123-45-67'; the second is labeled 'Пароль' and is empty. Below these fields are two buttons: 'Войти' (Login) and 'Регистрация' (Registration), both with rounded corners and thin borders.

Рисунок 6.1.1 – Форма авторизации клиента

Если учетная запись еще не создана, клиент переходит к регистрации. В форме регистрации указываются данные, необходимые для создания профиля и последующего оформления заказов. После успешной регистрации система сохраняет учетную запись и предоставляет доступ к основным функциям клиентского интерфейса. Форма регистрации клиента представлена на рисунке 6.1.2.



Регистрация

ФИО

Телефон

Пароль

Зарегистрироваться

Авторизация

Рисунок 6.1.2 – Форма регистрации клиента

После входа клиент переходит на страницу меню. В данном разделе отображаются доступные позиции, категории, названия, цены и основные сведения о блюдах и напитках. Клиент выбирает интересующую позицию из списка или пользуется фильтрацией по категориям. Страница меню в веб-интерфейсе представлена на рисунке 6.1.3.



МЕНЮ



Арахисовое печенье 50 гр

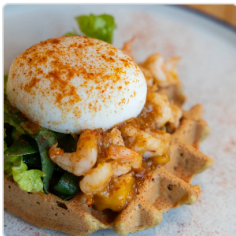
Десерты

129 Р

Ккал: 260 | Б: 10 | Ж: 17 | У: 18 (на порцию)

Состав: арахисовая паста, сироп топинамбура

Добавить



Безглютеновая вафля с креветкой и яйцом п... 210 гр

Вафли

459 Р

Ккал: 320 | Б: 15 | Ж: 24 | У: 11 (на порцию)

Состав: вода, кокосовое молоко ago d, креветки, огурец, романо, чеснок, чечевица красная, шпинат

Добавить



Буррито с индейкой и овощами 165 гр

Горячие блюда

429 Р

Фильтры

Цена:

0 - 1000 Р

Категории

Все

Блюда (24)

Напитки (27)

Добавки (12)

Рисунок 6.1.3 – Страница меню в веб-интерфейсе

Для изучения выбранной позиции клиент открывает карточку товара. В карточке отображаются состав, пищевая ценность, стоимость и дополнительные сведения, которые помогают принять решение о добавлении позиции в заказ. Карточка позиции меню с составом и пищевой ценностью представлена на рисунке 6.1.4.

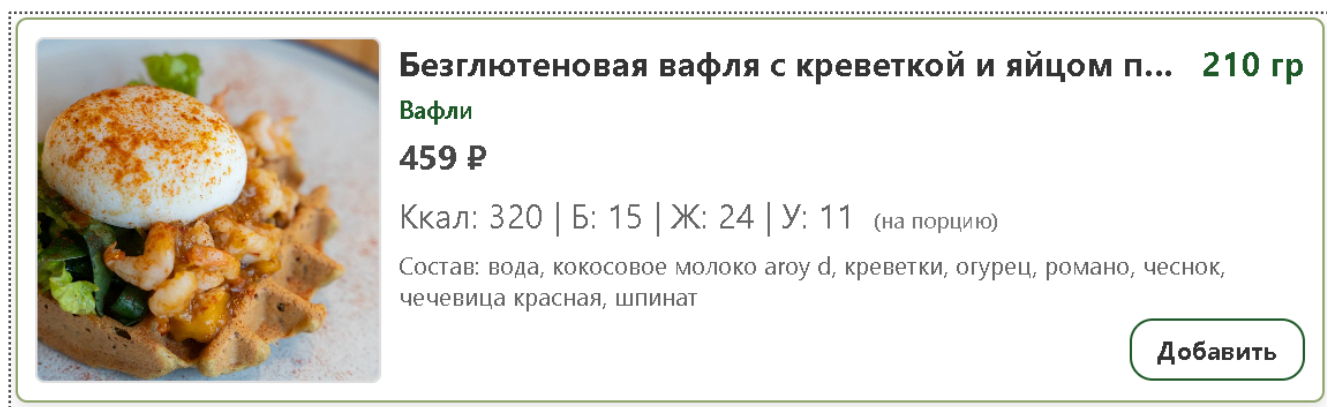


Рисунок 6.1.4 – Карточка позиции меню с составом и пищевой ценностью

После выбора позиции клиент добавляет ее в корзину. В корзине клиент проверяет состав заказа, изменяет количество позиций, удаляет лишние строки и при необходимости указывает комментарий. Перед подтверждением заказа система отображает итоговую стоимость. Корзина клиента перед оформлением заказа представлена на рисунке 6.1.5.

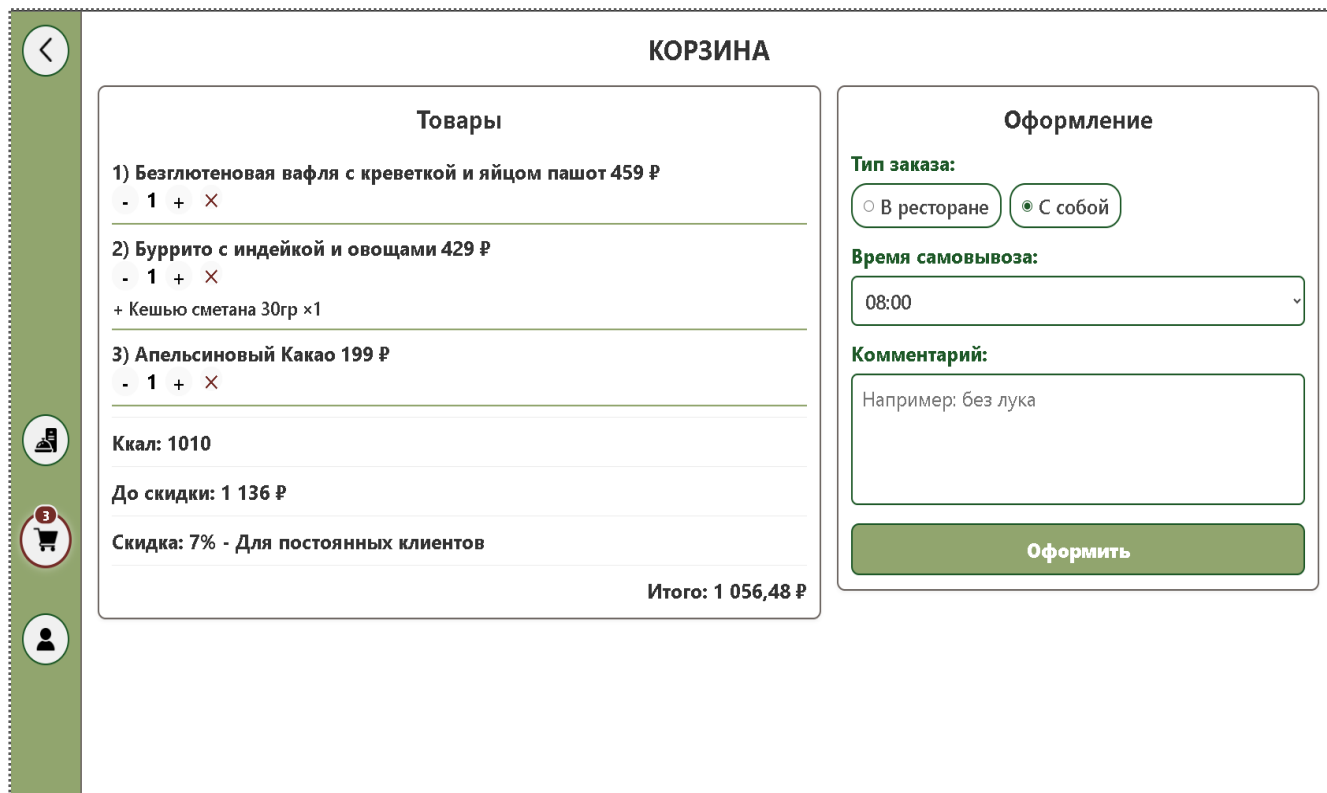


Рисунок 6.1.5 – Корзина клиента перед оформлением заказа

После оформления заказ сохраняется в системе и становится доступен сотрудникам кофейни для обработки. Клиент может открыть профиль, просмотреть личные данные и историю ранее оформленных заказов с их текущими статусами. Профиль клиента с историей заказов представлен на рисунке 6.1.6.

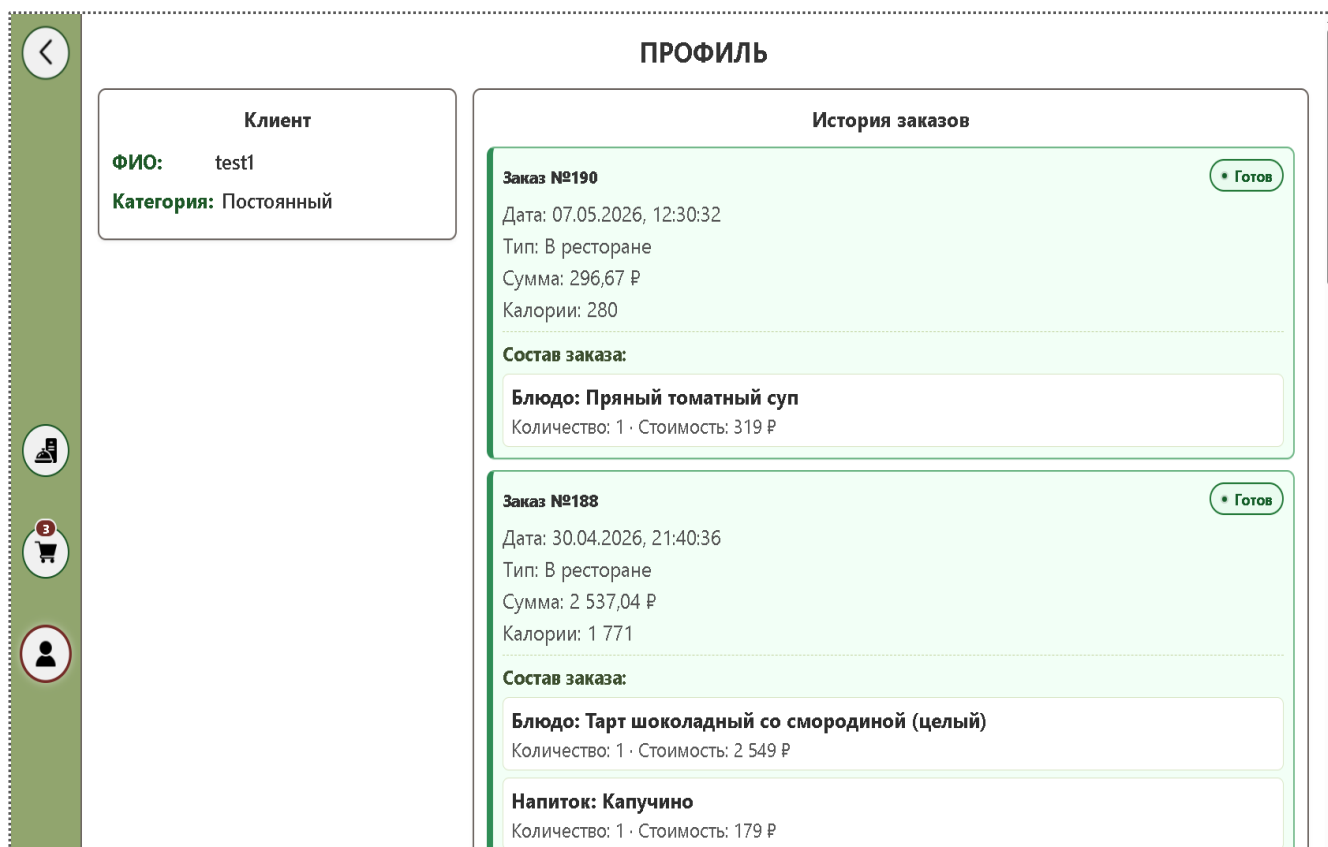


Рисунок 6.1.6 – Профиль клиента с историей заказов

6.2 Руководство для повара

Повар работает с системой через WPF-приложение. После запуска приложения сотрудник проходит авторизацию под учетной записью с соответствующей ролью, после чего получает доступ к производственным разделам. Основным рабочим экраном повара является очередь заказов, где отображаются заказы, ожидающие приготовления. Производственная очередь заказов повара представлена на рисунке 6.2.1.

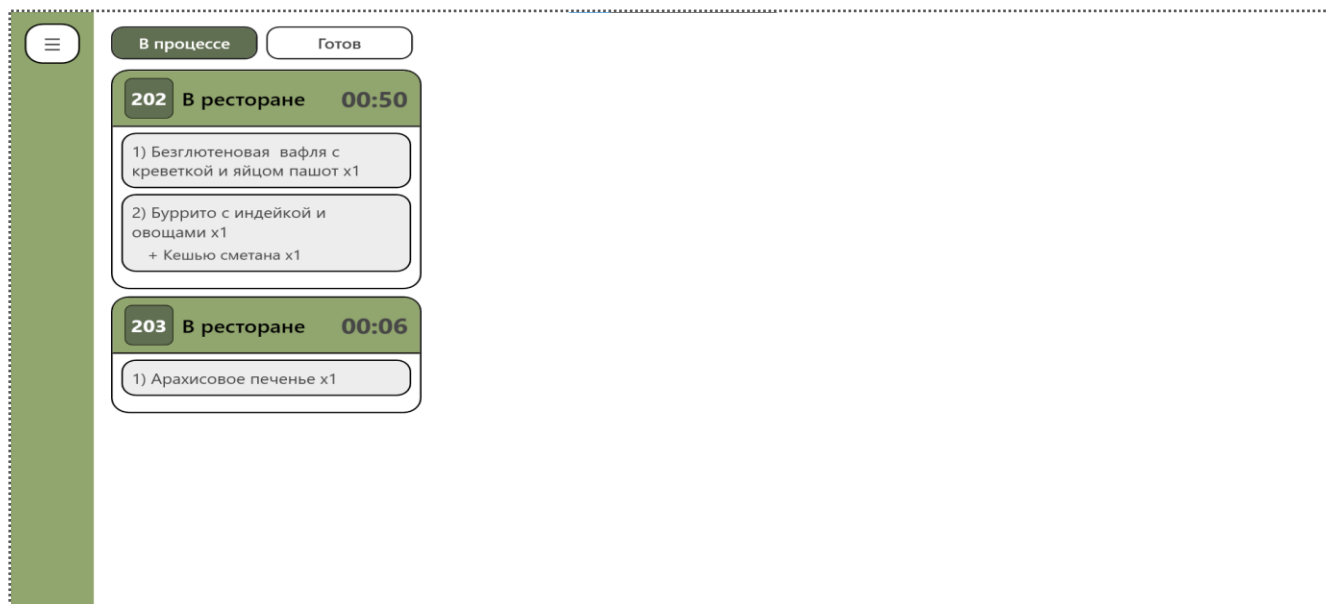


Рисунок 6.2.1 – Производственная очередь заказов повара

В очереди заказов повар выбирает конкретный заказ и открывает его подробное описание. В окне просмотра состава отображаются позиции заказа, количество, комментарии клиента и сведения, необходимые для приготовления. Окно просмотра состава заказа представлено на рисунке 6.2.2.

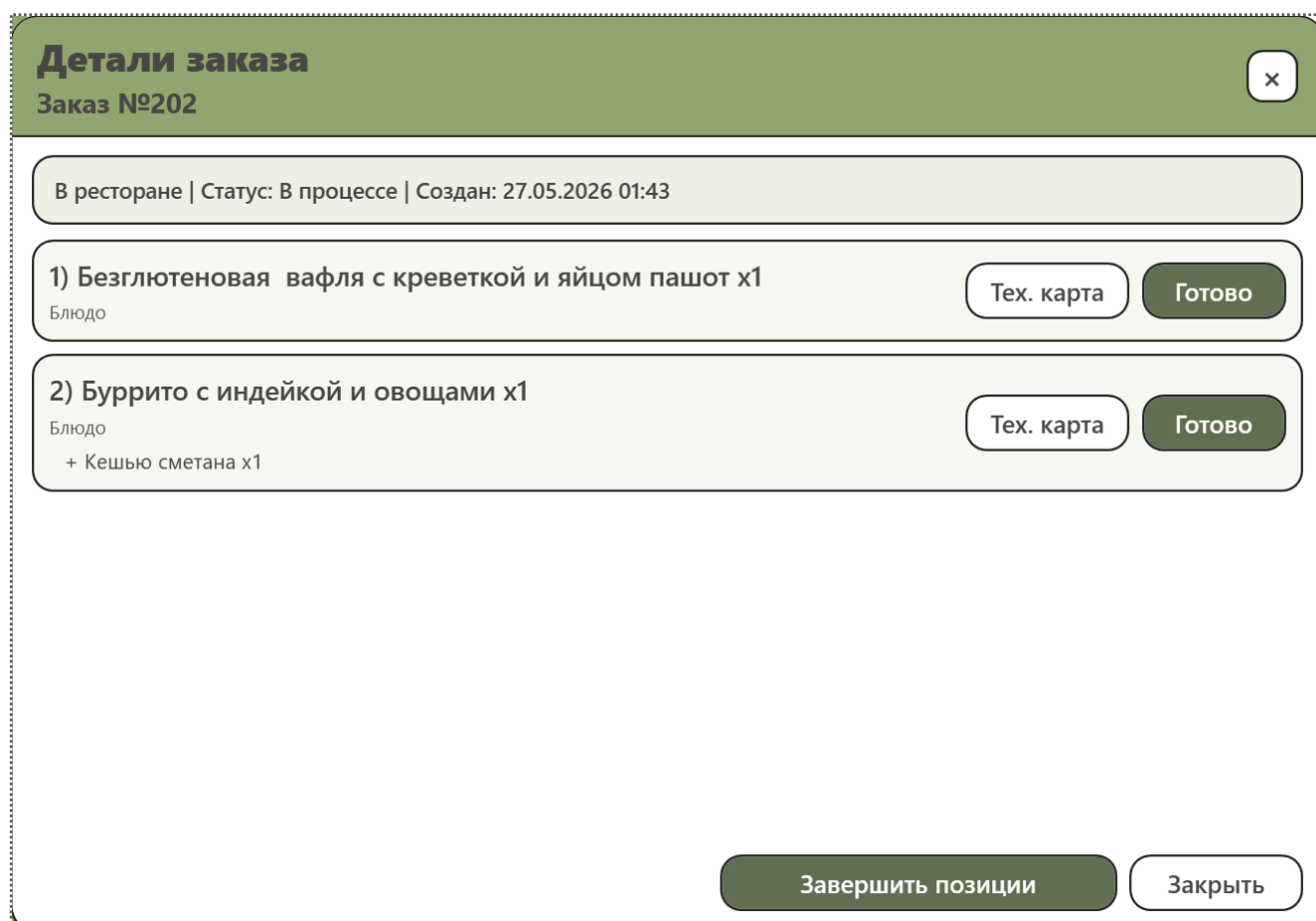


Рисунок 6.2.2 – Окно просмотра состава заказа

Для соблюдения рецептуры повар обращается к разделу технологических карт. В технологической карте указываются ингредиенты, нормы закладки, последовательность приготовления и связанные сведения по позиции меню. После завершения приготовления повар отмечает готовность позиции, что позволяет следующему сотруднику продолжить обработку заказа. Раздел технологических карт представлен на рисунке 6.2.3. Пример технологической карты представлен на рисунке 6.2.4.

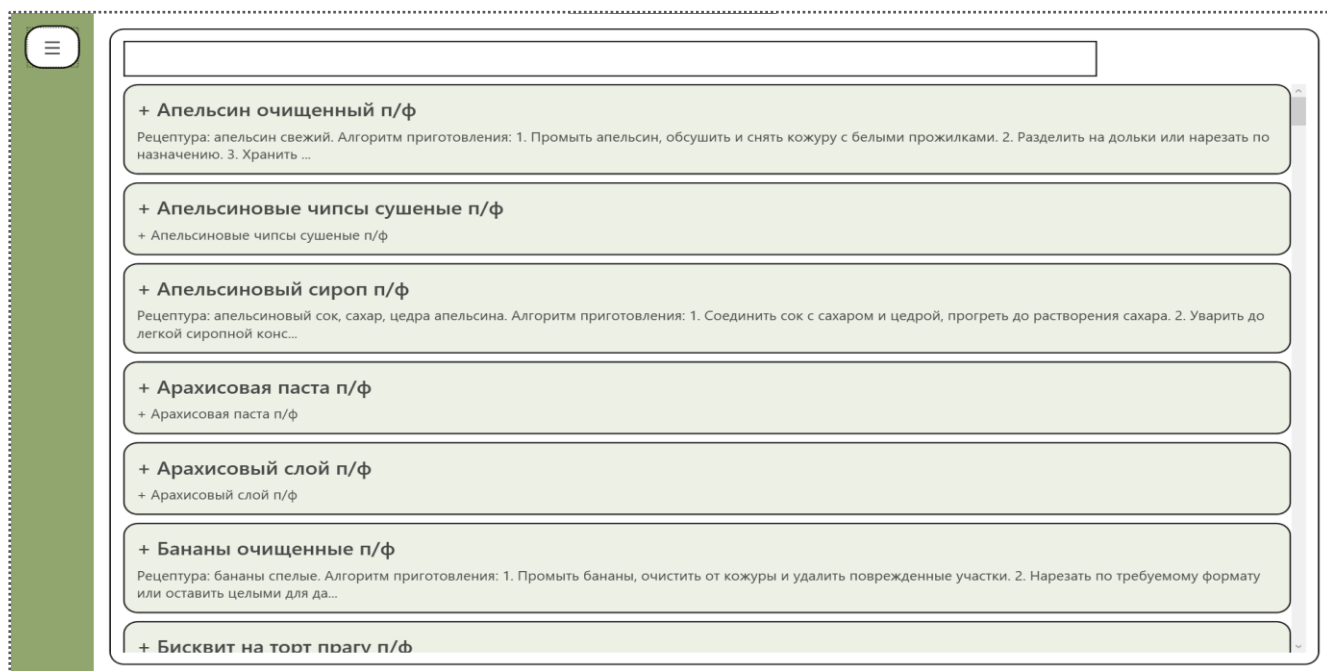


Рисунок 6.2.3 – Раздел технологических карт

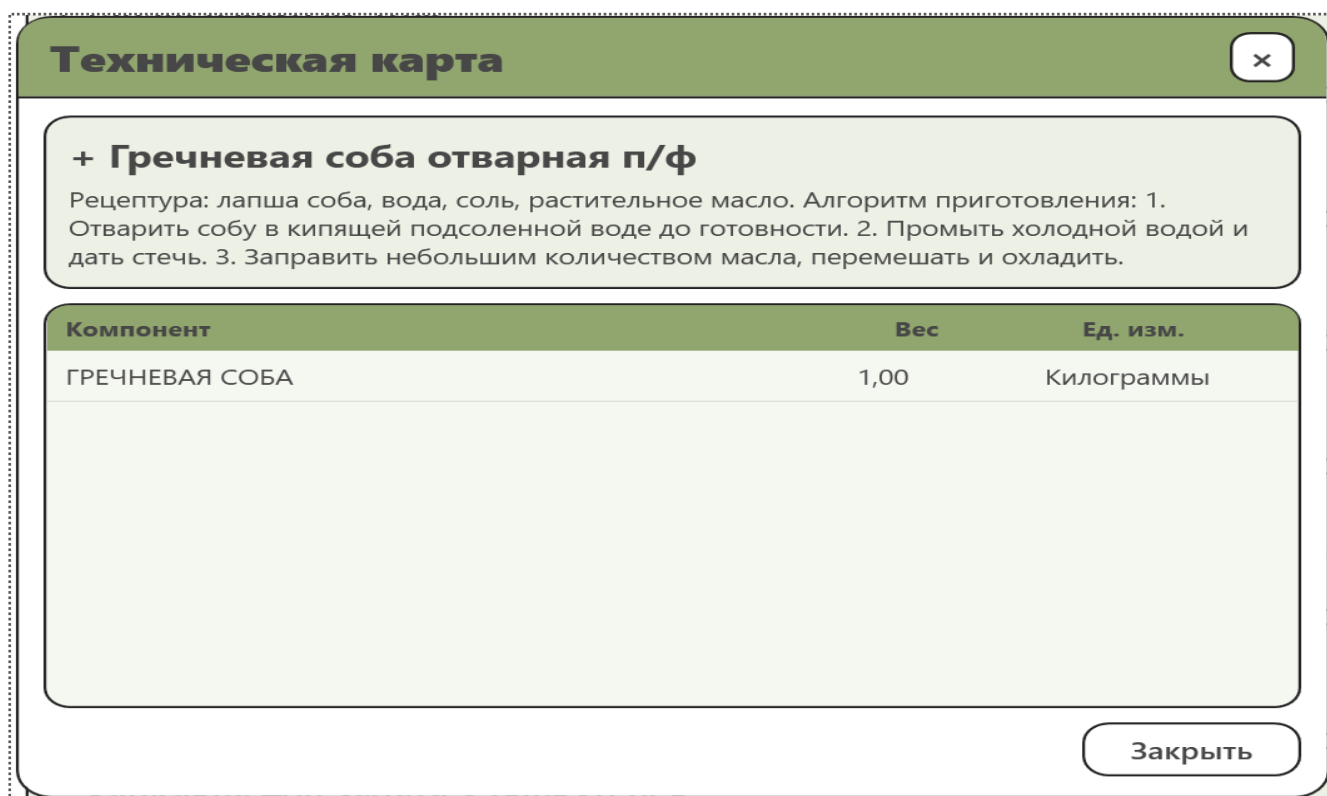


Рисунок 6.2.4 – Пример технической карты

6.3 Руководство для бариста-кассира

Бариста-кассир также выполняет вход в WPF-приложение под своей учетной записью. После авторизации сотруднику доступна очередь заказов, в которой отображаются заказы, требующие обработки, подготовки напитков, выдачи или изменения статуса. Очередь заказов бариста-кассира представлена на рисунке 6.3.1.

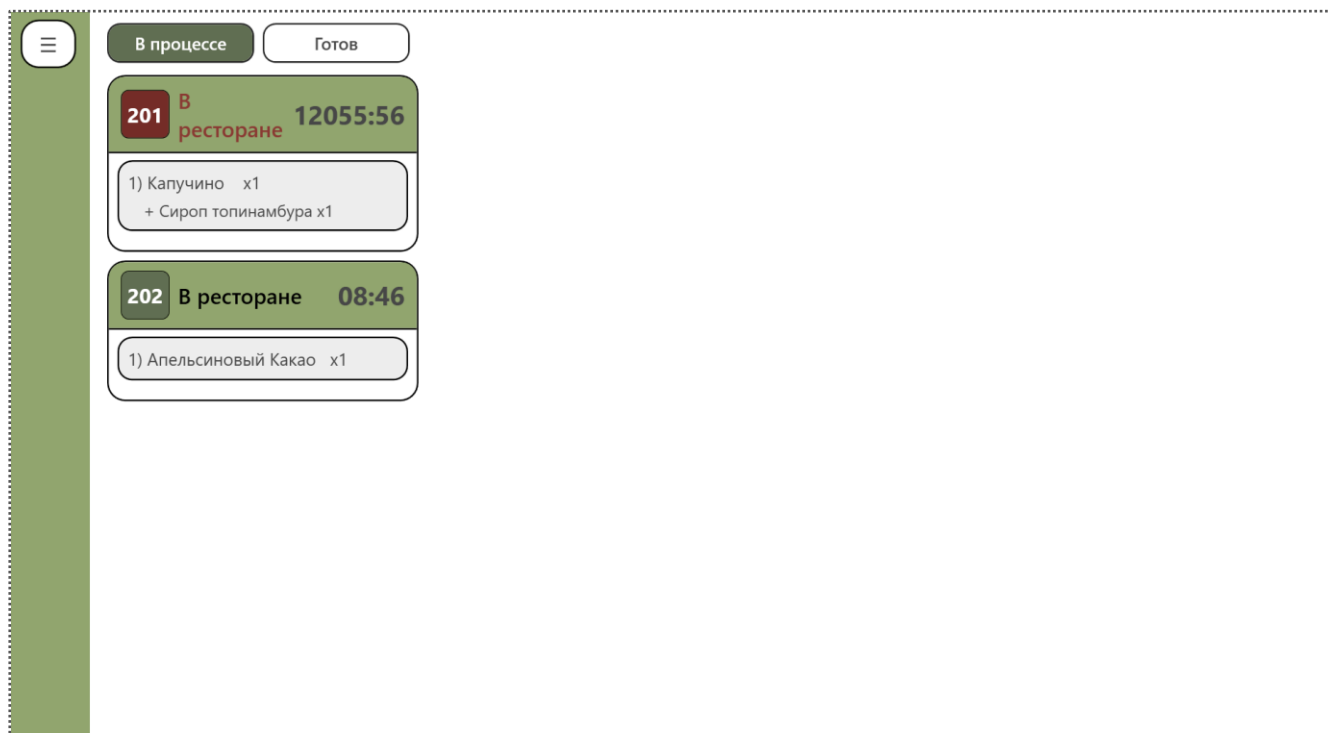


Рисунок 6.3.1 – Очередь заказов бариста-кассира

При работе с заказом бариста-кассир проверяет состав, комментарии клиента и текущий этап обработки. После выполнения нужного действия сотрудник изменяет статус заказа, например, переводит его в состояние готовности к выдаче или завершения. Изменение статуса заказа представлено на рисунке 6.3.2.

Детали заказа

Заказ №202

✕

В ресторане | Статус: В процессе | Создан: 27.05.2026 01:43

1) Апельсиновый Какао x1

Напиток

Тех. карта

Готово

Завершить позиции

Заккрыть

Рисунок 6.3.2 – Изменение статуса заказа

Если позиция временно недоступна для продажи, бариста-кассир использует раздел стоп-листа. В стоп-листе отображаются позиции, которые нельзя добавить в заказ из-за отсутствия ингредиентов, производственных ограничений или иных причин. Раздел стоп-листа представлен на рисунке 6.3.3.

☰

Стоп-лист

Добавить позицию

Поиск:

Голубая матча

Тип: Напиток
Статус: в стоп-листе
Категория: Неактивные
Ручной остаток: 0 порц.
Эффективный остаток: 0 порц.

Убрать из стоп-листа

Капучино

Тип: Напиток
Статус: ограничено по лимиту порций
Категория: Кофе
Ручной остаток: 20 порц.
Эффективный остаток: 20 порц.

Убрать из стоп-листа

Рисунок 6.3.3 – Раздел стоп-листа

6.4 Руководство для администратора

Администратор использует WPF-приложение для управления справочниками, сотрудниками, складскими данными и отчетами. После авторизации администратор выбирает нужный раздел в навигационной панели. Для работы с ассортиментом используется раздел управления позициями меню, где отображаются созданные блюда и напитки. Раздел управления позициями меню представлен на рисунке 6.4.1.

The screenshot displays the 'МЕНЮ' (Menu) management interface. On the left, a green sidebar contains a menu icon. The main area shows a list of menu items, each with a title, category, price, and availability status. The items are: 'Арахисовое печенье' (Peanut cookie) for 129 P, 'Баунти' (Bounty) for 169 P, and 'Безглютеновая вафля с креветкой и яйцом пашот' (Gluten-free waffle with shrimp and poached egg) for 459 P. Each item has 'Изменить' (Edit) and 'Удалить' (Delete) buttons. On the right, a 'Фильтры' (Filters) panel includes a 'Добавить' (Add) button, a 'Категории' (Categories) section with buttons for 'Все' (All), 'Блюда' (Dishes), 'Напитки' (Beverages), and 'Добавки' (Add-ons), and a 'Доступность' (Availability) dropdown menu set to 'Все' (All).

Рисунок 6.4.1 – Раздел управления позициями меню

При добавлении или редактировании позиции меню администратор открывает модальное окно, заполняет название, категорию, цену, описание и другие обязательные сведения. Перед сохранением система проверяет корректность введенных данных. Окно добавления или редактирования позиции меню представлено на рисунке 6.4.2.

The screenshot shows the 'Изменить позицию меню' (Edit menu item) modal window. It has a green header with a close button. The form is divided into several sections: 'Основное' (Basic) with fields for 'Тип позиции' (Type of item) set to 'Блюдо' (Dish), 'Название' (Name) set to 'Безглютеновая вафля с креветкой и яйцом пашот', 'Категория' (Category) set to 'Вафли' (Waffles), and 'Единица измерения' (Unit of measurement) set to 'Порция' (Portion); 'Цена и размер' (Price and size) with a 'Цена, P' (Price, P) field set to 459 and a checked 'Доступность' (Availability) checkbox labeled 'Позиция доступна в меню' (Item is available in menu); 'КБЖУ' (Calories, Fat, Protein, Carbohydrates) with 'Ккал' (Calories) set to 320 and 'Килоджоули' (Kilojoules) set to 1340; and 'Связи и изображение' (Links and image) with a 'Техкарта' (Technical card) field. At the bottom, there are 'Сохранить' (Save) and 'Отмена' (Cancel) buttons.

Рисунок 6.4.2 – Окно добавления или редактирования позиции меню

Для контроля сырья администратор открывает раздел складских остатков. В данном разделе отображаются ингредиенты, единицы измерения, количество на складе и сведения, необходимые для учета поступлений и списаний. Раздел складских остатков представлен на рисунке 6.4.3.

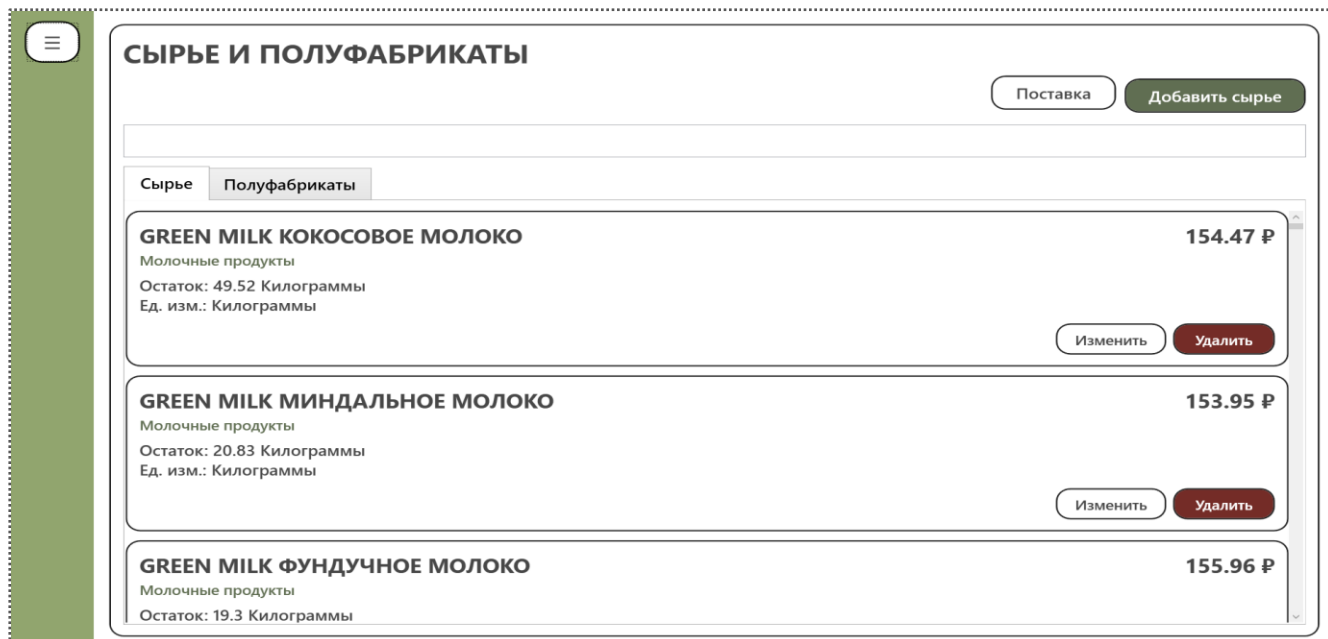


Рисунок 6.4.3 – Раздел складских остатков

Управление сотрудниками выполняется в отдельном разделе WPF-приложения. Администратор просматривает список сотрудников, добавляет новые учетные записи, редактирует данные и назначает роли, определяющие доступ к функциям системы. Раздел управления сотрудниками представлен на рисунке 6.4.4.



Рисунок 6.4.4 – Раздел управления сотрудниками

Для анализа деятельности кофейни администратор использует раздел отчетов и аналитики. В нем отображаются показатели продаж, сведения по заказам, складским операциям и другим данным, необходимым для управленческого контроля. Раздел отчетов и аналитики представлен на рисунке 6.4.5.



Рисунок 6.4.5 – Раздел отчётов и аналитики

Операции удаления записей выполняются только после дополнительного подтверждения. Такой порядок снижает риск случайного удаления позиций меню, сотрудников, справочников и связанных данных. Если сервер запрещает удаление из-за существующих связей, система отображает сообщение об ошибке.

6.5 Обработка ошибок и завершение работы

При некорректном вводе данных система отображает сообщение с описанием проблемы. Пользователь соответствующей роли проверяет обязательные поля, формат числовых значений, выбранные элементы списков и доступность соединения с сервером. Если операция недоступна из-за недостаточных прав, система ограничивает выполнение действия и сообщает о невозможности доступа.

После завершения работы пользователь выходит из учетной записи или закрывает приложение. Завершение сеанса предотвращает несанкционированный доступ к клиентским данным, заказам, складским сведениям и административным функциям системы.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы разработан программный комплекс Garden Nook для автоматизации работы кофейни здорового питания. Система включает клиентский веб-интерфейс, WPF-приложение для сотрудников, серверный API на ASP.NET Core и базу данных Microsoft SQL Server. Поставленная задача решена в полном объеме: реализованы основные функциональные модули, необходимые для оформления заказов, управления производственными процессами, складского учета и администрирования справочников.

В ходе работы были достигнуты следующие конкретные результаты:

Проведен анализ предметной области, ролей пользователей и бизнес-процессов кофейни здорового питания. На основе анализа выделены ключевые сценарии работы клиента, повара, бариста-кассира и администратора, а также определены основные информационные объекты системы: позиции меню, заказы, ингредиенты, технологические карты, складские остатки, сотрудники и клиенты.

Спроектирована архитектура программного комплекса с разделением на клиентский веб-интерфейс, служебное WPF-приложение, серверный API и реляционную базу данных. Такое разделение позволило отделить пользовательские сценарии клиента от внутренних операций сотрудников кофейни и централизовать обработку данных на серверной стороне.

Реализована авторизация пользователей и разграничение доступа по ролям. Клиент получает доступ к меню, корзине, профилю и истории заказов; повар работает с производственной очередью и технологическими картами; бариста-кассир обрабатывает заказы и стоп-лист; администратор управляет справочниками, сотрудниками, клиентами, скидками, складом и отчетами.

Реализован клиентский сценарий веб-интерфейса: просмотр меню, открытие карточки позиции с составом и пищевой ценностью, добавление товаров в корзину, изменение состава заказа, оформление заказа, просмотр профиля и истории заказов. Данные функции обеспечивают полный цикл взаимодействия клиента с кофейней без участия администратора.

Реализована обработка заказов сотрудниками через WPF-приложение. Сотрудники могут просматривать очередь заказов, открывать состав заказа, учитывать комментарии клиента, работать с технологическими картами и изменять статусы выполнения, что сокращает время передачи информации между залом, кассой и производственной зоной.

Реализованы административные функции управления меню, категориями, ингредиентами, заготовками, технологическими картами, сотрудниками, клиентами, скидками и категориями клиентов. Для операций удаления предусмотрено подтверждение, а серверная часть ограничивает удаление данных, связанных с другими объектами системы.

Реализован складской контур, включающий учет остатков ингредиентов, работу со стоп-листом, заготовками и актами списания. Это позволяет контролировать наличие сырья, фиксировать расход продуктов и предотвращать продажу позиций, которые временно недоступны для приготовления.

Реализованы отчеты и аналитические представления для контроля работы кофейни. Администратор получает сведения о заказах, продажах, складских операциях и связанных показателях, необходимых для оперативного управления ассортиментом и ресурсами.

Проведено тестирование программного комплекса по основным направлениям: функциональное тестирование пользовательских сценариев, проверка интерфейсов веб- и WPF-приложений, тестирование API-взаимодействия, проверка бизнес-логики, выборочные unit-проверки, тестирование безопасности и базовая оценка производительности. Результаты проверок подтвердили корректность основных сценариев работы системы.

Фрагменты исходного кода программного комплекса с поясняющим текстом приведены в Приложении Д.

Практическая значимость разработанного программного комплекса заключается в сокращении ручной обработки заказов, снижении количества ошибок при работе с рецептурами и складскими остатками, ускорении передачи информации между клиентским интерфейсом и сотрудниками кофейни, а также в повышении прозрачности состава и пищевой ценности позиций меню для клиента.

Дальнейшее развитие Garden Nook может включать интеграцию с кассовым оборудованием, расширение аналитических отчетов, внедрение уведомлений о прогнозируемом дефиците ингредиентов и разработку мобильного интерфейса для клиентов. Поставленная задача разработки программного комплекса Garden Nook выполнена в полном объеме.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- 1 Sommerville I. Software Engineering. 10th ed. – Boston: Pearson, 2016.
- 2 ISO/IEC 25010:2011. Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models.
- 3 Larman C. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development. 3rd ed. – Upper Saddle River: Prentice Hall, 2004.
- 4 ISO/IEC/IEEE 29148:2018. Systems and software engineering – Life cycle processes – Requirements engineering.
- 5 Документация WPF для .NET [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/> (дата обращения: 27.05.2026).
- 6 Создание Web API с ASP.NET Core [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/aspnet/core/web-api/> (дата обращения: 27.05.2026).
- 7 Документация ASP.NET Core [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/aspnet/core/> (дата обращения: 27.05.2026).
- 8 Привязка данных в WPF [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/data/> (дата обращения: 27.05.2026).
- 9 Обзор языка XAML в WPF [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/xaml/> (дата обращения: 27.05.2026).
- 10 Документация Entity Framework Core [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/ef/core/> (дата обращения: 27.05.2026).
- 11 Документация Microsoft SQL Server [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/sql/sql-server/> (дата обращения: 27.05.2026).
- 12 ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения.
- 13 Richardson L., Ruby S. RESTful Web Services. – Sebastopol: O'Reilly Media, 2007.
- 14 Fowler M. Patterns of Enterprise Application Architecture. – Boston: Addison-Wesley, 2002.
- 15 Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. – Boston: Prentice Hall, 2017.
- 16 Pressman R. S., Maxim B. R. Software Engineering: A Practitioner's Approach. 9th ed. – New York: McGraw-Hill Education, 2019.
- 17 Разделы безопасности ASP.NET Core [Электронный ресурс]. – URL: <https://learn.microsoft.com/ru-ru/aspnet/core/security/> (дата обращения: 27.05.2026).
- 18 OWASP Top 10:2021. The Ten Most Critical Web Application Security Risks [Электронный ресурс]. – URL: <https://owasp.org/Top10/> (дата обращения: 27.05.2026).

19 Руководство по архитектуре обработки запросов SQL Server [Электронный ресурс].
– URL: <https://learn.microsoft.com/ru-ru/sql/relational-databases/query-processing-architecture-guide>
(дата обращения: 27.05.2026).

20 Эффективные запросы в Entity Framework Core [Электронный ресурс]. – URL:
<https://learn.microsoft.com/ru-ru/ef/core/performance/efficient-querying> (дата обращения:
27.05.2026).

ПРИЛОЖЕНИЯ

Приложение А

Техническое задание на разработку программного комплекса «Garden Nook»

Организация-заказчик: Кофейня здорового питания "Garden Room".

Обоснование разработки. Разработка комплекса «Garden Nook» направлена на комплексную автоматизацию ключевых бизнес-процессов кофейни здорового питания. Внедрение системы позволит сократить время обслуживания клиентов, снизить нагрузку на персонал, минимизировать ошибки при приеме заказов, а также повысить количество посетителей за счет персонализированных предложений и системы лояльности. В перспективе это приведет к увеличению повторных покупок, улучшению общего клиентского опыта и оптимизации работы персонала.

Область применения программного средства. Назначение разрабатываемого комплекса «Garden Nook» – предоставление клиентам удобного инструмента для предварительного заказа и участия в программе лояльности, а также обеспечение персонала кофейни desktop-приложением для полного контроля бизнес-процессов и интеллектуальной собственности организации.

Функциональные требования

1. Административное desktop-приложение:
 - Управление товарами и категориями меню (добавление, редактирование, удаление позиций).
 - Контроль ингредиентов и остатков продукции в реальном времени.
 - Управление техническими картами полуфабрикатов и готовых товаров.
 - Обработка заказов и изменение их статусов.
 - Просмотр аналитики по продажам, популярности позиций и выручке.
 - Настройка и управление программой лояльности.
 - Управление составом персонала и правами доступа сотрудников
2. Гостевое веб-приложение:
 - Регистрация и авторизация пользователей по номеру телефона.
 - Просмотр актуального меню с возможностью фильтрации и поиска.
 - Формирование предварительного заказа.
 - Добавление комментария к заказу.
 - Отслеживание статуса заказа.
 - Просмотр истории заказов.
 - Переходы между уровнями программы лояльности.

Рисунок А.1 – Скан технического задания

Условия разработки

Технологии разработки

- Серверная часть – C#, ASP.NET Core. Используется для разработки серверной части системы и реализации веб-сервисов. Платформа ASP.NET Core обеспечивает создание высокопроизводительных и масштабируемых веб-приложений и предоставляет инструменты для обработки HTTP-запросов и взаимодействия с базой данных.
- Архитектурный стиль – REST API. Применяется для организации взаимодействия между клиентской и серверной частями системы. Использование REST-архитектуры обеспечивает стандартизированный обмен данными по протоколу HTTP.
- База данных – Microsoft SQL Server (MS SQL). Используется для хранения и управления данными системы. СУБД обеспечивает надежность хранения данных, поддержку транзакций и эффективную обработку запросов.
- Клиентская часть веб-приложения – ASP.NET Core. Используется для разработки пользовательского интерфейса веб-приложения и обеспечения взаимодействия пользователя с системой через веб-браузер.
- Административное приложение – WPF (C#). Нативное desktop-приложение для Windows, предназначенное для сотрудников кофейни и администраторов системы.

Сроки разработки ПС

Ожидается завершение разработки программного комплекса «Garden Nook» до июня 2026 года.

Условия тестирования

Модульное тестирование. Проверка корректности работы отдельных компонентов системы, включая расчет стоимости заказа, учет ингредиентов, применение скидок и работу программы лояльности.

Типы данных для тестирования ПС

- 1) Пользовательские данные – тестовые профили с номерами телефонов и именами.
- 2) Данные меню – товары с категориями, ценами и подробными описаниями.
- 3) Данные ингредиентов – ингредиенты с наименованиями и информацией об остатках.
- 4) Данные заказов – тестовые заказы с различными статусами и составами.
- 5) Данные технических карт – информация о рецептуре и составе товаров.

Управляющий организации:

Шутова А. В.



Рисунок А.2 – Скан технического задания

Приложение Б

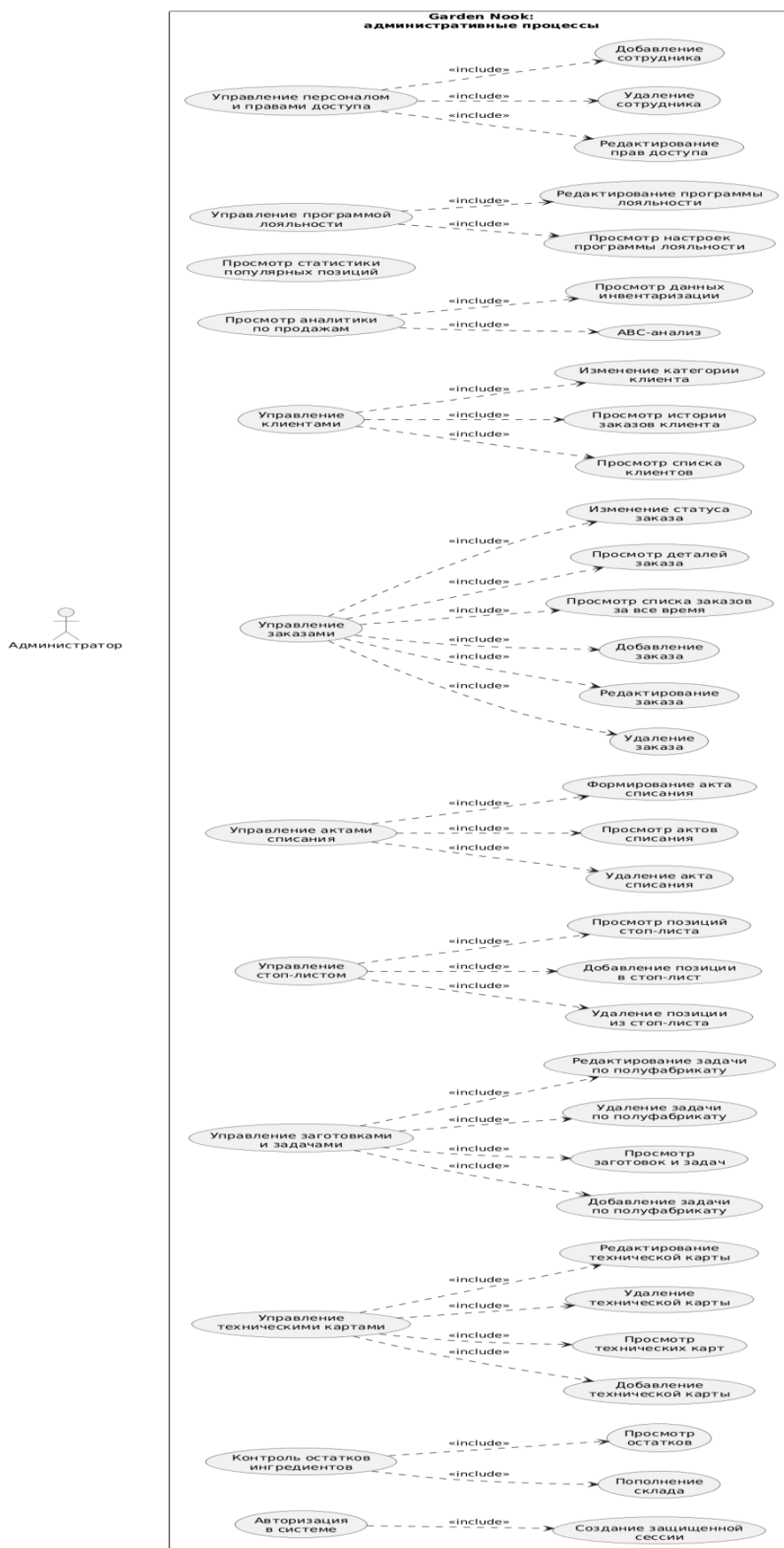


Рисунок Б.1 – Диаграмма вариантов использования администратора

Приложение В

Алгоритм действий по обработке заказа сотрудником

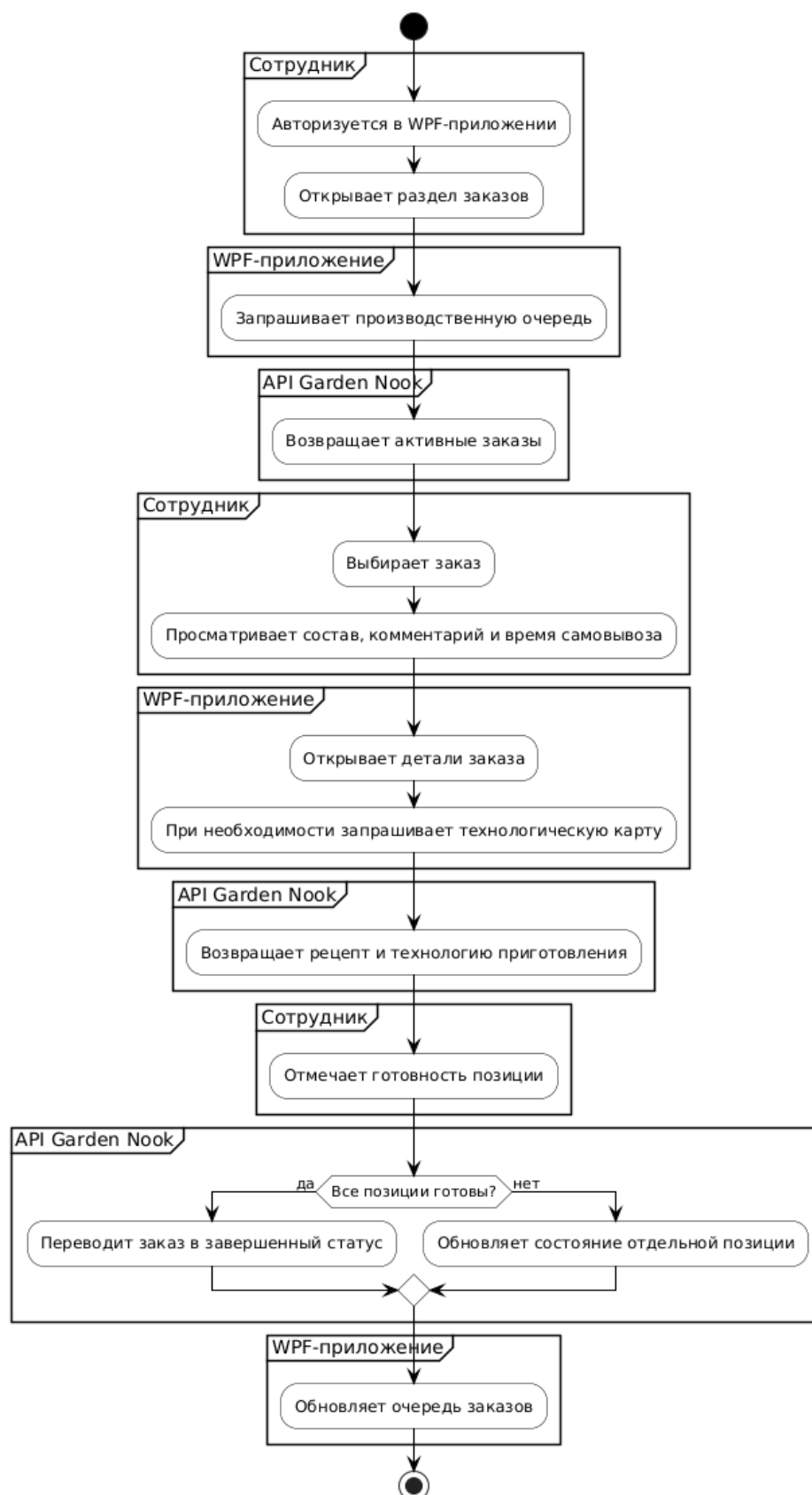


Рисунок В.1 – Алгоритм действий по обработке заказа сотрудником

Алгоритм действий по управлению меню

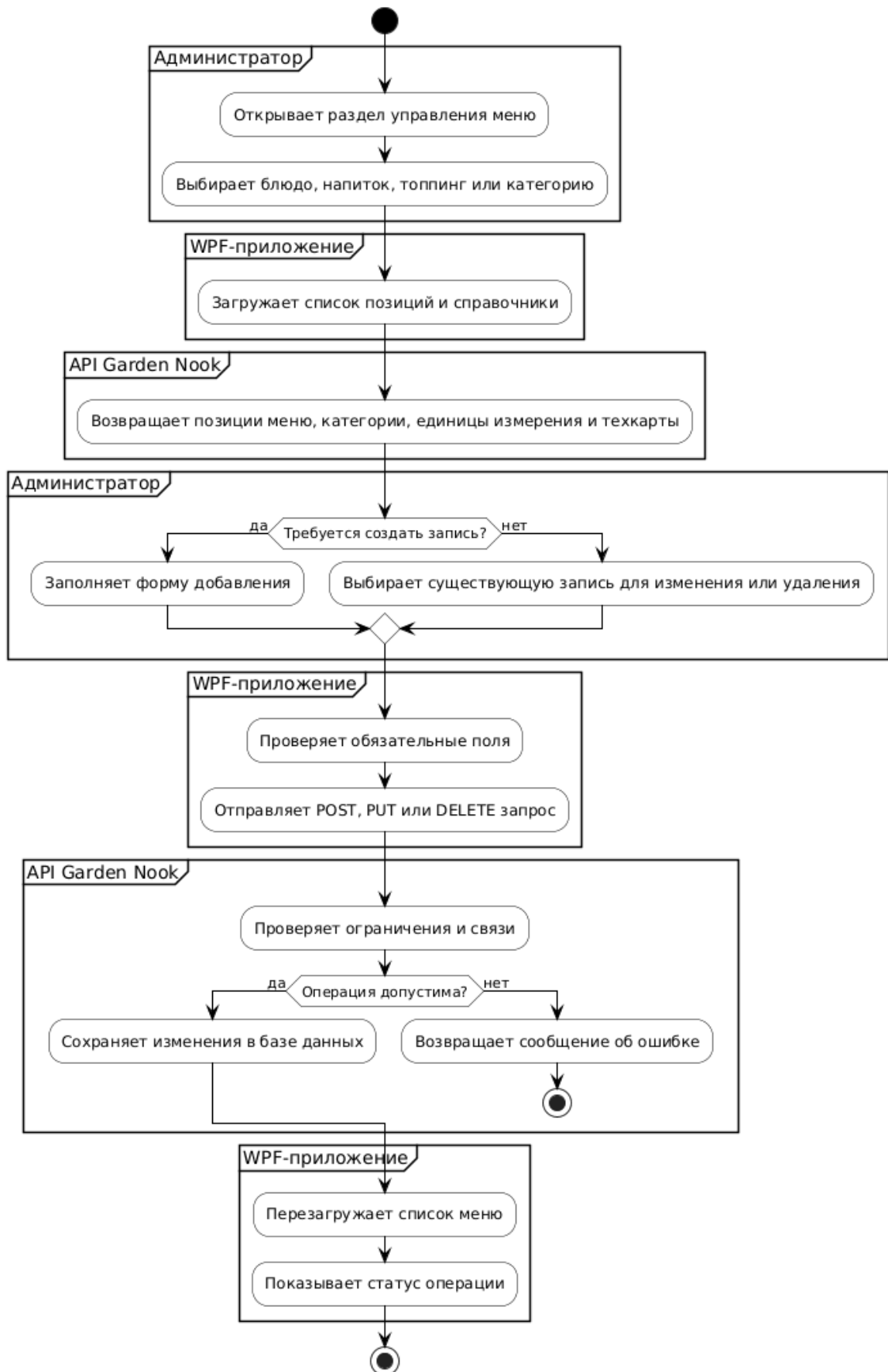


Рисунок В.2 – Алгоритм действий по управлению меню

Алгоритм действий по работе с технологическими картами

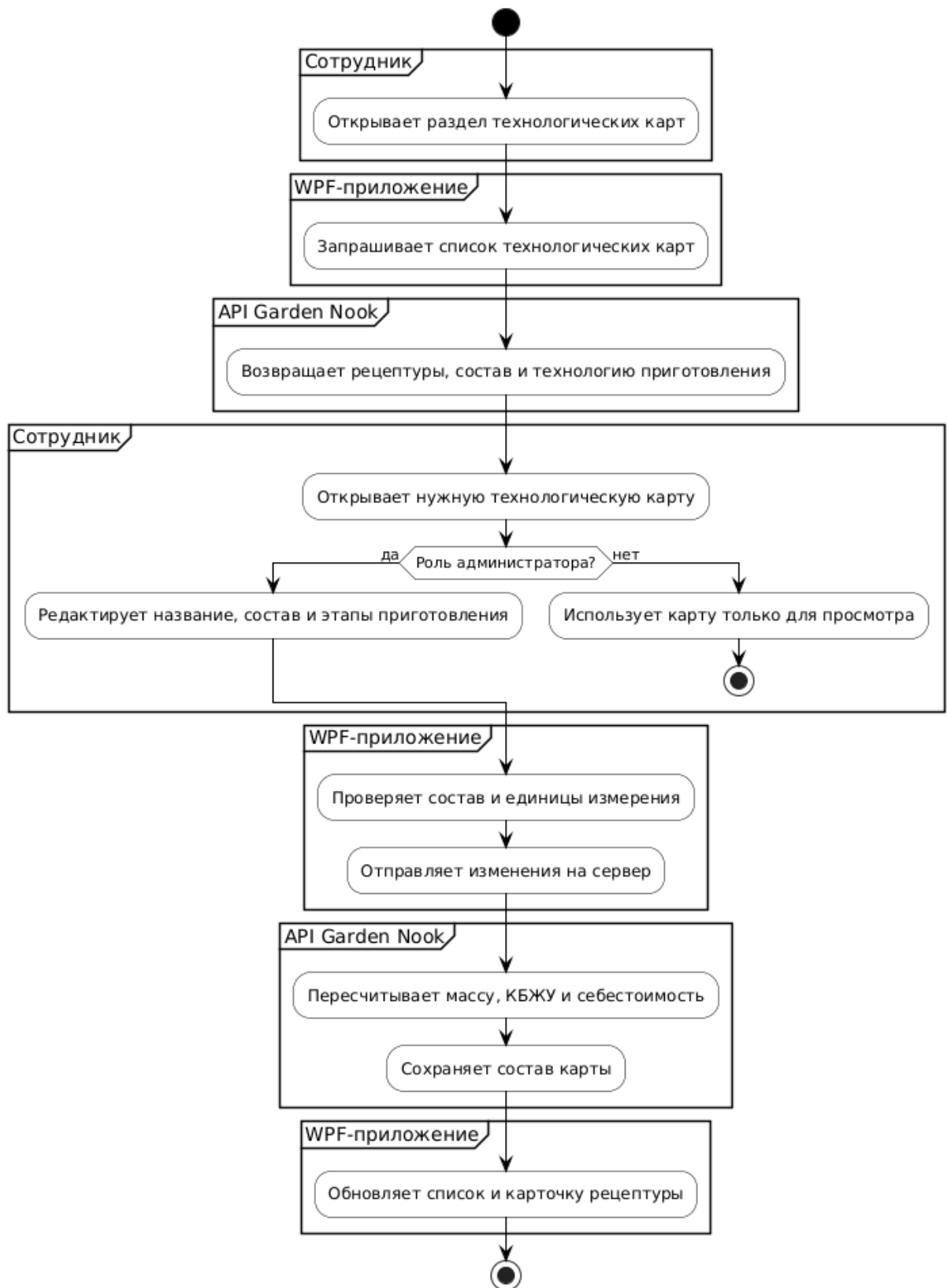


Рисунок В.3 – Алгоритм действий по работе с технологическими картами

Алгоритм действий по управлению складом и стоп-листом

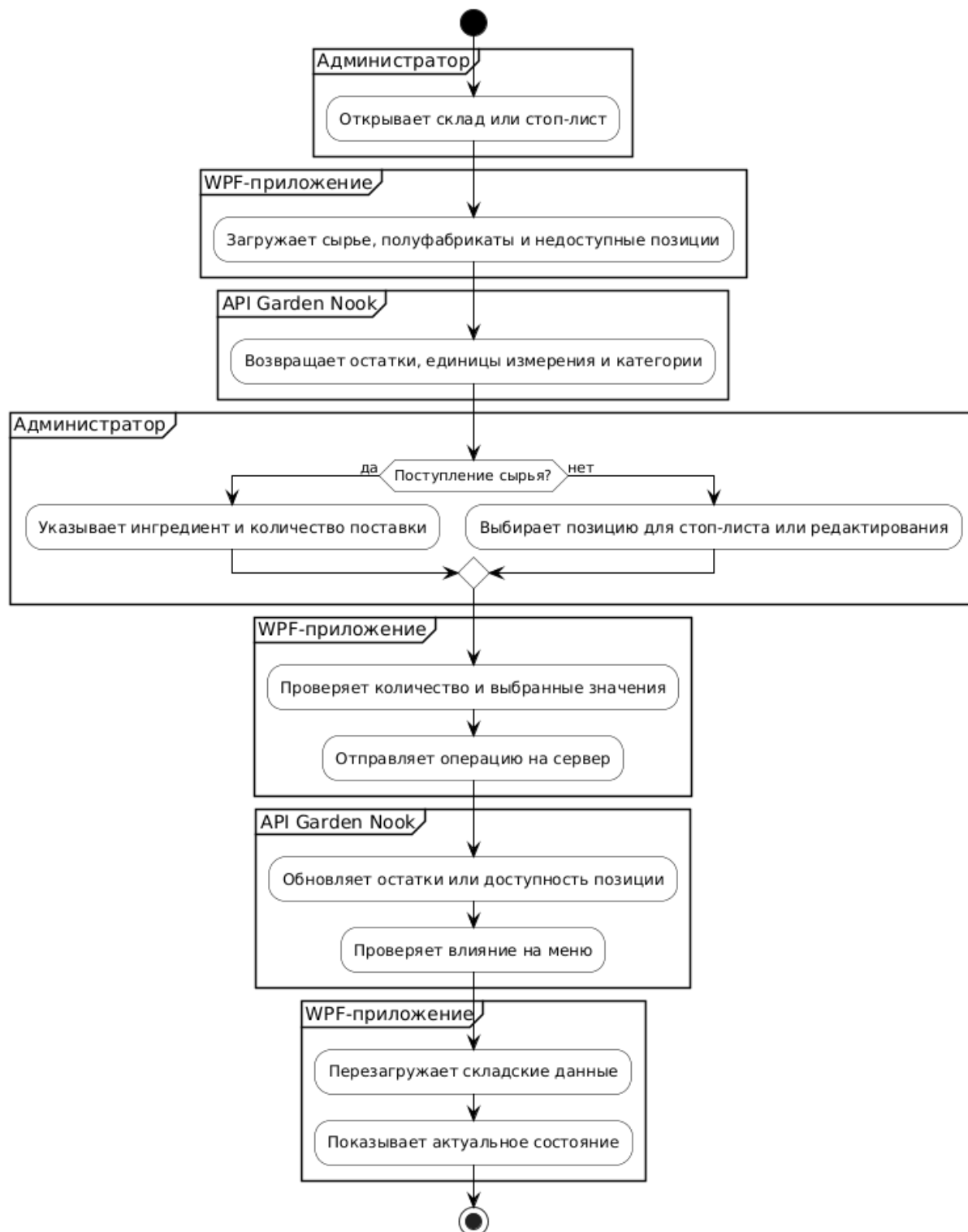


Рисунок В.4 – Алгоритм действий по управлению складом и стоп-листом

Алгоритм действий по списанию и заготовкам

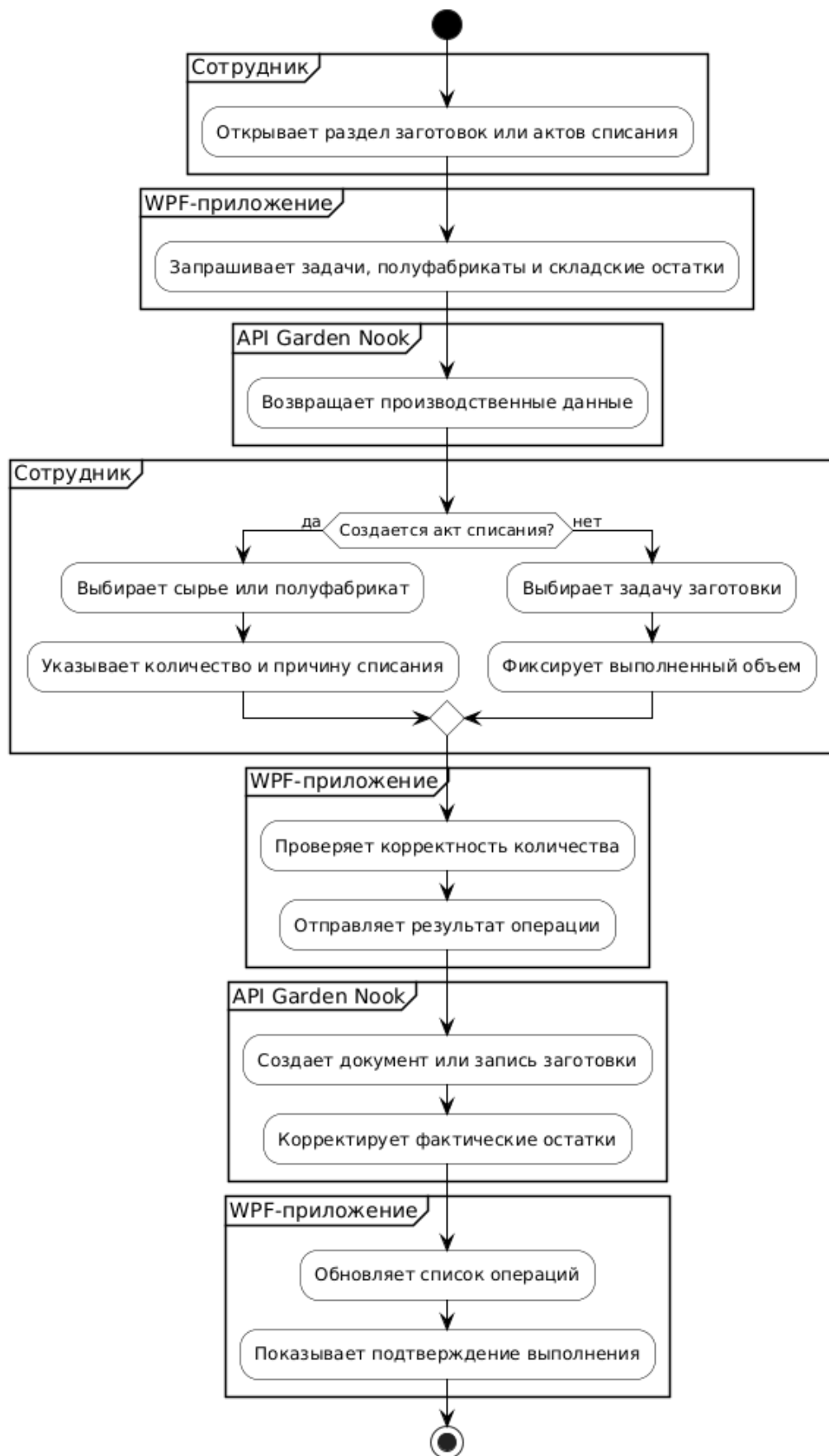


Рисунок В.5 – Алгоритм действий по списанию и заготовкам

Алгоритм действий по аналитике и администрированию

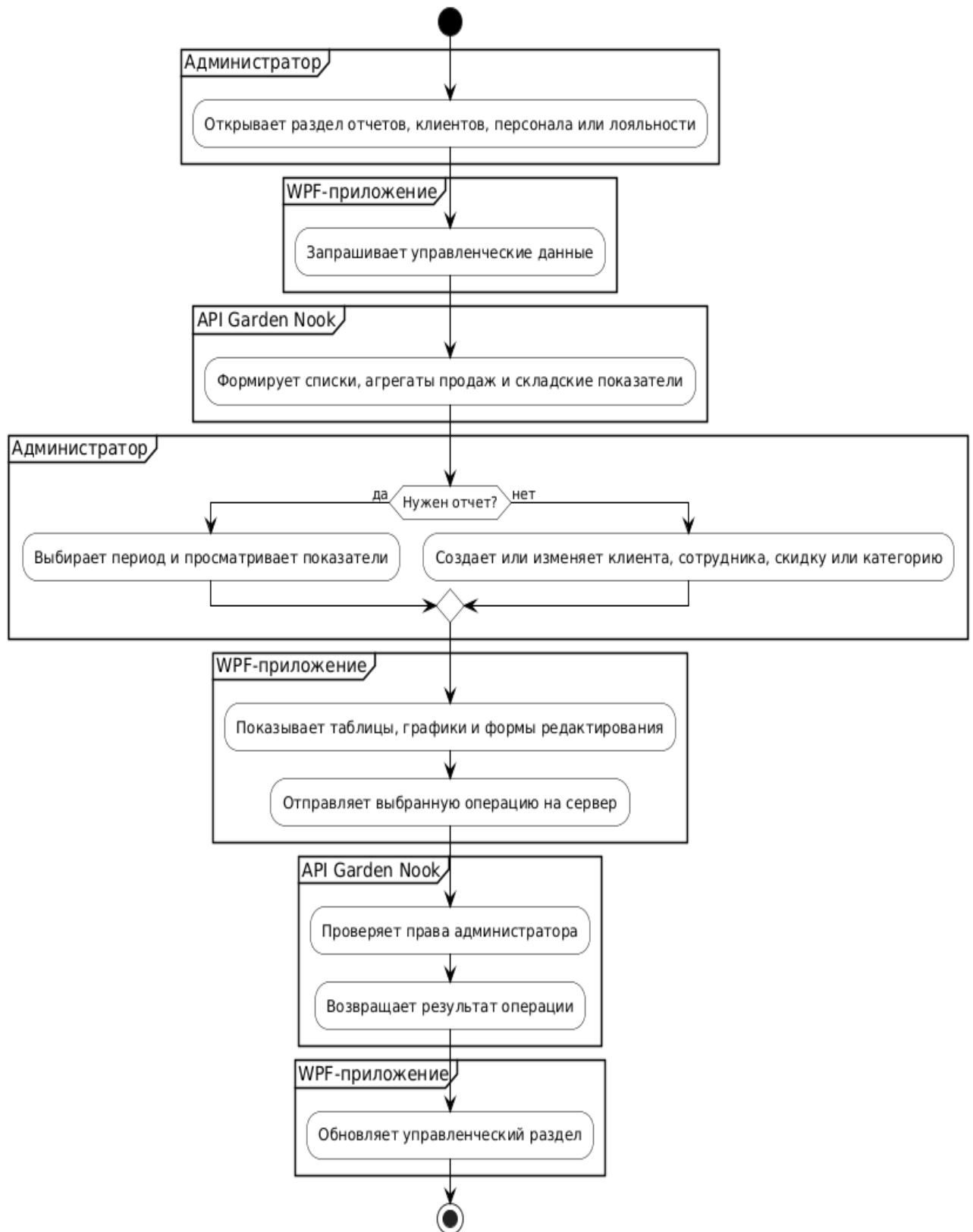


Рисунок В.6 – Алгоритм действий по аналитике и администрированию

Приложение Г

```
using GardenNookApi.Entities;
using GardenNookApi.Services;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Options;
using System.Globalization;
using System.Security.Claims;
using TransferModels.Kitchen;

namespace GardenNookApi.Controllers
{
    [ApiController]
    [Route("api/kitchen")]
    [Authorize]
    public class KitchenController : ControllerBase
    {
        private const string ActiveStatusTokenRu =
            "процесс";
        private const string ActiveStatusTokenEn =
            "process";
        private const string KitchenOrdersStatusActive =
            "active";
        private const string KitchenOrdersStatusReady =
            "ready";
        private const string ReadyStatusNameRu =
            "Готов";
        private const string ReadyStatusTokenRu =
            "готов";
        private const string ReadyStatusTokenEn = "ready";
        private const string CancelledStatusNameRu =
            "Отменен";
        private const string CancelledStatusTokenRu =
            "отмен";
        private const string CancelledStatusTokenEn =
            "cancel";
        private const string AdminRole =
            "Администратор";
        private const string CookRole = "Повар";
        private const string BaristaRole = "Бариста";
        private const string DishToppingCategoryToken =
            "к блюду";
        private const string DrinkToppingCategoryToken =
            "к напитку";
        private const string
            SystemRecommendationComment = "Рекомендовано
            системой";
        private const string
            SystemRecommendationSnoozePrefix =
            "Рекомендовано системой: скрыто до ";
        private const int CriticalDays = 14;
        private const int UnitGramsId = 2;
        private const int UnitMillilitersId = 3;
        private const int UnitPiecesId = 4;
        private const int UnitKilogramsId = 5;
        private const int UnitLitersId = 6;
        private const decimal DecimalEpsilon =
            0.000001m;

        private readonly ApplicationDbContext _db;
```

```
        private readonly IPreparationStockService
            _stockService;
        private readonly KitchenPickupFilterOptions
            _pickupFilterOptions;

        public KitchenController(
            ApplicationDbContext db,
            IPreparationStockService stockService,
            IOptions<KitchenPickupFilterOptions>
            pickupFilterOptions)
        {
            _db = db;
            _stockService = stockService;
            _pickupFilterOptions =
                pickupFilterOptions?.Value ?? new
                KitchenPickupFilterOptions();
        }

        [HttpGet("orders")]
        public async
            Task<ActionResult<KitchenOrdersResponse>>
            GetOrders([FromQuery] string? status = null)
        {
            var statusFilter =
                NormalizeKitchenOrdersStatus(status);
            if (statusFilter == null)
            {
                return BadRequest("Неизвестный фильтр
                статуса заказов");
            }

            var includeCompletedItems = statusFilter ==
                KitchenOrdersStatusReady;
            var today = DateTime.Today;
            var tomorrow = today.AddDays(1);

            var ordersQuery = _db.Orders
                .AsNoTracking()
                .Where(o =>
                    o.Status != null &&
                    o.Status.Name != null);

            ordersQuery = statusFilter ==
                KitchenOrdersStatusReady
                ? ordersQuery.Where(o =>
                    (EF.Functions.Like(o.Status!.Name!.ToLower(),
                    $"%{ReadyStatusTokenRu}%") ||

                    EF.Functions.Like(o.Status.Name.ToLower(),
                    $"%{ReadyStatusTokenEn}%")) &&
                    o.CreatedAt >= today &&
                    o.CreatedAt < tomorrow)
                : ordersQuery.Where(o =>
                    EF.Functions.Like(o.Status!.Name!.ToLower(),
                    $"%{ActiveStatusTokenRu}%") ||

                    EF.Functions.Like(o.Status.Name.ToLower(),
                    $"%{ActiveStatusTokenEn}%"));
```

```

var orderSources = await ordersQuery
    .OrderBy(o => o.CreatedAt)
    .ThenBy(o => o.Id)
    .Select(o => new
    {
        o.Id,
        o.Comment,
        o.CreatedAt,
        o.PickupAt,
        OrderType = o.OrderType != null ?
o.OrderType.Name : null,
        Status = o.Status != null ? o.Status.Name :
null
    })
    .ToListAsync();

if (orderSources.Count == 0)
{
    return Ok(new KitchenOrdersResponse());
}

var now = DateTime.Now;
var pickupWindow =
    TimeSpan.FromMinutes(Math.Max(0,
        _pickupFilterOptions.WindowMinutes));

var filteredOrderSources = orderSources
    .Where(o =>
        !o.PickupAt.HasValue ||
        IsWithinPickupWindow(o.PickupAt.Value,
now, pickupWindow))
    .ToList();

if (filteredOrderSources.Count == 0)
{
    return Ok(new KitchenOrdersResponse());
}

var overduePickupSources =
filteredOrderSources
    .Where(o => o.PickupAt <= now)
    .OrderByDescending(o => o.PickupAt)
    .ThenBy(o => o.Id)
    .ToList();

var futurePickupSources = filteredOrderSources
    .Where(o => o.PickupAt > now)
    .OrderBy(o => o.PickupAt)
    .ThenBy(o => o.Id)
    .ToList();

var noPickupSources = filteredOrderSources
    .Where(o => !o.PickupAt.HasValue)
    .OrderBy(o => o.CreatedAt)
    .ThenBy(o => o.Id)
    .ToList();

var orderedOrderSources =
overduePickupSources
    .Concat(futurePickupSources)
    .Concat(noPickupSources)
    .ToList();

var orderIds = orderedOrderSources

```

```

    .Select(o => o.Id)
    .ToList();

var ordersById =
orderedOrderSources.ToDictionary(
    o => o.Id,
    o => new KitchenOrderDto
    {
        OrderId = o.Id,
        Comment = o.Comment ?? string.Empty,
        CreatedAt = o.CreatedAt,
        PickupAt = o.PickupAt,
        OrderType = o.OrderType ?? string.Empty,
        Status = o.Status ?? string.Empty
    });

var dishSources = await _db.OrderDishItems
    .AsNoTracking()
    .Where(i =>
        i.OrderId.HasValue &&
        orderIds.Contains(i.OrderId.Value) &&
        (includeCompletedItems || !i.IsCompleted))
    .OrderBy(i => i.Id)
    .Select(i => new DishSource
    {
        ItemId = i.Id,
        OrderId = i.OrderId!.Value,
        Name = i.Dish != null ? i.Dish.Name : null,
        Quantity = i.Quantity ?? 0m
    })
    .ToListAsync();

var dishItemIds = dishSources
    .Select(i => i.ItemId)
    .ToList();

var dishToppingSources = dishItemIds.Count ==
0
    ? new List<DishToppingSource>()
    : await _db.DishToppings
        .AsNoTracking()
        .Where(t => t.OrderDishItemId.HasValue
            && dishItemIds.Contains(t.OrderDishItemId.Value))
        .OrderBy(t => t.Id)
        .Select(t => new DishToppingSource
        {
            OrderDishItemId =
t.OrderDishItemId!.Value,
            Name = t.Topping != null ?
t.Topping.Name : null,
            Quantity = t.Quantity ?? 0m
        })
        .ToListAsync();

var dishToppingsById =
dishToppingSources
    .GroupBy(t => t.OrderDishItemId)
    .ToDictionary(
        g => g.Key,
        g => g.Select(t => new
KitchenOrderDishToppingDto
        {
            Name = t.Name ?? "Без названия",
            Quantity = t.Quantity

```

```

        }).ToListAsync();

        foreach (var dishSource in dishSources)
        {
            if
(!ordersById.TryGetValue(dishSource.OrderId, out var
order))
                continue;

            var dish = new KitchenOrderDishDto
            {
                ItemId = dishSource.ItemId,
                Name = dishSource.Name ?? "Без
названия",
                Quantity = dishSource.Quantity
            };

            if
(dishToppingsById.TryGetValue(dishSource.ItemId
, out var dishToppings))
            {
                dish.Toppings = dishToppings;
            }

            order.Dishes.Add(dish);
        }

        var drinkSources = await _db.OrderDrinkItems
.AsNoTracking()
.Where(i =>
    i.OrderId.HasValue &&
    orderIds.Contains(i.OrderId.Value) &&
    (includeCompletedItems || !i.IsCompleted))
.OrderBy(i => i.Id)
.Select(i => new DrinkSource
{
    ItemId = i.Id,
    OrderId = i.OrderId!.Value,
    Name = i.Drink != null ? i.Drink.Name :
null,
    Quantity = i.Quantity ?? 0m
})
.ToListAsync();

        var drinkItemIds = drinkSources
.Select(i => i.ItemId)
.ToList();

        var drinkToppingSources = drinkItemIds.Count
== 0
        ? new List<DrinkToppingSource>()
        : await _db.DrinkToppings
.AsNoTracking()
.Where(t => t.OrderDrinkItemId.HasValue
&& drinkItemIds.Contains(t.OrderDrinkItemId.Value))
.OrderBy(t => t.Id)
.Select(t => new DrinkToppingSource
{
    OrderDrinkItemId =
t.OrderDrinkItemId!.Value,
    Name = t.Topping != null ?
t.Topping.Name : null,
    Quantity = t.Quantity ?? 0m
})

```

```

        .ToListAsync();

        var drinkToppingsById =
drinkToppingSources
.GroupBy(t => t.OrderDrinkItemId)
.ToDictionary(
    g => g.Key,
    g => g.Select(t => new
KitchenOrderDrinkToppingDto
{
    Name = t.Name ?? "Без названия",
    Quantity = t.Quantity
}).ToListAsync());

        foreach (var drinkSource in drinkSources)
        {
            if
(!ordersById.TryGetValue(drinkSource.OrderId, out var
order))
                continue;

            var drink = new KitchenOrderDrinkDto
            {
                ItemId = drinkSource.ItemId,
                Name = drinkSource.Name ?? "Без
названия",
                Quantity = drinkSource.Quantity
            };

            if
(drinkToppingsById.TryGetValue(drinkSource.Item
Id, out var drinkToppings))
            {
                drink.Toppings = drinkToppings;
            }

            order.Drinks.Add(drink);
        }

        var standaloneToppings = await
_db.OrderToppingItems
.AsNoTracking()
.Where(i =>
    orderIds.Contains(i.OrderId) &&
    (includeCompletedItems || !i.IsCompleted))
.OrderBy(i => i.Id)
.Select(i => new
{
    i.Id,
    i.OrderId,
    Name = i.Topping != null ? i.Topping.Name
: null,
    Quantity = (decimal)i.Quantity,
    CategoryName = i.Topping != null &&
i.Topping.Category != null
? i.Topping.Category.Name
: null
})
.ToListAsync();

        foreach (var topping in standaloneToppings)
        {
            if (!ordersById.TryGetValue(topping.OrderId,
out var order))

```

```

        continue;

        order.Toppings.Add(new
KitchenOrderStandaloneToppingDto
        {
            ItemId = topping.Id,
            Name = topping.Name ?? "Без названия",
            Quantity = topping.Quantity,
            CategoryName = topping.CategoryName ??
string.Empty
        });
    }

    var resultOrders = orderedOrderSources
        .Select(o => ordersById[o.Id])
        .Where(o => o.Dishes.Count > 0 ||
o.Drinks.Count > 0 || o.Toppings.Count > 0)
        .ToList();

    return Ok(new KitchenOrdersResponse
    {
        Orders = resultOrders
    });
}

[HttpGet("stop-list/positions")]
public async
Task<ActionResult<KitchenStopListPositionsResponse>
> GetStopListPositions()
{
    await
_stockService.RefreshMenuAvailabilityAsync();

    var limitsByKey = await
_db.MenuItemPortionLimits
        .AsNoTracking()
        .ToDictionaryAsync(
            x => BuildMenuItemLimitKey(x.ItemType,
x.ItemId),
            x => RoundTo2(Math.Max(0m,
x.RemainingPortions)));

    var dishSources = await _db.Dishes
        .AsNoTracking()
        .OrderBy(d => d.Name)
        .ThenBy(d => d.Id)
        .Select(d => new
        {
            d.Id,
            d.Name,
            CategoryName = d.Category != null ?
d.Category.Name : null,
            d.IsAvailable,
            d.TechnicalCardId,
            UnitName = d.UnitOfMeasure != null ?
d.UnitOfMeasure.Name : null
        })
        .ToListAsync();

    var drinkSources = await _db.Drinks
        .AsNoTracking()
        .OrderBy(d => d.Name)
        .ThenBy(d => d.Id)
        .Select(d => new

```

```

        {
            d.Id,
            d.Name,
            CategoryName = d.Category != null ?
d.Category.Name : null,
            d.IsAvailable,
            d.TechnicalCardId,
            d.Quantity,
            UnitName = d.UnitOfMeasure != null ?
d.UnitOfMeasure.Name : null
        })
        .ToListAsync();

    var toppingSources = await
_db.ToppingsAndSyrups
        .AsNoTracking()
        .OrderBy(t => t.Name)
        .ThenBy(t => t.Id)
        .Select(t => new
        {
            t.Id,
            t.Name,
            CategoryName = t.Category != null ?
t.Category.Name : null,
            t.IsAvailable,
            t.TechnicalCardId,
            t.Quantity,
            UnitName = t.UnitOfMeasure != null ?
t.UnitOfMeasure.Name : null
        })
        .ToListAsync();

    var technicalCardIds = dishSources
        .Where(x => x.TechnicalCardId.HasValue)
        .Select(x => x.TechnicalCardId!.Value)
        .Concat(drinkSources
            .Where(x => x.TechnicalCardId.HasValue)
            .Select(x => x.TechnicalCardId!.Value))
        .Concat(toppingSources
            .Where(x => x.TechnicalCardId.HasValue)
            .Select(x => x.TechnicalCardId!.Value))
        .Distinct()
        .ToList();

    var autoPortionsByTechnicalCard = await
BuildAutoAvailablePortionsByTechnicalCardAsync(tec
hnicalCardIds);

    var outputByTechnicalCard = await
BuildOutputWeightByTechnicalCardAsync(technicalCar
dIds);

    var positions = new
List<KitchenStopListPositionDto>(dishSources.Count +
drinkSources.Count + toppingSources.Count);

    positions.AddRange(dishSources.Select(source
=> BuildStopListPosition(
        KitchenItemTypes.Dish,
        source.Id,
        source.Name,
        source.CategoryName,

BuildTechnicalCardVolumeWeight(source.TechnicalCar
dId, outputByTechnicalCard, source.UnitName),

```

```

        source.IsAvailable,
        source.TechnicalCardId,
        limitsByKey,
        autoPortionsByTechnicalCard));

    positions.AddRange(drinkSources.Select(source
=> BuildStopListPosition(
        KitchenItemTypes.Drink,
        source.Id,
        source.Name,
        source.CategoryName,
        BuildVolumeWeight(source.Quantity,
source.UnitName),
        source.IsAvailable,
        source.TechnicalCardId,
        limitsByKey,
        autoPortionsByTechnicalCard)));

    positions.AddRange(toppingSources.Select(source =>
BuildStopListPosition(
        KitchenItemTypes.Topping,
        source.Id,
        source.Name,
        source.CategoryName,
        BuildVolumeWeight(source.Quantity,
source.UnitName),
        source.IsAvailable,
        source.TechnicalCardId,
        limitsByKey,
        autoPortionsByTechnicalCard)));

    positions = positions

.Where(IsStopListPositionVisibleForCurrentRole)
    .OrderBy(x => x.Name)
    .ThenBy(x => x.ItemType)
    .ThenBy(x => x.ItemId)
    .ToList();

    return Ok(new
KitchenStopListPositionsResponse
    {
        Positions = positions
    });
}

[HttpPost("stop-list/items")]
public async Task<IActionResult>
AddStopListItem([FromBody]
KitchenStopListItemRequest request)
{
    if (request == null || request.ItemId <= 0)
    {
        return BadRequest("Некорректные данные
позиции");
    }

    if (!TryParseItemType(request.ItemType, out var
itemType))
    {
        return BadRequest("Неизвестный тип
позиции");
    }
}

```

```

        if (request.RemainingPortions < 0m)
        {
            return BadRequest("Остаток порций не
может быть отрицательным.");
        }

        if (!await
CanCurrentRoleManageStopListItemAsync(itemType,
request.ItemId))
        {
            return Forbid();
        }

        var roundedRemainingPortions =
RoundTo2(request.RemainingPortions);
        var now = DateTime.Now;
        var itemTypeToken =
ToItemTypeToken(itemType);

        await using var tx = await
_db.Database.BeginTransactionAsync();

        var itemExists = await
SetMenuItemAvailabilityAsync(
            itemType,
            request.ItemId,
            roundedRemainingPortions > 0m,
            saveChanges: false);
        if (!itemExists)
        {
            await tx.RollbackAsync();
            return NotFound("Позиция не найдена");
        }

        await
UpsertMenuItemPortionLimitAsync(itemTypeToken,
request.ItemId, roundedRemainingPortions, now);
        await _db.SaveChangesAsync();

        if (roundedRemainingPortions > 0m)
        {
            await
_stockService.RefreshMenuAvailabilityAsync();
        }

        await tx.CommitAsync();
        return NoContent();
    }

    [HttpDelete("stop-
list/items/{itemType}/{itemId:int}")]
    public async Task<IActionResult>
    RemoveStopListItem(string itemType, int itemId)
    {
        if (itemId <= 0)
        {
            return BadRequest("Некорректный
идентификатор позиции");
        }

        if (!TryParseItemType(itemType, out var
parsedItemType))
        {

```

```

        return BadRequest("Неизвестный тип
позиции");
    }

    if (!await
CanCurrentRoleManageStopListItemAsync(parsedItemT
ype, itemId))
    {
        return Forbid();
    }

    await using var tx = await
_db.Database.BeginTransactionAsync();

    var itemTypeToken =
ToItemTypeToken(parsedItemType);
    var menuItemLimit = await
_db.MenuItemPortionLimits
.FirstOrDefaultAsync(x =>
    x.ItemType == itemTypeToken &&
    x.ItemId == itemId);
    if (menuItemLimit != null)
    {

_db.MenuItemPortionLimits.Remove(menuItemLimit);
    }

    var itemExists = await
SetMenuItemAvailabilityAsync(parsedItemType, itemId,
true, saveChanges: false);
    if (!itemExists)
    {
        await tx.RollbackAsync();
        return NotFound("Позиция не найдена");
    }

    await _db.SaveChangesAsync();
    await
_stockService.RefreshMenuAvailabilityAsync();
    await tx.CommitAsync();
    return NoContent();
}

[HttpPost("orders/{orderId:int}/items/{itemType}/{itemI
d:int}/complete")]
public async
Task<ActionResult<KitchenCompleteOrderItemRespons
e>> CompleteOrderItem(
    int orderId,
    string itemType,
    int itemId)
{
    if (!TryParseItemType(itemType, out var
parsedItemType))
    {
        return BadRequest("Неизвестный тип
позиции заказа");
    }

    var orderExists = await _db.Orders.AnyAsync(o
=> o.Id == orderId);
    if (!orderExists)
    {

```

```

        return NotFound("Заказ не найден");
    }

    var itemCompleted = await
MarkItemCompletedAsync(orderId, parsedItemType,
itemId);
    if (!itemCompleted)
    {
        return NotFound("Позиция заказа не
найдена");
    }

    try
    {
        var orderCompleted = await
TryMarkOrderReadyIfCompletedAsync(orderId);
        var orderStatusName = await
GetOrderStatusNameAsync(orderId);

        return Ok(new
KitchenCompleteOrderItemResponse
        {
            OrderId = orderId,
            ItemType =
ToItemTypeToken(parsedItemType),
            ItemId = itemId,
            OrderCompleted = orderCompleted,
            OrderStatus = orderStatusName
        });
    }
    catch (InvalidOperationException ex)
    {
        return StatusCode(500, ex.Message);
    }
}

[HttpPost("orders/{orderId:int}/complete")]
public async
Task<ActionResult<KitchenCompleteOrderResponse>>
CompleteOrder(int orderId)
{
    var order = await
_db.Orders.FirstOrDefaultAsync(o => o.Id == orderId);
    if (order == null)
    {
        return NotFound("Заказ не найден");
    }

    await
MarkAllOrderItemsCompletedAsync(orderId);

    var readyStatus = await
ResolveReadyStatusAsync();
    if (readyStatus == null)
    {
        return StatusCode(500, "Статус готовности
заказа не найден");
    }

    order.StatusId = readyStatus.Value.Id;
    await _db.SaveChangesAsync();

    return Ok(new KitchenCompleteOrderResponse
    {

```

```

        OrderId = orderId,
        OrderStatus = readyStatus.Value.Name
    });
}

[HttpPost("orders/{orderId:int}/cancel")]
[Authorize(Roles = "Администратор")]
public async
Task<ActionResult<KitchenCompleteOrderResponse>>
CancelOrder(int orderId)
{
    var cancelledStatus = await
    ResolveCancelledStatusAsync();
    if (cancelledStatus == null)
    {
        return StatusCode(500, "Статус отмены
заказа не найден");
    }

    var order = await
    _db.Orders.FirstOrDefaultAsync(o => o.Id == orderId);
    if (order == null)
    {
        return NotFound("Заказ не найден");
    }

    order.StatusId = cancelledStatus.Value.Id;
    await _db.SaveChangesAsync();

    return Ok(new KitchenCompleteOrderResponse
    {
        OrderId = order.Id,
        OrderStatus = cancelledStatus.Value.Name
    });
}

[HttpGet("orders/{orderId:int}/items/{itemType}/{itemId:int}/technical-card")]
public async
Task<ActionResult<KitchenTechnicalCardResponse>>
GetTechnicalCard(
    int orderId,
    string itemType,
    int itemId)
{
    if (!TryParseItemType(itemType, out var
    parsedItemType))
    {
        return BadRequest("Неизвестный тип
    позиции заказа");
    }

    var source = await
    LoadTechnicalCardSourceAsync(orderId,
    parsedItemType, itemId);
    if (source == null)
    {
        return NotFound("Позиция заказа не
    найдена");
    }

    if (!source.TechnicalCardId.HasValue)
    {

```

```

        return NotFound("Тех. карта для позиции не
    найдена");
    }

    var technicalCardId =
    source.TechnicalCardId.Value;
    var response = await
    BuildTechnicalCardResponseAsync(technicalCardId,
    source.ItemName);
    if (response == null)
    {
        return NotFound("Тех. карта не найдена");
    }

    return Ok(response);
}

[HttpGet("technical-cards")]
public async
Task<ActionResult<KitchenTechnicalCardsResponse>>
GetTechnicalCards()
{
    var technicalCards = await _db.TechnicalCards
    .AsNoTracking()
    .OrderBy(c => c.Name)
    .ThenBy(c => c.Id)
    .Select(c => new
    KitchenTechnicalCardListItemDto
    {
        TechnicalCardId = c.Id,
        CardName =
        string.IsNullOrEmpty(c.Name)
        ? $"Technical card #{c.Id}"
        : c.Name,
        Description = c.Description ?? string.Empty
    })
    .ToListAsync();

    return Ok(new KitchenTechnicalCardsResponse
    {
        TechnicalCards = technicalCards
    });
}

[HttpGet("technical-cards/{technicalCardId:int}")]
public async
Task<ActionResult<KitchenTechnicalCardResponse>>
GetTechnicalCardById(int technicalCardId)
{
    var response = await
    BuildTechnicalCardResponseAsync(technicalCardId,
    null);
    if (response == null)
    {
        return NotFound("Technical card not found");
    }

    return Ok(response);
}

[HttpGet("technical-cards/edit-options")]
[Authorize(Roles = "Администратор")]

```



```

        public async
        Task<ActionResult<KitchenTechnicalCardEditOptionsR
        esponse>> GetTechnicalCardEditOptions()
        {
            var ingredients = await _db.Ingredients
                .AsNoTracking()
                .OrderBy(i => i.Name)
                .ThenBy(i => i.Id)
                .Select(i => new
            KitchenTechnicalCardReferenceDto
            {
                Id = i.Id,
                Name = string.IsNullOrEmpty(i.Name)
? $"Ingredient #{i.Id}" : i.Name,
                UnitOfMeasureId = i.UnitOfMeasureId,
                UnitName = i.UnitOfMeasure != null ?
(i.UnitOfMeasure.Name ?? string.Empty) : string.Empty
            })
                .ToListAsync();

            var semiFinisheds = await _db.SemiFinisheds
                .AsNoTracking()
                .OrderBy(s => s.Name)
                .ThenBy(s => s.Id)
                .Select(s => new
            KitchenTechnicalCardReferenceDto
            {
                Id = s.Id,
                Name =
string.IsNullOrEmpty(s.Name) ? $"SemiFinished
#{s.Id}" : s.Name,
                UnitOfMeasureId = s.UnitOfMeasureId,
                UnitName = s.UnitOfMeasure != null ?
(s.UnitOfMeasure.Name ?? string.Empty) : string.Empty
            })
                .ToListAsync();

            var units = await _db.UnitsOfMeasures
                .AsNoTracking()
                .OrderBy(u => u.Name)
                .ThenBy(u => u.Id)
                .Select(u => new
            KitchenTechnicalCardUnitDto
            {
                Id = u.Id,
                Name =
string.IsNullOrEmpty(u.Name) ? $"Unit #{u.Id}" :
u.Name
            })
                .ToListAsync();

            var bindingOptions = new
            List<KitchenTechnicalCardBindingOptionDto>();
            bindingOptions.AddRange(await _db.Dishes
                .AsNoTracking()
                .OrderBy(x => x.Name)
                .ThenBy(x => x.Id)
                .Select(x => new
            KitchenTechnicalCardBindingOptionDto
            {
                Kind =
            KitchenTechnicalCardBindingKinds.Dish,
                ItemId = x.Id,

```

```

                Name =
string.IsNullOrEmpty(x.Name) ? $"Dish #{x.Id}" :
x.Name,
                TechnicalCardId = x.TechnicalCardId
            })
                .ToListAsync());
            bindingOptions.AddRange(await _db.Drinks
                .AsNoTracking()
                .OrderBy(x => x.Name)
                .ThenBy(x => x.Id)
                .Select(x => new
            KitchenTechnicalCardBindingOptionDto
            {
                Kind =
            KitchenTechnicalCardBindingKinds.Drink,
                ItemId = x.Id,
                Name =
string.IsNullOrEmpty(x.Name) ? $"Drink #{x.Id}"
: x.Name,
                TechnicalCardId = x.TechnicalCardId
            })
                .ToListAsync());
            bindingOptions.AddRange(await
            _db.ToppingsAndSyrups
                .AsNoTracking()
                .OrderBy(x => x.Name)
                .ThenBy(x => x.Id)
                .Select(x => new
            KitchenTechnicalCardBindingOptionDto
            {
                Kind =
            KitchenTechnicalCardBindingKinds.Topping,
                ItemId = x.Id,
                Name =
string.IsNullOrEmpty(x.Name) ? $"Topping
#{x.Id}" : x.Name,
                TechnicalCardId = x.TechnicalCardId
            })
                .ToListAsync());
            bindingOptions.AddRange(semiFinisheds.Select(x =>
            new KitchenTechnicalCardBindingOptionDto
            {
                Kind =
            KitchenTechnicalCardBindingKinds.SemiFinished,
                ItemId = x.Id,
                Name = x.Name,
                TechnicalCardId = null
            }
            ));

            var semiFinishedCardIds = await
            _db.SemiFinisheds
                .AsNoTracking()
                .Select(x => new { x.Id, x.TechnicalCardId })
                .ToDictionaryAsync(x => x.Id, x =>
            x.TechnicalCardId);

            foreach (var option in bindingOptions.Where(x
            => x.Kind ==
            KitchenTechnicalCardBindingKinds.SemiFinished))
            {
                option.TechnicalCardId =
                semiFinishedCardIds.TryGetValue(option.ItemId, out
                var technicalCardId)

```

```

        ? technicalCardId
        : null;
    }

    return Ok(new
KitchenTechnicalCardEditOptionsResponse
    {
        Ingredients = ingredients,
        SemiFinisheds = semiFinisheds,
        Units = units,
        BindingOptions = bindingOptions
    });
}

[HttpGet("technical-
cards/{technicalCardId:int}/edit")]
[Authorize(Roles = "Администратор")]
public async
Task<ActionResult<KitchenTechnicalCardEditResponse
>> GetTechnicalCardForEdit(int technicalCardId)
{
    var response = await
BuildTechnicalCardEditResponseAsync(technicalCardId
);
    if (response == null)
    {
        return NotFound("Technical card not found");
    }

    return Ok(response);
}

[HttpPost("technical-cards")]
[Authorize(Roles = "Администратор")]
public async
Task<ActionResult<KitchenTechnicalCardResponse>>
CreateTechnicalCard(KitchenTechnicalCardUpsertRequ
est request)
{
    var validationError =
ValidateTechnicalCardRequest(request);
    if (validationError != null)
    {
        return BadRequest(validationError);
    }

    var referenceError = await
ValidateTechnicalCardReferencesAsync(request);
    if (referenceError != null)
    {
        return BadRequest(referenceError);
    }

    await using var transaction = await
_db.Database.BeginTransactionAsync();

    var technicalCard = new TechnicalCard
    {
        Name = request.CardName.Trim(),
        Description =
NormalizeDescription(request.Description)
    };
    _db.TechnicalCards.Add(technicalCard);

```

```

        await _db.SaveChangesAsync();

        await
ReplaceTechnicalCardDetailsAsync(technicalCard.Id,
request);
        await transaction.CommitAsync();

        var response = await
BuildTechnicalCardResponseAsync(technicalCard.Id,
null);
        return
CreatedAtAction(nameof(GetTechnicalCardById), new
{ technicalCardId = technicalCard.Id }, response);
    }

    [HttpPut("technical-cards/{technicalCardId:int}")]
    [Authorize(Roles = "Администратор")]
    public async
    Task<ActionResult<KitchenTechnicalCardResponse>>
    UpdateTechnicalCard(
        int technicalCardId,
        KitchenTechnicalCardUpsertRequest request)
    {
        var validationError =
ValidateTechnicalCardRequest(request);
        if (validationError != null)
        {
            return BadRequest(validationError);
        }

        var referenceError = await
ValidateTechnicalCardReferencesAsync(request);
        if (referenceError != null)
        {
            return BadRequest(referenceError);
        }

        var technicalCard = await
_db.TechnicalCards.FirstOrDefaultAsync(c => c.Id ==
technicalCardId);
        if (technicalCard == null)
        {
            return NotFound("Technical card not found");
        }

        await using var transaction = await
_db.Database.BeginTransactionAsync();

        technicalCard.Name = request.CardName.Trim();
        technicalCard.Description =
NormalizeDescription(request.Description);
        await
ReplaceTechnicalCardDetailsAsync(technicalCard.Id,
request);
        await transaction.CommitAsync();

        var response = await
BuildTechnicalCardResponseAsync(technicalCard.Id,
null);
        return Ok(response);
    }

    [HttpDelete("technical-
cards/{technicalCardId:int}")]

```

```

[Authorize(Roles = "Администратор")]
public async Task<IActionResult>
DeleteTechnicalCard(int technicalCardId)
{
    var technicalCard = await
_db.TechnicalCards.FirstOrDefaultAsync(c => c.Id ==
technicalCardId);
    if (technicalCard == null)
    {
        return NotFound("Technical card not found");
    }

    var linkedNames = await
LoadTechnicalCardLinkedNamesAsync(technicalCardId
);
    if (linkedNames.Count > 0)
    {
        return BadRequest("Нельзя удалить
техкарту: она привязана к позициям: " + string.Join(",
", linkedNames));
    }

    await using var transaction = await
_db.Database.BeginTransactionAsync();

    var ingredientRows = await
_db.TechnicalCardIngredientCompositions
.Where(x => x.TechnicalCardId ==
technicalCardId)
.ToListAsync();
    var semiFinishedRows = await
_db.TechnicalCardSemiFinishedCompositions
.Where(x => x.TechnicalCardId ==
technicalCardId)
.ToListAsync();

_db.TechnicalCardIngredientCompositions.RemoveRang
e(ingredientRows);

_db.TechnicalCardSemiFinishedCompositions.RemoveR
ange(semiFinishedRows);
_db.TechnicalCards.Remove(technicalCard);

    await _db.SaveChangesAsync();
    await transaction.CommitAsync();

    return NoContent();
}

[HttpGet("preparations")]
public async
Task<ActionResult<KitchenPreparationsBoardResponse
>> GetPreparationsBoard()
{
    await CleanupNonPositivePreparationsAsync();
    await EnsureSystemPreparationTasksAsync();
    var visibleSemiFinishedIds = await
ResolveVisiblePreparationSemiFinishedIdsAsync();

    var tasks = await _db.PreparationTasks
.AsNoTracking()
.OrderByDescending(t => t.CreatedAt)
.ThenByDescending(t => t.Id)

```

```

.Select(t => new KitchenPreparationTaskDto
{
    TaskId = t.Id,
    SemiFinishedId = t.SemiFinishedId,
    TaskText = t.TaskText,
    IsLinkedToSemiFinished =
t.SemiFinishedId.HasValue,
    TechnicalCardId = t.SemiFinished != null ?
t.SemiFinished.TechnicalCardId : null,
    SemiFinishedName =
t.SemiFinishedId.HasValue
? (t.SemiFinished != null &&
!string.IsNullOrEmpty(t.SemiFinished.Name)
? t.SemiFinished.Name
: $"SemiFinished
#{t.SemiFinishedId.Value}")
: string.Empty,
    Comment = t.Comment ?? string.Empty,
    CreatedAt = t.CreatedAt
})
.ToListAsync();

tasks = tasks
.Where(t =>
!IsSnoozedSystemRecommendationComment(t.Commen
t))
.ToList();

if (visibleSemiFinishedIds != null)
{
    tasks = tasks
.Where(t => !t.SemiFinishedId.HasValue ||
visibleSemiFinishedIds.Contains(t.SemiFinishedId.Value
))
.ToList();
}

var existingPreparations = await _db.Preparations
.AsNoTracking()
.Where(p =>
p.SemiFinishedId.HasValue &&
p.StockGrams.HasValue &&
p.StockGrams.Value > 0m)
.OrderByDescending(p => p.ProductionDate)
.ThenByDescending(p => p.Id)
.Select(p => new
KitchenPreparationListItemDto
{
    PreparationId = p.Id,
    SemiFinishedId = p.SemiFinishedId!.Value,
    TechnicalCardId = p.SemiFinished != null ?
p.SemiFinished.TechnicalCardId : null,
    PreparationName = p.SemiFinished != null
&& !string.IsNullOrEmpty(p.SemiFinished.Name)
? p.SemiFinished.Name
: (string.IsNullOrEmpty(p.Name) ?
$"Preparation #{p.Id}" : p.Name),
    StockGrams = p.StockGrams ?? 0m,
    ProductionDate =
p.ProductionDate.HasValue
?
p.ProductionDate.Value.ToDateTime(TimeOnly.MinVal
ue)
: null

```

```

    })
    .ToListAsync();

    if (visibleSemiFinishedIds != null)
    {
        existingPreparations = existingPreparations
            .Where(p =>
                visibleSemiFinishedIds.Contains(p.SemiFinishedId))
            .ToList();
    }

    var semiFinishedOptions = await
        _db.SemiFinisheds
            .AsNoTracking()
            .OrderBy(sf => sf.Name)
            .ThenBy(sf => sf.Id)
            .Select(sf => new
                KitchenSemiFinishedOptionDto
                {
                    SemiFinishedId = sf.Id,
                    TechnicalCardId = sf.TechnicalCardId,
                    Name =
                        string.IsNullOrEmpty(sf.Name)
                            ? $"SemiFinished #{sf.Id}"
                            : sf.Name
                })
            .ToListAsync();

    if (visibleSemiFinishedIds != null)
    {
        semiFinishedOptions = semiFinishedOptions
            .Where(sf =>
                visibleSemiFinishedIds.Contains(sf.SemiFinishedId))
            .ToList();
    }

    return Ok(new
        KitchenPreparationsBoardResponse
        {
            Tasks = tasks,
            ExistingPreparations = existingPreparations,
            SemiFinishedOptions = semiFinishedOptions
        });
}

private async Task
CleanupNonPositivePreparationsAsync()
{
    var invalidPreparations = await _db.Preparations
        .Where(p => !p.StockGrams.HasValue ||
            p.StockGrams.Value <= 0m)
        .ToListAsync();

    if (invalidPreparations.Count == 0)
    {
        return;
    }

    _db.Preparations.RemoveRange(invalidPreparations);
    await _db.SaveChangesAsync();
    await
        _stockService.RefreshMenuAvailabilityAsync();
}

```

```

private async Task<HashSet<int>?>
ResolveVisiblePreparationSemiFinishedIdsAsync()
{
    if (IsCurrentUserInRole(AdminRole))
    {
        return null;
    }

    if (IsCurrentUserInRole(CookRole))
    {
        return await
            LoadSemiFinishedIdsUsedByDishesAsync();
    }

    if (IsCurrentUserInRole(BaristaRole))
    {
        return await
            LoadSemiFinishedIdsUsedByDrinksAsync();
    }

    return new HashSet<int>();
}

private async Task<HashSet<int>>
LoadSemiFinishedIdsUsedByDishesAsync()
{
    return (await
        _db.TechnicalCardSemiFinishedCompositions
            .AsNoTracking()
            .Where(c =>
                c.SemiFinishedId.HasValue &&
                c.TechnicalCardId.HasValue &&
                c.TechnicalCard != null &&
                c.TechnicalCard.Dishes.Any())
            .Select(c => c.SemiFinishedId!.Value)
            .Distinct()
            .ToListAsync())
        .ToHashSet();
}

private async Task<HashSet<int>>
LoadSemiFinishedIdsUsedByDrinksAsync()
{
    return (await
        _db.TechnicalCardSemiFinishedCompositions
            .AsNoTracking()
            .Where(c =>
                c.SemiFinishedId.HasValue &&
                c.TechnicalCardId.HasValue &&
                c.TechnicalCard != null &&
                c.TechnicalCard.Drinks.Any())
            .Select(c => c.SemiFinishedId!.Value)
            .Distinct()
            .ToListAsync())
        .ToHashSet();
}

private async Task
EnsureSystemPreparationTasksAsync()
{
    var today =
        DateOnly.FromDateTime(DateTime.Today);
}

```

```

        var criticalBoundaryDate = today.AddDays(-
CriticalDays);

        var latestProductionDates = await
_db.Preparations
        .AsNoTracking()
        .Where(p =>
            p.SemiFinishedId.HasValue &&
            p.ProductionDate.HasValue &&
            p.StockGrams.HasValue &&
            p.StockGrams.Value > 0m)
        .GroupBy(p => p.SemiFinishedId!.Value)
        .Select(g => new
        {
            SemiFinishedId = g.Key,
            LatestProductionDate = g.Max(p =>
p.ProductionDate)
        })
        .ToListAsync();

        var staleSemiFinishedIds = latestProductionDates
        .Where(x =>
x.LatestProductionDate.HasValue &&
x.LatestProductionDate.Value < criticalBoundaryDate)
        .Select(x => x.SemiFinishedId)
        .ToHashSet();

        var systemTasks = await _db.PreparationTasks
        .Where(t =>
            t.SemiFinishedId.HasValue &&
            (t.Comment ==
SystemRecommendationComment ||
            (t.Comment != null &&
t.Comment.StartsWith(SystemRecommendationSnoozeP
refix))))
        .OrderByDescending(t => t.CreatedAt)
        .ThenByDescending(t => t.Id)
        .ToListAsync();

        var now = DateTime.Now;
        var hasChanges = false;
        var activeSystemTaskSemiFinishedIds = new
HashSet<int>();
        var snoozedSystemTaskSemiFinishedIds = new
HashSet<int>();

        foreach (var group in systemTasks
        .Where(t => t.SemiFinishedId.HasValue)
        .GroupBy(t => t.SemiFinishedId!.Value))
        {
            var semiFinishedId = group.Key;
            var orderedTasks = group
                .OrderByDescending(t => t.CreatedAt)
                .ThenByDescending(t => t.Id)
                .ToList();

            if
(!staleSemiFinishedIds.Contains(semiFinishedId))
            {
                _db.PreparationTasks.RemoveRange(orderedTasks);
                hasChanges = true;
                continue;
            }

```

```

        var latestTask = orderedTasks[0];
        if
        (TryParseSnoozedSystemRecommendationUntil(latestTa
sk.Comment, out var snoozedUntilDate))
        {
            if (snoozedUntilDate > today)
            {
                snoozedSystemTaskSemiFinishedIds.Add(semiFinishedI
d);
            }
            else
            {
                latestTask.Comment =
SystemRecommendationComment;
                latestTask.CreatedAt = now;
                hasChanges = true;

                activeSystemTaskSemiFinishedIds.Add(semiFinishedId)
                ;
            }
        }
        else
        {
            activeSystemTaskSemiFinishedIds.Add(semiFinishedId)
            ;
        }

        if (orderedTasks.Count > 1)
        {
            _db.PreparationTasks.RemoveRange(orderedTasks.Skip(
1));
            hasChanges = true;
        }

        var missingSemiFinishedIds =
staleSemiFinishedIds
        .Except(activeSystemTaskSemiFinishedIds)
        .Except(snoozedSystemTaskSemiFinishedIds)
        .ToList();

        if (missingSemiFinishedIds.Count > 0)
        {
            var semiFinishedNamesById = await
_db.SemiFinisheds
                .AsNoTracking()
                .Where(sf =>
                    missingSemiFinishedIds.Contains(sf.Id))
                .Select(sf => new
                {
                    sf.Id,
                    sf.Name
                })
                .ToDictionaryAsync(
                    sf => sf.Id,
                    sf =>
string.IsNullOrEmpty(sf.Name)
                        ? $"SemiFinished #{sf.Id}"
                        : sf.Name!);

```

```

        foreach (var semiFinishedId in
missingSemiFinishedIds)
        {
            var taskText =
semiFinishedNamesById.TryGetValue(semiFinishedId,
out var semiFinishedName)
                ? semiFinishedName
                : $"SemiFinished #{semiFinishedId}";

            _db.PreparationTasks.Add(new
PreparationTask
            {
                SemiFinishedId = semiFinishedId,
                TaskText = taskText,
                Comment =
SystemRecommendationComment,
                CreatedAt = now
            });
        }

        hasChanges = true;
    }

    if (!hasChanges)
    {
        return;
    }

    await _db.SaveChangesAsync();
}

[HttpPost("preparations/tasks")]
public async
Task<ActionResult<KitchenPreparationTaskDto>>
CreatePreparationTask(
    [FromBody]
KitchenCreatePreparationTaskRequest request)
{
    if (IsCurrentUserInRole(AdminRole))
    {
        return Forbid();
    }

    if (request == null)
    {
        return BadRequest("Request body is
required.");
    }

    var normalizedTaskText =
NormalizeTaskText(request.TaskText);
    if (normalizedTaskText == null)
    {
        return BadRequest("TaskText is required.");
    }

    if (normalizedTaskText.Length > 255)
    {
        return BadRequest("TaskText is too long. Max
length is 255 characters.");
    }

    if (request.SemiFinishedId.HasValue &&
request.SemiFinishedId.Value <= 0)

```

```

    {
        return BadRequest("SemiFinishedId must be
greater than zero.");
    }

    var normalizedComment =
NormalizeComment(request.Comment);
    if (normalizedComment != null &&
normalizedComment.Length > 500)
    {
        return BadRequest("Comment is too long. Max
length is 500 characters.");
    }

    int? semiFinishedId = null;
    int? technicalCardId = null;
    string semiFinishedName = string.Empty;
    var visibleSemiFinishedIds = await
ResolveVisiblePreparationSemiFinishedIdsAsync();

    if (request.SemiFinishedId.HasValue)
    {
        if (visibleSemiFinishedIds != null &&
!visibleSemiFinishedIds.Contains(request.SemiFinishedI
d.Value))
        {
            return Forbid();
        }

        var semiFinished = await _db.SemiFinisheds
.AsNoTracking()
.Where(sf => sf.Id ==
request.SemiFinishedId.Value)
.Select(sf => new
        {
            sf.Id,
            sf.Name,
            sf.TechnicalCardId
        })
.FirstOrDefaultAsync();

        if (semiFinished == null)
        {
            return NotFound("Semi-finished item not
found.");
        }

        semiFinishedId = semiFinished.Id;
        technicalCardId =
semiFinished.TechnicalCardId;
        semiFinishedName =
string.IsNullOrEmpty(semiFinished.Name)
? $"SemiFinished #{semiFinished.Id}"
: semiFinished.Name;
    }

    var task = new PreparationTask
    {
        SemiFinishedId = semiFinishedId,
        TaskText = normalizedTaskText,
        Comment = normalizedComment,
        CreatedAt = DateTime.Now
    };

```

```

        _db.PreparationTasks.Add(task);
        await _db.SaveChangesAsync();

        return Ok(new KitchenPreparationTaskDto
        {
            TaskId = task.Id,
            SemiFinishedId = task.SemiFinishedId,
            TaskText = task.TaskText,
            IsLinkedToSemiFinished =
task.SemiFinishedId.HasValue,
            TechnicalCardId = technicalCardId,
            SemiFinishedName = semiFinishedName,
            Comment = task.Comment ?? string.Empty,
            CreatedAt = task.CreatedAt
        });
    }

    [HttpDelete("preparations/tasks/{taskId:int}")]
    public async Task<IActionResult>
DeletePreparationTask(int taskId)
    {
        if (IsCurrentUserInRole(AdminRole))
        {
            return Forbid();
        }

        var task = await
_db.PreparationTasks.FirstOrDefaultAsync(t => t.Id ==
taskId);
        if (task == null)
        {
            return NotFound("Preparation task not
found.");
        }

        if (task.SemiFinishedId.HasValue &&
IsSystemRecommendationComment(task.Comment))
        {
            var snoozedUntilDate =
DateOnly.FromDateTime(DateTime.Today).AddDays(1)
;
            task.Comment =
BuildSnoozedSystemRecommendationComment(snooze
dUntilDate);
            task.CreatedAt = DateTime.Now;
        }
        else
        {
            _db.PreparationTasks.Remove(task);
        }

        await _db.SaveChangesAsync();

        return NoContent();
    }

    [HttpDelete("preparations/{preparationId:int}")]
    public async Task<IActionResult>
DeletePreparation(int preparationId)
    {
        var preparation = await
_db.Preparations.FirstOrDefaultAsync(p => p.Id ==
preparationId);

```

```

        if (preparation == null)
        {
            return NotFound("Preparation not found.");
        }

        _db.Preparations.Remove(preparation);
        await _db.SaveChangesAsync();
        await
_stockService.RefreshMenuAvailabilityAsync();

        return NoContent();
    }

    [HttpPost("preparations/tasks/{taskId:int}/complete")]
    public async
Task<ActionResult<KitchenPreparationListItemDto>>
CompletePreparationTask(
        int taskId,
        [FromBody]
KitchenCompletePreparationTaskRequest request)
    {
        if (IsCurrentUserInRole(AdminRole))
        {
            return Forbid();
        }

        var task = await _db.PreparationTasks
.Include(t => t.SemiFinished)
.FirstOrDefaultAsync(t => t.Id == taskId);

        if (task == null)
        {
            return NotFound("Preparation task not
found.");
        }

        if (!task.SemiFinishedId.HasValue)
        {
            _db.PreparationTasks.Remove(task);
            await _db.SaveChangesAsync();
            return NoContent();
        }

        if (request == null)
        {
            return BadRequest("Request body is
required.");
        }

        if (request.StockGrams <= 0m)
        {
            return BadRequest("StockGrams must be
greater than zero.");
        }

        var productionDate = (request.ProductionDate ??
DateTime.Today).Date;
        var roundedStock =
decimal.Round(request.StockGrams, 2,
MidpointRounding.AwayFromZero);
        var preparationName =
!string.IsNullOrEmpty(task.SemiFinished?.Name)
? task.SemiFinished.Name

```

```

        : task.TaskText;

var preparation = new Preparation
{
    Name = preparationName,
    SemiFinishedId = task.SemiFinishedId.Value,
    StockGrams = roundedStock,
    ProductionDate =
DateOnly.FromDateTime(productionDate)
};

_db.Preparations.Add(preparation);
_db.PreparationTasks.Remove(task);

await _db.SaveChangesAsync();
await
_stockService.RefreshMenuAvailabilityAsync();

return Ok(new KitchenPreparationListItemDto
{
    PreparationId = preparation.Id,
    SemiFinishedId = task.SemiFinishedId.Value,
    TechnicalCardId =
task.SemiFinished?.TechnicalCardId,
    PreparationName = preparationName,
    StockGrams = roundedStock,
    ProductionDate = productionDate
});
}

[HttpGet("write-off/board")]
public async
Task<ActionResult<KitchenWriteOffBoardResponse>>
GetWriteOffBoard()
{
    var writeOffTypes = await _db.WriteOffTypes
        .AsNoTracking()
        .OrderBy(x => x.Id)
        .Select(x => new KitchenWriteOffTypeDto
        {
            WriteOffTypeId = x.Id,
            Name = x.Name
        })
        .ToListAsync();

    var gramsUnitName = await
_db.UnitsOfMeasures
        .AsNoTracking()
        .Where(x => x.Id == UnitGramsId)
        .Select(x => x.Name)
        .FirstOrDefaultAsync() ?? "г";

    var semiFinishedStockById = await
_db.Preparations
        .AsNoTracking()
        .Where(x =>
            x.SemiFinishedId.HasValue &&
            x.StockGrams.HasValue &&
            x.StockGrams.Value > 0m)
        .GroupBy(x => x.SemiFinishedId!.Value)
        .Select(g => new
        {
            SemiFinishedId = g.Key,

```

```

            AvailableStockGrams = g.Sum(x =>
x.StockGrams ?? 0m)
        })
        .ToDictionaryAsync(x => x.SemiFinishedId, x
=> RoundTo2(Math.Max(0m,
x.AvailableStockGrams)));

    var semiFinishedOptions = await
_db.SemiFinisheds
        .AsNoTracking()
        .OrderBy(x => x.Name)
        .ThenBy(x => x.Id)
        .Select(x => new
        {
            x.Id,
            x.Name
        })
        .ToListAsync();

    var semiFinishedDtos = semiFinishedOptions
        .Select(x => new
KitchenWriteOffSemiFinishedOptionDto
        {
            SemiFinishedId = x.Id,
            Name =
string.IsNullOrEmpty(x.Name) ? $"Полуфабрикат
#{x.Id}" : x.Name!,
            UnitOfMeasureId = UnitGramsId,
            UnitName = gramsUnitName,
            AvailableStock =
semiFinishedStockById.TryGetValue(x.Id, out var
stock)
                ? stock
                : 0m
        })
        .ToListAsync();

    var ingredientDtos = await _db.Ingredients
        .AsNoTracking()
        .OrderBy(x => x.Name)
        .ThenBy(x => x.Id)
        .Select(x => new
KitchenWriteOffIngredientOptionDto
        {
            IngredientId = x.Id,
            Name =
string.IsNullOrEmpty(x.Name) ? $"Сырье #{x.Id}"
: x.Name!,
            UnitOfMeasureId = x.UnitOfMeasureId,
            UnitName = x.UnitOfMeasure != null &&
!string.IsNullOrEmpty(x.UnitOfMeasure.Name)
                ? x.UnitOfMeasure.Name
                : string.Empty,
            AvailableStock =
RoundTo2(ToNonNegative(x.Stock))
        })
        .ToListAsync();

    var acts = await _db.WriteOffActs
        .AsNoTracking()
        .Include(x => x.Staff)
        .Include(x => x.IngredientItems)
        .ThenInclude(x => x.Ingredient)
        .Include(x => x.IngredientItems)

```



```

        .ThenInclude(x => x.UnitOfMeasure)
        .Include(x => x.IngredientItems)
        .ThenInclude(x => x.WriteOffType)
        .Include(x => x.SemiFinishedItems)
        .ThenInclude(x => x.SemiFinished)
        .Include(x => x.SemiFinishedItems)
        .ThenInclude(x => x.UnitOfMeasure)
        .Include(x => x.SemiFinishedItems)
        .ThenInclude(x => x.WriteOffType)
        .OrderByDescending(x => x.Date)
        .ThenByDescending(x => x.Id)
        .Take(100)
        .ToListAsync();

return Ok(new KitchenWriteOffBoardResponse
{
    WriteOffTypes = writeOffTypes,
    SemiFinishedOptions = semiFinishedDtos,
    IngredientOptions = ingredientDtos,
    Acts =
acts.Select(BuildWriteOffActDto).ToList()
});
}

[HttpPost("write-off/acts")]
public async
Task<ActionResult<KitchenWriteOffActDto>>
CreateWriteOffAct(
    [FromBody] KitchenCreateWriteOffActRequest
request)
{
    if (request == null)
    {
        return BadRequest("Тело запроса
обязательно.");
    }

    request.IngredientLines ??= new
List<KitchenCreateIngredientWriteOffLineRequest>();
    request.SemiFinishedLines ??= new
List<KitchenCreateSemiFinishedWriteOffLineRequest>(
);

    if (request.IngredientLines.Count == 0 &&
request.SemiFinishedLines.Count == 0)
    {
        return BadRequest("Добавьте хотя бы одну
строку состава акта.");
    }

    var normalizedComment =
NormalizeComment(request.Comment);
    if (normalizedComment != null &&
normalizedComment.Length > 500)
    {
        return BadRequest("Комментарий слишком
длинный. Максимум 500 символов.");
    }

    if (request.IngredientLines.Any(x =>
x.IngredientId <= 0) ||
request.SemiFinishedLines.Any(x =>
x.SemiFinishedId <= 0))
    {

```

```

        return BadRequest("В составе акта есть
некорректная позиция.");
    }

    if (request.IngredientLines.Any(x => x.Quantity
<= 0m) ||
request.SemiFinishedLines.Any(x =>
x.Quantity <= 0m))
    {
        return BadRequest("Количество в каждой
строке должно быть больше нуля.");
    }

    var writeOffTypeIds = request.IngredientLines
.Select(x => x.WriteOffTypeId)
.Concat(request.SemiFinishedLines.Select(x
=> x.WriteOffTypeId))
.Distinct()
.ToList();

    if (writeOffTypeIds.Any(x => x <= 0))
    {
        return BadRequest("В составе акта есть
некорректный тип списания.");
    }

    var existingWriteOffTypeIds = await
_db.WriteOffTypes
.AsNoTracking()
.Where(x => writeOffTypeIds.Contains(x.Id))
.Select(x => x.Id)
.ToListAsync();
    if (existingWriteOffTypeIds.Count !=
writeOffTypeIds.Count)
    {
        return NotFound("Тип списания не
найден.");
    }

    var ingredientIds =
request.IngredientLines.Select(x =>
x.IngredientId).Distinct().ToList();
    var semiFinishedIds =
request.SemiFinishedLines.Select(x =>
x.SemiFinishedId).Distinct().ToList();

    var ingredientsById = ingredientIds.Count == 0
? new Dictionary<int, Ingredient>()
: await _db.Ingredients
.Include(x => x.UnitOfMeasure)
.Where(x => ingredientIds.Contains(x.Id))
.ToDictionaryAsync(x => x.Id);
    if (ingredientsById.Count !=
ingredientIds.Count)
    {
        return NotFound("Сырье из состава акта не
найден.");
    }

    var semiFinishedById = semiFinishedIds.Count
== 0
? new Dictionary<int, SemiFinished>()
: await _db.SemiFinisheds

```

```

        .Where(x =>
semiFinishedIds.Contains(x.Id))
        .ToDictionaryAsync(x => x.Id);
        if (semiFinishedById.Count !=
semiFinishedIds.Count)
        {
            return NotFound("Полуфабрикат из состава
акта не найден.");
        }

        foreach (var group in
request.IngredientLines.GroupBy(x => x.IngredientId))
        {
            var ingredient = ingredientsById[group.Key];
            var availableBase =
ConvertToBaseUnits(ToNonNegative(ingredient.Stock),
ingredient.UnitOfMeasureId);
            var requestedBase = group.Sum(x =>
ConvertToBaseUnits(RoundTo2(x.Quantity),
ingredient.UnitOfMeasureId));
            if (availableBase + DecimalEpsilon <
requestedBase)
            {
                var name =
string.IsNullOrEmpty(ingredient.Name) ? $"Сырье
#{ingredient.Id}" : ingredient.Name;
                return BadRequest($"Недостаточно
остатка сырья \"{name}\" для списания.");
            }
        }

        var preparationsBySemiFinished =
semiFinishedIds.Count == 0
            ? new Dictionary<int, List<Preparation>>()
            : (await _db.Preparations
                .Where(x =>
                    x.SemiFinishedId.HasValue &&
semiFinishedIds.Contains(x.SemiFinishedId.Value) &&
                    x.StockGrams.HasValue &&
                    x.StockGrams.Value > 0m)
                .OrderBy(x => x.ProductionDate ??
DateOnly.MinValue)
                .ThenBy(x => x.Id)
                .ToListAsync())
                .GroupBy(x => x.SemiFinishedId!.Value)
                .ToDictionary(x => x.Key, x => x.ToList());

        foreach (var group in
request.SemiFinishedLines.GroupBy(x =>
x.SemiFinishedId))
        {
            preparationsBySemiFinished.TryGetValue(group.Key,
out var preparationRows);
            var available = (preparationRows ?? new
List<Preparation>()).Sum(x =>
ToNonNegative(x.StockGrams));
            var requested = group.Sum(x =>
RoundTo2(x.Quantity));
            if (available + DecimalEpsilon < requested)
            {
                var semiFinished =
semiFinishedById[group.Key];

```

```

                var name =
string.IsNullOrEmpty(semiFinished.Name) ?
$"Полуфабрикат #{semiFinished.Id}" :
semiFinished.Name;
                return BadRequest($"Недостаточно
остатка полуфабриката \"{name}\" для списания.");
            }
        }

        await using var tx = await
_db.Database.BeginTransactionAsync();

        foreach (var group in
request.IngredientLines.GroupBy(x => x.IngredientId))
        {
            var ingredient = ingredientsById[group.Key];
            var availableBase =
ConvertToBaseUnits(ToNonNegative(ingredient.Stock),
ingredient.UnitOfMeasureId);
            var requestedBase = group.Sum(x =>
ConvertToBaseUnits(RoundTo2(x.Quantity),
ingredient.UnitOfMeasureId));
            var remainingBase = Math.Max(0m,
availableBase - requestedBase);
            ingredient.Stock =
RoundTo2(ConvertFromBaseUnits(remainingBase,
ingredient.UnitOfMeasureId));
        }

        foreach (var group in
request.SemiFinishedLines.GroupBy(x =>
x.SemiFinishedId))
        {
            var remaining = group.Sum(x =>
RoundTo2(x.Quantity));
            foreach (var preparation in
preparationsBySemiFinished[group.Key])
            {
                var stock =
ToNonNegative(preparation.StockGrams);
                if (stock <= DecimalEpsilon)
                {
                    continue;
                }

                var toTake = Math.Min(stock, remaining);
                preparation.StockGrams = RoundTo2(stock
- toTake);
                remaining -= toTake;

                if (remaining <= DecimalEpsilon)
                {
                    break;
                }
            }
        }

        var depletedPreparations =
preparationsBySemiFinished.Values
            .SelectMany(x => x)
            .Where(x => !x.StockGrams.HasValue ||
x.StockGrams.Value <= 0m)
            .ToList();
        if (depletedPreparations.Count > 0)

```

```

    {
        _db.Preparations.RemoveRange(depletedPreparations);
    }

    var act = new WriteOffAct
    {
        Date = (request.Date ?? DateTime.Now).Date,
        Comment = normalizedComment,
        StaffId = TryResolveStaffId()
    };

    foreach (var line in request.IngredientLines)
    {
        var ingredient =
            ingredientsById[line.IngredientId];
        act.IngredientItems.Add(new
            IngredientWriteOffActItem
            {
                IngredientId = line.IngredientId,
                Quantity = RoundTo2(line.Quantity),
                UnitOfMeasureId =
                    ingredient.UnitOfMeasureId,
                WriteOffTypeId = line.WriteOffTypeId
            });
    }

    foreach (var line in request.SemiFinishedLines)
    {
        act.SemiFinishedItems.Add(new
            SemiFinishedWriteOffActItem
            {
                SemiFinishedId = line.SemiFinishedId,
                Quantity = RoundTo2(line.Quantity),
                UnitOfMeasureId = UnitGramsId,
                WriteOffTypeId = line.WriteOffTypeId
            });
    }

    _db.WriteOffActs.Add(act);
    await _db.SaveChangesAsync();
    await
        _stockService.RefreshMenuAvailabilityAsync();
    await tx.CommitAsync();

    var createdAct = await _db.WriteOffActs
        .AsNoTracking()
        .Include(x => x.Staff)
        .Include(x => x.IngredientItems)
        .ThenInclude(x => x.Ingredient)
        .Include(x => x.IngredientItems)
        .ThenInclude(x => x.UnitOfMeasure)
        .Include(x => x.IngredientItems)
        .ThenInclude(x => x.WriteOffType)
        .Include(x => x.SemiFinishedItems)
        .ThenInclude(x => x.SemiFinished)
        .Include(x => x.SemiFinishedItems)
        .ThenInclude(x => x.UnitOfMeasure)
        .Include(x => x.SemiFinishedItems)
        .ThenInclude(x => x.WriteOffType)
        .FirstAsync(x => x.Id == act.Id);

    return Ok(BuildWriteOffActDto(createdAct));
}

```

```

[HttpDelete("write-off/acts/{actId:int}")]
[Authorize(Roles = "Администратор")]
public async Task<ActionResult>
DeleteWriteOffAct(int actId)
{
    if (actId <= 0)
    {
        return BadRequest("Некорректный
идентификатор акта.");
    }

    var act = await _db.WriteOffActs
        .Include(x => x.IngredientItems)
        .ThenInclude(x => x.Ingredient)
        .Include(x => x.SemiFinishedItems)
        .ThenInclude(x => x.SemiFinished)
        .FirstOrDefaultAsync(x => x.Id == actId);

    if (act == null)
    {
        return NotFound("Акт списания не найден.");
    }

    await using var tx = await
        _db.Database.BeginTransactionAsync();

    foreach (var group in
        act.IngredientItems.GroupBy(x => x.IngredientId))
    {
        var firstLine = group.First();
        var ingredient = firstLine.Ingredient;
        if (ingredient == null)
        {
            continue;
        }

        var currentBase =
            ConvertToBaseUnits(ToNonNegative(ingredient.Stock),
                ingredient.UnitOfMeasureId);
        var restoreBase = group.Sum(x =>
            ConvertToBaseUnits(RoundTo2(x.Quantity),
                x.UnitOfMeasureId));
        ingredient.Stock =
            RoundTo2(ConvertFromBaseUnits(currentBase +
                restoreBase, ingredient.UnitOfMeasureId));
    }

    foreach (var group in
        act.SemiFinishedItems.GroupBy(x =>
            x.SemiFinishedId))
    {
        var firstLine = group.First();
        var semiFinished = firstLine.SemiFinished;
        var stockGrams = RoundTo2(group.Sum(x =>
            x.Quantity));
        if (stockGrams <= 0m)
        {
            continue;
        }

        _db.Preparations.Add(new Preparation
        {

```

```

        Name = semiFinished != null &&
!string.IsNullOrEmpty(semiFinished.Name)
        ? semiFinished.Name
        : $"SemiFinished #{group.Key}",
        SemiFinishedId = group.Key,
        StockGrams = stockGrams,
        ProductionDate =
DateOnly.FromDateTime(act.Date.Date)
    });
}

_db.IngredientWriteOffActItems.RemoveRange(act.Ingr
edientItems);

_db.SemiFinishedWriteOffActItems.RemoveRange(act.S
emiFinishedItems);
_db.WriteOffActs.Remove(act);

await _db.SaveChangesAsync();
await
_stockService.RefreshMenuAvailabilityAsync();
await tx.CommitAsync();

return NoContent();
}

private async Task<bool>
MarkItemCompletedAsync(int orderId,
KitchenOrderItemType itemType, int itemId)
{
    switch (itemType)
    {
        case KitchenOrderItemType.Dish:
        {
            var entity = await _db.OrderDishItems
                .FirstOrDefaultAsync(i => i.Id ==
itemId && i.OrderId == orderId);

            if (entity == null)
            {
                return false;
            }

            entity.IsCompleted = true;
            break;
        }
        case KitchenOrderItemType.Drink:
        {
            var entity = await _db.OrderDrinkItems
                .FirstOrDefaultAsync(i => i.Id ==
itemId && i.OrderId == orderId);

            if (entity == null)
            {
                return false;
            }

            entity.IsCompleted = true;
            break;
        }
        case KitchenOrderItemType.Topping:
        {
            var entity = await _db.OrderToppingItems

```

```

                .FirstOrDefaultAsync(i => i.Id ==
itemId && i.OrderId == orderId);

            if (entity == null)
            {
                return false;
            }

            entity.IsCompleted = true;
            break;
        }
        default:
        {
            return false;
        }
    }

    await _db.SaveChangesAsync();
    return true;
}

private async Task
MarkAllOrderItemsCompletedAsync(int orderId)
{
    await _db.OrderDishItems
        .Where(i => i.OrderId == orderId &&
!i.IsCompleted)
        .ExecuteUpdateAsync(setters =>
setters.SetProperty(i => i.IsCompleted, true));

    await _db.OrderDrinkItems
        .Where(i => i.OrderId == orderId &&
!i.IsCompleted)
        .ExecuteUpdateAsync(setters =>
setters.SetProperty(i => i.IsCompleted, true));

    await _db.OrderToppingItems
        .Where(i => i.OrderId == orderId &&
!i.IsCompleted)
        .ExecuteUpdateAsync(setters =>
setters.SetProperty(i => i.IsCompleted, true));
}

private async Task<bool>
TryMarkOrderReadyIfCompletedAsync(int orderId)
{
    var hasPendingDishItems = await
_db.OrderDishItems.AnyAsync(i => i.OrderId ==
orderId && !i.IsCompleted);
    if (hasPendingDishItems)
    {
        return false;
    }

    var hasPendingDrinkItems = await
_db.OrderDrinkItems.AnyAsync(i => i.OrderId ==
orderId && !i.IsCompleted);
    if (hasPendingDrinkItems)
    {
        return false;
    }

    var hasPendingToppingItems = await
_db.OrderToppingItems.AnyAsync(i => i.OrderId ==
orderId && !i.IsCompleted);
    if (hasPendingToppingItems)

```

```

        {
            return false;
        }

        var readyStatus = await
ResolveReadyStatusAsync();
        if (readyStatus == null)
        {
            throw new
InvalidOperationException("Статус готовности заказа
не найден");
        }

        var order = await
_db.Orders.FirstOrDefaultAsync(o => o.Id == orderId);
        if (order == null)
        {
            return false;
        }

        if (order.StatusId == readyStatus.Value.Id)
        {
            return true;
        }

        order.StatusId = readyStatus.Value.Id;
        await _db.SaveChangesAsync();
        return true;
    }

    private async Task<string>
GetOrderStatusNameAsync(int orderId)
    {
        var status = await _db.Orders
            .AsNoTracking()
            .Where(o => o.Id == orderId)
            .Select(o => o.Status != null ? o.Status.Name :
null)
            .FirstOrDefaultAsync();

        return status ?? string.Empty;
    }

    private async Task<(int Id, string Name)?>
ResolveReadyStatusAsync()
    {
        var status = await _db.OrderStatuses
            .AsNoTracking()
            .Where(s => s.Name != null &&
                (EF.Functions.Like(s.Name.ToLower(),
                    $"%{ReadyStatusTokenRu}%") ||
                    EF.Functions.Like(s.Name.ToLower(),
                    $"%{ReadyStatusTokenEn}%")))
            .OrderBy(s => s.Id)
            .Select(s => new { s.Id, s.Name })
            .FirstOrDefaultAsync();

        if (status != null)
        {
            return (status.Id, status.Name ??
ReadyStatusNameRu);
        }

        var fallback = await _db.OrderStatuses
            .AsNoTracking()
            .Where(s => s.Name ==
ReadyStatusNameRu)
            .Select(s => new { s.Id, s.Name })
            .FirstOrDefaultAsync();

        if (fallback != null)
        {
            return (fallback.Id, fallback.Name ??
ReadyStatusNameRu);
        }

        return null;
    }

    private async Task<TechnicalCardSource?>
LoadTechnicalCardSourceAsync(
        int orderId,
        KitchenOrderItemType itemType,
        int itemId)
    {
        switch (itemType)
        {
            case KitchenOrderItemType.Dish:
            {
                return await _db.OrderDishItems
                    .AsNoTracking()
                    .Where(i => i.Id == itemId &&
i.OrderId == orderId)
                    .AsNoTracking()
                    .Where(s => s.Name == ReadyStatusNameRu)
                    .Select(s => new { s.Id, s.Name })
                    .FirstOrDefaultAsync();

                if (fallback != null)
                {
                    return (fallback.Id, fallback.Name ??
ReadyStatusNameRu);
                }

                return null;
            }

            private async Task<(int Id, string Name)?>
ResolveCancelledStatusAsync()
            {
                var status = await _db.OrderStatuses
                    .AsNoTracking()
                    .Where(s => s.Name != null &&
                        (EF.Functions.Like(s.Name.ToLower(),
                            $"%{CancelledStatusTokenRu}%") ||
                            EF.Functions.Like(s.Name.ToLower(),
                            $"%{CancelledStatusTokenEn}%")))
                    .OrderBy(s => s.Id)
                    .Select(s => new { s.Id, s.Name })
                    .FirstOrDefaultAsync();

                if (status != null)
                {
                    return (status.Id, status.Name ??
CancelledStatusNameRu);
                }

                var fallback = await _db.OrderStatuses
                    .AsNoTracking()
                    .Where(s => s.Name ==
CancelledStatusNameRu)
                    .Select(s => new { s.Id, s.Name })
                    .FirstOrDefaultAsync();

                if (fallback != null)
                {
                    return (fallback.Id, fallback.Name ??
CancelledStatusNameRu);
                }

                return null;
            }

            private async Task<TechnicalCardSource?>
LoadTechnicalCardSourceAsync(
                int orderId,
                KitchenOrderItemType itemType,
                int itemId)
            {
                switch (itemType)
                {
                    case KitchenOrderItemType.Dish:
                    {
                        return await _db.OrderDishItems
                            .AsNoTracking()
                            .Where(i => i.Id == itemId &&
i.OrderId == orderId)
                            .AsNoTracking()
                            .Where(s => s.Name == ReadyStatusNameRu)
                            .Select(s => new { s.Id, s.Name })
                            .FirstOrDefaultAsync();

                        if (fallback != null)
                        {
                            return (fallback.Id, fallback.Name ??
CancelledStatusNameRu);
                        }

                        return null;
                    }
                }
            }
        }
    }

```

```

        .Select(i => new TechnicalCardSource
        {
            TechnicalCardId = i.Dish != null ?
i.Dish.TechnicalCardId : null,
            ItemName = i.Dish != null ?
i.Dish.Name : null
        })
        .FirstOrDefaultAsync();
    }
    case KitchenOrderItemType.Drink:
    {
        return await _db.OrderDrinkItems
        .AsNoTracking()
        .Where(i => i.Id == itemId &&
i.OrderId == orderId)
        .Select(i => new TechnicalCardSource
        {
            TechnicalCardId = i.Drink != null ?
i.Drink.TechnicalCardId : null,
            ItemName = i.Drink != null ?
i.Drink.Name : null
        })
        .FirstOrDefaultAsync();
    }
    case KitchenOrderItemType.Topping:
    {
        return await _db.OrderToppingItems
        .AsNoTracking()
        .Where(i => i.Id == itemId &&
i.OrderId == orderId)
        .Select(i => new TechnicalCardSource
        {
            TechnicalCardId = i.Topping != null
? i.Topping.TechnicalCardId : null,
            ItemName = i.Topping != null ?
i.Topping.Name : null
        })
        .FirstOrDefaultAsync();
    }
    default:
        return null;
    }
}

private async
Task<List<KitchenTechnicalCardComponentDto>>
LoadTechnicalCardComponentsAsync(int
technicalCardId)
{
    var ingredientRows = await
_db.TechnicalCardIngredientCompositions
        .AsNoTracking()
        .Where(c => c.TechnicalCardId.HasValue &&
c.TechnicalCardId.Value == technicalCardId)
        .OrderBy(c => c.Id)
        .Select(c => new
        {
            Name = c.Ingredient != null ?
c.Ingredient.Name : null,
            Unit = c.UnitOfMeasure != null ?
c.UnitOfMeasure.Name : null,
            c.OutputWeight,
            c.NetWeight,
            c.GrossWeight

```

```

        })
        .ToListAsync();

    var semiFinishedRows = await
_db.TechnicalCardSemiFinishedCompositions
        .AsNoTracking()
        .Where(c => c.TechnicalCardId.HasValue &&
c.TechnicalCardId.Value == technicalCardId)
        .OrderBy(c => c.Id)
        .Select(c => new
        {
            Name = c.SemiFinished != null ?
c.SemiFinished.Name : null,
            Unit = c.UnitOfMeasure != null ?
c.UnitOfMeasure.Name : null,
            c.OutputWeight,
            c.NetWeight,
            c.GrossWeight
        })
        .ToListAsync();

    var components = new
List<KitchenTechnicalCardComponentDto>(ingredientR
ows.Count + semiFinishedRows.Count);

    components.AddRange(ingredientRows.Select(r
=> new KitchenTechnicalCardComponentDto
    {
        Name = r.Name ?? "Без названия",
        Weight =
PickComponentWeight(r.OutputWeight, r.NetWeight,
r.GrossWeight),
        Unit = r.Unit ?? string.Empty
    }));

    components.AddRange(semiFinishedRows.Select(r =>
new KitchenTechnicalCardComponentDto
    {
        Name = r.Name ?? "Без названия",
        Weight =
PickComponentWeight(r.OutputWeight, r.NetWeight,
r.GrossWeight),
        Unit = r.Unit ?? string.Empty
    }));

    return components;
}

private async
Task<KitchenTechnicalCardEditResponse?>
BuildTechnicalCardEditResponseAsync(int
technicalCardId)
{
    var technicalCard = await _db.TechnicalCards
        .AsNoTracking()
        .Where(c => c.Id == technicalCardId)
        .Select(c => new
        {
            c.Id,
            c.Name,
            c.Description
        })
        .FirstOrDefaultAsync();

```

```

        if (technicalCard == null)
        {
            return null;
        }

        var ingredientLines = await
        _db.TechnicalCardIngredientCompositions
            .AsNoTracking()
            .Where(c => c.TechnicalCardId ==
technicalCardId && c.IngredientId.HasValue)
            .OrderBy(c => c.Id)
            .Select(c => new
KitchenTechnicalCardCompositionLineDto
            {
                ItemId = c.IngredientId!.Value,
                UnitOfMeasureId = c.UnitOfMeasureId,
                GrossWeight = c.GrossWeight,
                ColdLossPercent = c.ColdLossPercent,
                NetWeight = c.NetWeight,
                HotLossPercent = c.HotLossPercent,
                OutputWeight = c.OutputWeight
            })
            .ToListAsync();

        var semiFinishedLines = await
        _db.TechnicalCardSemiFinishedCompositions
            .AsNoTracking()
            .Where(c => c.TechnicalCardId ==
technicalCardId && c.SemiFinishedId.HasValue)
            .OrderBy(c => c.Id)
            .Select(c => new
KitchenTechnicalCardCompositionLineDto
            {
                ItemId = c.SemiFinishedId!.Value,
                UnitOfMeasureId = c.UnitOfMeasureId,
                GrossWeight = c.GrossWeight,
                ColdLossPercent = c.ColdLossPercent,
                NetWeight = c.NetWeight,
                HotLossPercent = c.HotLossPercent,
                OutputWeight = c.OutputWeight
            })
            .ToListAsync();

        var bindings = new
List<KitchenTechnicalCardBindingDto>();
        bindings.AddRange(await _db.Dishes
            .AsNoTracking()
            .Where(x => x.TechnicalCardId ==
technicalCardId)
            .Select(x => new
KitchenTechnicalCardBindingDto
            {
                Kind =
KitchenTechnicalCardBindingKinds.Dish,
                ItemId = x.Id
            })
            .ToListAsync());
        bindings.AddRange(await _db.Drinks
            .AsNoTracking()
            .Where(x => x.TechnicalCardId ==
technicalCardId)
            .Select(x => new
KitchenTechnicalCardBindingDto

```

```

            {
                Kind =
KitchenTechnicalCardBindingKinds.Drink,
                ItemId = x.Id
            })
            .ToListAsync());
        bindings.AddRange(await
        _db.ToppingsAndSyrups
            .AsNoTracking()
            .Where(x => x.TechnicalCardId ==
technicalCardId)
            .Select(x => new
KitchenTechnicalCardBindingDto
            {
                Kind =
KitchenTechnicalCardBindingKinds.Topping,
                ItemId = x.Id
            })
            .ToListAsync());
        bindings.AddRange(await _db.SemiFinisheds
            .AsNoTracking()
            .Where(x => x.TechnicalCardId ==
technicalCardId)
            .Select(x => new
KitchenTechnicalCardBindingDto
            {
                Kind =
KitchenTechnicalCardBindingKinds.SemiFinished,
                ItemId = x.Id
            })
            .ToListAsync());

        return new KitchenTechnicalCardEditResponse
        {
            TechnicalCardId = technicalCard.Id,
            CardName = technicalCard.Name ??
string.Empty,
            Description = technicalCard.Description ??
string.Empty,
            IngredientLines = ingredientLines,
            SemiFinishedLines = semiFinishedLines,
            Bindings = bindings
        };
    }

    private async Task<string?>
ValidateTechnicalCardReferencesAsync(KitchenTechnic
alCardUpsertRequest request)
    {
        var ingredientIds =
request.IngredientLines.Select(x =>
x.ItemId).Distinct().ToList();
        var semiFinishedCompositionIds =
request.SemiFinishedLines.Select(x =>
x.ItemId).Distinct().ToList();
        var unitIds = request.IngredientLines
            .Concat(request.SemiFinishedLines)
            .Where(x => x.UnitOfMeasureId.HasValue)
            .Select(x => x.UnitOfMeasureId!.Value)
            .Distinct()
            .ToList();

        if (ingredientIds.Count > 0)
        {

```

```

        var existingCount = await
_db.Ingredients.CountAsync(x =>
ingredientIds.Contains(x.Id));
        if (existingCount != ingredientIds.Count)
        {
            return "В составе указаны
несуществующие ингредиенты.";
        }

        if (semiFinishedCompositionIds.Count > 0)
        {
            var existingCount = await
_db.SemiFinisheds.CountAsync(x =>
semiFinishedCompositionIds.Contains(x.Id));
            if (existingCount !=
semiFinishedCompositionIds.Count)
            {
                return "В составе указаны
несуществующие полуфабрикаты.";
            }

            if (unitIds.Count > 0)
            {
                var existingCount = await
_db.UnitsOfMeasures.CountAsync(x =>
unitIds.Contains(x.Id));
                if (existingCount != unitIds.Count)
                {
                    return "В составе указаны
несуществующие единицы измерения.";
                }
            }

            return null;
        }

private async Task
ReplaceTechnicalCardDetailsAsync(
int technicalCardId,
KitchenTechnicalCardUpsertRequest request)
{
    var oldIngredientRows = await
_db.TechnicalCardIngredientCompositions
.Where(x => x.TechnicalCardId ==
technicalCardId)
.ToListAsync();
    var oldSemiFinishedRows = await
_db.TechnicalCardSemiFinishedCompositions
.Where(x => x.TechnicalCardId ==
technicalCardId)
.ToListAsync();

_db.TechnicalCardIngredientCompositions.RemoveRange(
oldIngredientRows);

_db.TechnicalCardSemiFinishedCompositions.RemoveRange(
oldSemiFinishedRows);

_db.TechnicalCardIngredientCompositions.AddRange(re

```

```

quest.IngredientLines.Select(line => new
TechnicalCardIngredientComposition
{
    TechnicalCardId = technicalCardId,
    IngredientId = line.ItemId,
    UnitOfMeasureId = line.UnitOfMeasureId,
    GrossWeight = line.GrossWeight,
    ColdLossPercent = line.ColdLossPercent,
    NetWeight = line.NetWeight,
    HotLossPercent = line.HotLossPercent,
    OutputWeight = line.OutputWeight
}));

_db.TechnicalCardSemiFinishedCompositions.AddRange(
request.SemiFinishedLines.Select(line => new
TechnicalCardSemiFinishedComposition
{
    TechnicalCardId = technicalCardId,
    SemiFinishedId = line.ItemId,
    UnitOfMeasureId = line.UnitOfMeasureId,
    GrossWeight = line.GrossWeight,
    ColdLossPercent = line.ColdLossPercent,
    NetWeight = line.NetWeight,
    HotLossPercent = line.HotLossPercent,
    OutputWeight = line.OutputWeight
}));

await _db.SaveChangesAsync();
}

private async Task<List<string>>
LoadTechnicalCardLinkedNamesAsync(int
technicalCardId)
{
    var names = new List<string>();
    names.AddRange(await _db.Dishes
.AsNoTracking()
.Where(x => x.TechnicalCardId ==
technicalCardId)
.Select(x => "блюдо: " + (x.Name ??
$"#{x.Id}"))
.ToListAsync());
    names.AddRange(await _db.Drinks
.AsNoTracking()
.Where(x => x.TechnicalCardId ==
technicalCardId)
.Select(x => "напиток: " + (x.Name ??
$"#{x.Id}"))
.ToListAsync());
    names.AddRange(await _db.ToppingsAndSyrups
.AsNoTracking()
.Where(x => x.TechnicalCardId ==
technicalCardId)
.Select(x => "добавка: " + (x.Name ??
$"#{x.Id}"))
.ToListAsync());
    names.AddRange(await _db.SemiFinisheds
.AsNoTracking()
.Where(x => x.TechnicalCardId ==
technicalCardId)
.Select(x => "полуфабрикат: " + (x.Name ??
$"#{x.Id}"))
.ToListAsync());
}

```



```

        return names;
    }

    private static string?
    ValidateTechnicalCardRequest(KitchenTechnicalCardU
    psertRequest request)
    {
        if (request == null)
        {
            return "Не переданы данные техкарты.";
        }

        if
        (string.IsNullOrEmpty(request.CardName))
        {
            return "Укажите название техкарты.";
        }

        request.IngredientLines ??= new
        List<KitchenTechnicalCardCompositionLineDto>();
        request.SemiFinishedLines ??= new
        List<KitchenTechnicalCardCompositionLineDto>();
        request.Bindings ??= new
        List<KitchenTechnicalCardBindingDto>();

        if (request.IngredientLines.Any(x => x.ItemId <=
        0) ||
            request.SemiFinishedLines.Any(x => x.ItemId
        <= 0))
        {
            return "В составе есть строки без выбранной
        позиции.";
        }

        return null;
    }

    private static string? NormalizeDescription(string?
    description)
    {
        return string.IsNullOrEmpty(description) ?
        null : description.Trim();
    }

    private async
    Task<KitchenTechnicalCardResponse?>
    BuildTechnicalCardResponseAsync(int technicalCardId,
    string? fallbackCardName)
    {
        var technicalCard = await _db.TechnicalCards
        .AsNoTracking()
        .Where(c => c.Id == technicalCardId)
        .Select(c => new
        {
            c.Id,
            c.Name,
            c.Description
        })
        .FirstOrDefaultAsync();

        if (technicalCard == null)
        {
            return null;
        }
    }

```

```

    }

    var components = await
    LoadTechnicalCardComponentsAsync(technicalCardId);

    return new KitchenTechnicalCardResponse
    {
        TechnicalCardId = technicalCard.Id,
        CardName =
        string.IsNullOrEmpty(technicalCard.Name)
        ? (fallbackCardName ?? $"Technical card
        #{technicalCard.Id}")
        : technicalCard.Name,
        Description = technicalCard.Description ??
        string.Empty,
        Components = components
    };
}

    private static decimal
    PickComponentWeight(decimal? outputWeight,
    decimal? netWeight, decimal? grossWeight)
    {
        if (outputWeight.HasValue &&
        outputWeight.Value > 0)
        {
            return outputWeight.Value;
        }

        if (netWeight.HasValue && netWeight.Value >
        0)
        {
            return netWeight.Value;
        }

        if (grossWeight.HasValue &&
        grossWeight.Value > 0)
        {
            return grossWeight.Value;
        }

        return 0m;
    }

    private static KitchenStopListPositionDto
    BuildStopListPosition(
        string itemType,
        int itemId,
        string? itemName,
        string? categoryName,
        string? volumeWeight,
        bool isAvailable,
        int? technicalCardId,
        IReadOnlyDictionary<string, decimal>
        limitsByKey,
        IReadOnlyDictionary<int, decimal?>
        autoPortionsByTechnicalCard)
    {
        var itemTypeToken =
        itemType?.Trim().ToLowerInvariant() ?? string.Empty;
        var key =
        BuildMenuItemLimitKey(itemTypeToken, itemId);
        var manualRemainingPortions =
        limitsByKey.TryGetValue(key, out var manualValue)
    }

```

```

        ? manualValue
        : (decimal?)null;

        decimal? autoAvailablePortions = null;
        if (technicalCardId.HasValue &&
            autoPortionsByTechnicalCard.TryGetValue(technicalCardId.Value, out var autoValue))
        {
            autoAvailablePortions = autoValue;
        }

        var effectiveRemainingPortions =
            ResolveEffectiveRemainingPortions(
                manualRemainingPortions,
                autoAvailablePortions);

        return new KitchenStopListPositionDto
        {
            ItemType = itemTypeToken,
            ItemId = itemId,
            Name =
                string.IsNullOrEmpty(itemName)
                    ? $"Позиция #{itemId}"
                    : itemName,
            Category = categoryName ?? string.Empty,
            VolumeWeight = volumeWeight ??
                string.Empty,
            IsAvailable = isAvailable,
            ManualRemainingPortions =
                manualRemainingPortions,
            AutoAvailablePortions =
                autoAvailablePortions,
            EffectiveRemainingPortions =
                effectiveRemainingPortions
        };
    }

    private bool
    IsStopListPositionVisibleForCurrentRole(KitchenStopListPositionDto position)
    {
        if (IsCurrentUserInRole(AdminRole))
        {
            return true;
        }

        var itemType =
            position.ItemType?.Trim().ToLowerInvariant() ??
            string.Empty;
        if (IsCurrentUserInRole(CookRole))
        {
            return itemType == KitchenItemTypes.Dish ||
                (itemType == KitchenItemTypes.Topping
                &&
                    CategoryContainsToken(position.Category,
                        DishToppingCategoryToken));
        }

        if (IsCurrentUserInRole(BaristaRole))
        {
            return itemType == KitchenItemTypes.Drink ||
                (itemType == KitchenItemTypes.Topping
                &&
                    CategoryContainsToken(position.Category,
                        DrinkToppingCategoryToken));
        }

        return false;
    }

    private async Task<bool>
    CanCurrentRoleManageStopListItemAsync(KitchenOrderItemType itemType, int itemId)
    {
        if (IsCurrentUserInRole(AdminRole))
        {
            return true;
        }

        if (IsCurrentUserInRole(CookRole))
        {
            return itemType switch
            {
                KitchenOrderItemType.Dish => true,
                KitchenOrderItemType.Topping => await
                    ToppingCategoryContainsTokenAsync(itemId,
                        DishToppingCategoryToken),
                _ => false
            };
        }

        if (IsCurrentUserInRole(BaristaRole))
        {
            return itemType switch
            {
                KitchenOrderItemType.Drink => true,
                KitchenOrderItemType.Topping => await
                    ToppingCategoryContainsTokenAsync(itemId,
                        DrinkToppingCategoryToken),
                _ => false
            };
        }

        return false;
    }

    private async Task<bool>
    ToppingCategoryContainsTokenAsync(int toppingId,
        string token)
    {
        var categoryName = await
            _db.ToppingsAndSyrups
                .AsNoTracking()
                .Where(t => t.Id == toppingId)
                .Select(t => t.Category != null ?
                    t.Category.Name : null)
                .FirstOrDefaultAsync();

        return CategoryContainsToken(categoryName,
            token);
    }

    private static bool CategoryContainsToken(string?
        categoryName, string token)
    {
        return (categoryName ?? string.Empty)
            .Contains(token, StringComparison.OrdinalIgnoreCase);
    }

```

```

        CategoryContainsToken(position.Category,
            DrinkToppingCategoryToken));
    }

    return false;
}

private async Task<bool>
CanCurrentRoleManageStopListItemAsync(KitchenOrderItemType itemType, int itemId)
{
    if (IsCurrentUserInRole(AdminRole))
    {
        return true;
    }

    if (IsCurrentUserInRole(CookRole))
    {
        return itemType switch
        {
            KitchenOrderItemType.Dish => true,
            KitchenOrderItemType.Topping => await
                ToppingCategoryContainsTokenAsync(itemId,
                    DishToppingCategoryToken),
            _ => false
        };
    }

    if (IsCurrentUserInRole(BaristaRole))
    {
        return itemType switch
        {
            KitchenOrderItemType.Drink => true,
            KitchenOrderItemType.Topping => await
                ToppingCategoryContainsTokenAsync(itemId,
                    DrinkToppingCategoryToken),
            _ => false
        };
    }

    return false;
}

private async Task<bool>
ToppingCategoryContainsTokenAsync(int toppingId,
    string token)
{
    var categoryName = await
        _db.ToppingsAndSyrups
            .AsNoTracking()
            .Where(t => t.Id == toppingId)
            .Select(t => t.Category != null ?
                t.Category.Name : null)
            .FirstOrDefaultAsync();

    return CategoryContainsToken(categoryName,
        token);
}

private static bool CategoryContainsToken(string?
    categoryName, string token)
{
    return (categoryName ?? string.Empty)
        .Contains(token, StringComparison.OrdinalIgnoreCase);
}

```

```

.Trim()
.ToLowerInvariant()
.Contains(token);
}

private async Task<Dictionary<int, decimal>>
BuildOutputWeightByTechnicalCardAsync(ICollection<int> technicalCardIds)
{
    if (technicalCardIds.Count == 0)
    {
        return new Dictionary<int, decimal>();
    }

    var ingredientRows = await
_db.TechnicalCardIngredientCompositions
.AsNoTracking()
.Where(x => x.TechnicalCardId.HasValue &&
technicalCardIds.Contains(x.TechnicalCardId.Value))
.Select(x => new
{
    TechnicalCardId =
x.TechnicalCardId!.Value,
    x.OutputWeight,
    x.NetWeight,
    x.GrossWeight
})
.ToListAsync();

    var semiFinishedRows = await
_db.TechnicalCardSemiFinishedCompositions
.AsNoTracking()
.Where(x => x.TechnicalCardId.HasValue &&
technicalCardIds.Contains(x.TechnicalCardId.Value))
.Select(x => new
{
    TechnicalCardId =
x.TechnicalCardId!.Value,
    x.OutputWeight,
    x.NetWeight,
    x.GrossWeight
})
.ToListAsync();

    return ingredientRows
.Select(x => new
{
    x.TechnicalCardId,
    Weight =
PickComponentWeight(x.OutputWeight, x.NetWeight,
x.GrossWeight)
})
.Concat(semiFinishedRows.Select(x => new
{
    x.TechnicalCardId,
    Weight =
PickComponentWeight(x.OutputWeight, x.NetWeight,
x.GrossWeight)
}))
.GroupBy(x => x.TechnicalCardId)
.ToDictionary(x => x.Key, x =>
RoundTo2(x.Sum(y => y.Weight)));
}

```

```

private static string
BuildTechnicalCardVolumeWeight(
    int? technicalCardId,
    IReadOnlyDictionary<int, decimal>
outputByTechnicalCard,
    string? unitName)
{
    if (!technicalCardId.HasValue ||
!outputByTechnicalCard.TryGetValue(technicalCardId.
Value, out var value) ||
    value <= 0m)
    {
        return string.Empty;
    }

    return BuildVolumeWeight(value, unitName);
}

private static string BuildVolumeWeight(decimal?
quantity, string? unitName)
{
    if (!quantity.HasValue || quantity.Value <= 0m)
    {
        return string.Empty;
    }

    var unit = ToShortUnitName(unitName);
    return string.IsNullOrEmpty(unit)
        ? quantity.Value.ToString("0.##",
CultureInfo.CurrentCulture)
        : $"{quantity.Value.ToString("0.##",
CultureInfo.CurrentCulture)} {unit}";
}

private static string ToShortUnitName(string?
unitName)
{
    var normalized = (unitName ??
string.Empty).Trim().ToLowerInvariant();
    return normalized switch
    {
        "килограмм" or "килограмма" or
"килограммы" or "кг." => "кг",
        "грамм" or "грамма" or "граммы" or "гр" or
"гр." or "г." => "г",
        "литр" or "литра" or "литры" or "л." => "л",
        "миллилитр" or "миллилитра" or
"миллилитры" or "мл." => "мл",
        "штука" or "штуки" or "штук" or "шт." =>
"шт",
        _ => (unitName ?? string.Empty).Trim()
    };
}

private static decimal?
ResolveEffectiveRemainingPortions(decimal?
manualRemainingPortions, decimal?
autoAvailablePortions)
{
    if (manualRemainingPortions.HasValue &&
autoAvailablePortions.HasValue)
    {

```

```

        return
        Math.Min(manualRemainingPortions.Value,
        autoAvailablePortions.Value);
    }

    return manualRemainingPortions ??
    autoAvailablePortions;
}

private async Task<Dictionary<int, decimal?>>
BuildAutoAvailablePortionsByTechnicalCardAsync(
    IReadOnlyCollection<int> technicalCardIds)
{
    if (technicalCardIds.Count == 0)
    {
        return new Dictionary<int, decimal?>();
    }

    var requirementRows = await
    _db.TechnicalCardSemiFinishedCompositions
        .AsNoTracking()
        .Where(x =>
            x.TechnicalCardId.HasValue &&
            x.SemiFinishedId.HasValue &&

technicalCardIds.Contains(x.TechnicalCardId.Value))
        .Select(x => new
        {
            TechnicalCardId =
            x.TechnicalCardId!.Value,
            SemiFinishedId = x.SemiFinishedId!.Value,
            RequiredBase = ConvertToBaseUnits(
                PickComponentWeight(x.OutputWeight,
                x.NetWeight, x.GrossWeight),
                x.UnitOfMeasureId)
        })
        .ToListAsync();

    var requirementsByCard = requirementRows
        .Where(x => x.RequiredBase >
        DecimalEpsilon)
        .GroupBy(x => new { x.TechnicalCardId,
        x.SemiFinishedId })
        .Select(g => new
        {
            g.Key.TechnicalCardId,
            g.Key.SemiFinishedId,
            RequiredBase = g.Sum(x =>
            x.RequiredBase)
        })
        .GroupBy(x => x.TechnicalCardId)
        .ToDictionary(
            g => g.Key,
            g => g.Select(x => new
            CardSemiFinishedRequirement(
                x.SemiFinishedId,
                x.RequiredBase)).ToList());

    var semiFinishedIds =
    requirementsByCard.Values
        .SelectMany(x => x)
        .Select(x => x.SemiFinishedId)
        .Distinct()
        .ToList();

```

```

        var stockBySemiFinished =
        semiFinishedIds.Count == 0
        ? new Dictionary<int, decimal>()
        : await _db.Preparations
            .AsNoTracking()
            .Where(x =>
                x.SemiFinishedId.HasValue &&
                x.StockGrams.HasValue &&
                x.StockGrams.Value > 0m &&

semiFinishedIds.Contains(x.SemiFinishedId.Value))
            .GroupBy(x => x.SemiFinishedId!.Value)
            .Select(g => new
            {
                SemiFinishedId = g.Key,
                Available = g.Sum(x => x.StockGrams ??
                0m)
            })
            .ToDictionaryAsync(x => x.SemiFinishedId,
            x => Math.Max(0m, x.Available));

    var result = technicalCardIds
        .Distinct()
        .ToDictionary(x => x, _ => (decimal?)null);

    foreach (var technicalCardId in technicalCardIds)
    {
        if
        (!requirementsByCard.TryGetValue(technicalCardId, out
        var requirements) || requirements.Count == 0)
        {
            result[technicalCardId] = null;
            continue;
        }

        decimal? minAvailablePortions = null;

        foreach (var requirement in requirements)
        {
            if (requirement.RequiredBase <=
            DecimalEpsilon)
            {
                continue;
            }

            stockBySemiFinished.TryGetValue(requirement.SemiFi
            nishedId, out var stockBase);
            var availablePortions =
            Math.Floor(stockBase / requirement.RequiredBase);
            if (availablePortions < 0m)
            {
                availablePortions = 0m;
            }

            minAvailablePortions =
            !minAvailablePortions.HasValue
            ? availablePortions
            : Math.Min(minAvailablePortions.Value,
            availablePortions);
        }
    }

```

```

        result[technicalCardId] =
minAvailablePortions.HasValue
    ? Math.Max(0m,
minAvailablePortions.Value)
    : null;
    }

    return result;
}

private async Task
UpsertMenuItemPortionLimitAsync(
    string itemTypeToken,
    int itemId,
    decimal remainingPortions,
    DateTime now)
{
    var row = await _db.MenuItemPortionLimits
        .FirstOrDefaultAsync(x =>
            x.ItemType == itemTypeToken &&
            x.ItemId == itemId);

    if (row == null)
    {
        _db.MenuItemPortionLimits.Add(new
MenuItemPortionLimit
        {
            ItemType = itemTypeToken,
            ItemId = itemId,
            RemainingPortions = remainingPortions,
            CreatedAt = now,
            UpdatedAt = now
        });
        return;
    }

    row.RemainingPortions = remainingPortions;
    row.UpdatedAt = now;
}

private static string BuildMenuItemLimitKey(string
itemType, int itemId)
{
    return
    $"{itemType.Trim().ToLowerInvariant()}.{itemId}";
}

private static decimal ConvertToBaseUnits(decimal
value, int? unitOfMeasureId)
{
    if (value <= 0m)
    {
        return 0m;
    }

    return unitOfMeasureId switch
    {
        UnitKilogramsId => value * 1000m,
        UnitLitersId => value * 1000m,
        UnitGramsId => value,
        UnitMillilitersId => value,
        UnitPiecesId => value,
        _ => value
    };
};

```

```

    }

    private static decimal
ConvertFromBaseUnits(decimal value, int?
unitOfMeasureId)
    {
        if (value <= 0m)
        {
            return 0m;
        }

        return unitOfMeasureId switch
        {
            UnitKilogramsId => value / 1000m,
            UnitLitersId => value / 1000m,
            UnitGramsId => value,
            UnitMillilitersId => value,
            UnitPiecesId => value,
            _ => value
        };
    }

    private static decimal RoundTo2(decimal value)
    {
        return Math.Round(value, 2,
MidpointRounding.AwayFromZero);
    }

    private static decimal ToNonNegative(decimal?
value)
    {
        return value.HasValue && value.Value > 0m
            ? value.Value
            : 0m;
    }

    private static KitchenWriteOffActDto
BuildWriteOffActDto(WriteOffAct act)
    {
        return new KitchenWriteOffActDto
        {
            ActId = act.Id,
            Date = act.Date,
            Comment = act.Comment ?? string.Empty,
            StaffId = act.StaffId,
            StaffFullName = act.Staff != null &&
!string.IsNullOrEmpty(act.Staff.FullName)
                ? act.Staff.FullName!
                : string.Empty,
            IngredientLines = act.IngredientItems
                .OrderBy(x => x.Id)
                .Select(x => new
KitchenWriteOffActLineDto
                {
                    LineId = x.Id,
                    ItemId = x.IngredientId,
                    ItemName =
string.IsNullOrEmpty(x.Ingredient?.Name)
                        ? $"Сырье #{x.IngredientId}"
                        : x.Ingredient.Name!,
                    Quantity = RoundTo2(x.Quantity),
                    UnitOfMeasureId = x.UnitOfMeasureId,
                    UnitName = x.UnitOfMeasure?.Name ??
string.Empty,

```

```

        WriteOffTypeId = x.WriteOffTypeId,
        WriteOffTypeName =
x.WriteOffType?.Name ?? string.Empty
    })
    .ToList(),
    SemiFinishedLines = act.SemiFinishedItems
    .OrderBy(x => x.Id)
    .Select(x => new
KitchenWriteOffActLineDto
    {
        LineId = x.Id,
        ItemId = x.SemiFinishedId,
        ItemName =
string.IsNullOrEmpty(x.SemiFinished?.Name)
            ? $"Полуфабрикат
#{x.SemiFinishedId}"
            : x.SemiFinished.Name!,
        Quantity = RoundTo2(x.Quantity),
        UnitOfMeasureId = x.UnitOfMeasureId,
        UnitName = x.UnitOfMeasure?.Name ??
string.Empty,
        WriteOffTypeId = x.WriteOffTypeId,
        WriteOffTypeName =
x.WriteOffType?.Name ?? string.Empty
    })
    .ToList()
    };
}

private int? TryResolveStaffId()
{
    var raw =
User.FindFirstValue(ClaimTypes.NameIdentifier);
    return int.TryParse(raw, out var staffId) &&
staffId > 0
        ? staffId
        : (int?)null;
}

private async Task<bool>
SetMenuItemAvailabilityAsync(
    KitchenOrderItemType itemType,
    int itemId,
    bool isAvailable,
    bool saveChanges = true)
{
    switch (itemType)
    {
        case KitchenOrderItemType.Dish:
        {
            var dish = await
_db.Dishes.FirstOrDefaultAsync(x => x.Id == itemId);
            if (dish == null)
            {
                return false;
            }

            dish.IsAvailable = isAvailable;
            break;
        }
        case KitchenOrderItemType.Drink:
        {
            var drink = await
_db.Drinks.FirstOrDefaultAsync(x => x.Id == itemId);

```

```

            if (drink == null)
            {
                return false;
            }

            drink.IsAvailable = isAvailable;
            break;
        }
    }
    case KitchenOrderItemType.Topping:
    {
        var topping = await
_db.ToppingsAndSyrups.FirstOrDefaultAsync(x => x.Id
== itemId);
        if (topping == null)
        {
            return false;
        }

        topping.IsAvailable = isAvailable;
        break;
    }
    default:
        return false;
}

if (saveChanges)
{
    await _db.SaveChangesAsync();
}

return true;
}

private static bool TryParseItemType(string
itemTypeToken, out KitchenOrderItemType itemType)
{
    var normalized =
itemTypeToken?.Trim().ToLowerInvariant() ??
string.Empty;

    switch (normalized)
    {
        case KitchenItemTypes.Dish:
        case "блюдо":
        case "блюда":
            itemType = KitchenOrderItemType.Dish;
            return true;
        case KitchenItemTypes.Drink:
        case "напиток":
        case "напитки":
            itemType = KitchenOrderItemType.Drink;
            return true;
        case KitchenItemTypes.Topping:
        case "добавка":
        case "добавки":
            itemType =
KitchenOrderItemType.Topping;
            return true;
        default:
            itemType = KitchenOrderItemType.Dish;
            return false;
    }
}

```

```

private static string
ToItemTypeToken(KitchenOrderItemType itemType)
{
    return itemType switch
    {
        KitchenOrderItemType.Dish =>
KitchenItemTypes.Dish,
        KitchenOrderItemType.Drink =>
KitchenItemTypes.Drink,
        KitchenOrderItemType.Topping =>
KitchenItemTypes.Topping,
        _ => string.Empty
    };
}

private static string?
NormalizeKitchenOrdersStatus(string? status)
{
    var normalized =
status?.Trim().ToLowerInvariant() ?? string.Empty;
    if (string.IsNullOrEmpty(normalized) ||
        normalized == KitchenOrdersStatusActive ||
        normalized.Contains(ActiveStatusTokenRu) ||
        normalized.Contains(ActiveStatusTokenEn))
    {
        return KitchenOrdersStatusActive;
    }

    if (normalized == KitchenOrdersStatusReady ||
        normalized.Contains(ReadyStatusTokenRu) ||
        normalized.Contains(ReadyStatusTokenEn))
    {
        return KitchenOrdersStatusReady;
    }

    return null;
}

private static bool
IsWithinPickupWindow(DateTime pickupAt, DateTime
now, TimeSpan pickupWindow)
{
    var delta = pickupAt - now;
    if (delta < TimeSpan.Zero)
    {
        delta = delta.Negate();
    }

    return delta <= pickupWindow;
}

private static bool
IsSystemRecommendationComment(string? comment)
{
    return string.Equals(comment,
SystemRecommendationComment,
StringComparison.Ordinal) ||

IsSnoozedSystemRecommendationComment(comment);
}

private static bool
IsSnoozedSystemRecommendationComment(string?
comment)

```

```

{
    return
TryParseSnoozedSystemRecommendationUntil(commen
t, out _);
}

private static bool
TryParseSnoozedSystemRecommendationUntil(string?
comment, out DateOnly snoozedUntilDate)
{
    snoozedUntilDate = default;

    if (string.IsNullOrEmpty(comment))
    {
        return false;
    }

    if
(!comment.StartsWith(SystemRecommendationSnoozeP
refix, StringComparison.Ordinal))
    {
        return false;
    }

    var rawDate =
comment.Substring(SystemRecommendationSnoozePrefi
x.Length).Trim();
    return DateOnly.TryParseExact(
        rawDate,
        "yyyy-MM-dd",
        CultureInfo.InvariantCulture,
        DateTimeStyles.None,
        out snoozedUntilDate);
}

private static string
BuildSnoozedSystemRecommendationComment(DateO
nly snoozedUntilDate)
{
    return
$"{{SystemRecommendationSnoozePrefix}}{snoozedUnti
lDate:yyyy-MM-dd}";
}

private static string? NormalizeComment(string?
comment)
{
    if (string.IsNullOrEmpty(comment))
    {
        return null;
    }

    return comment.Trim();
}

private bool IsCurrentUserInRole(string role)
{
    return User.IsInRole(role);
}

private static string? NormalizeTaskText(string?
taskText)
{
    if (string.IsNullOrEmpty(taskText))

```

```

    {
        return null;
    }

    return taskText.Trim();
}

private enum KitchenOrderItemType
{
    Dish,
    Drink,
    Topping
}

private sealed class DishSource
{
    public int ItemId { get; set; }
    public int OrderId { get; set; }
    public string? Name { get; set; }
    public decimal Quantity { get; set; }
}

private sealed class DishToppingSource
{
    public int OrderDishItemId { get; set; }
    public string? Name { get; set; }
    public decimal Quantity { get; set; }
}

private sealed class DrinkSource
{
    public int ItemId { get; set; }

```

```

    public int OrderId { get; set; }
    public string? Name { get; set; }
    public decimal Quantity { get; set; }
}

private sealed class DrinkToppingSource
{
    public int OrderDrinkItemId { get; set; }
    public string? Name { get; set; }
    public decimal Quantity { get; set; }
}

private sealed class TechnicalCardSource
{
    public int? TechnicalCardId { get; set; }
    public string? ItemName { get; set; }
}

private sealed class CardSemiFinishedRequirement
{
    public CardSemiFinishedRequirement(int
semiFinishedId, decimal requiredBase)
    {
        SemiFinishedId = semiFinishedId;
        RequiredBase = requiredBase;
    }

    public int SemiFinishedId { get; }
    public decimal RequiredBase { get; }
}
}

```


Приложение Д

Фрагменты листинга программного кода серверной части, веб-клиента и WPF-приложения с пояснением назначения основных классов, методов и моделей обмена.

Приложение Е

Акт о внедрении или справка об апробации программного комплекса при наличии может быть размещена в данном приложении.